

קריפטוגרפיה וורילוג

שיעור 1

הבדלים בין VHDL לVerilog

```

1 // This is a Verilog 1995 counter
2
3
4 module cntr
5 (
6     clk,
7     reset,
8     q,
9     tc
10 );
11
12 input    clk;
13 input    reset;
14 output [3:0] q;
15 output    tc;
16
17 reg [3:0] q;
18 wire    tc;
19
20 always @ (posedge clk)
21     if (reset)
22         q <= 4'b0;
23     else
24         q <= q + 1;
25
26 assign tc = (q == 4'hf) ? 1'b1 : 1'b0;
27
28 endmodule
29

```

```

1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use IEEE.std_logic_unsigned.all;
4
5 entity CNTR is
6 port
7 (
8     CLK: in std_logic;
9     RESET: in std_logic;
10    Q: out std_logic_vector(3 downto 0);
11    TC: out std_logic
12 );
13 end CNTR;
14
15 architecture CNTR_ARC of CNTR is
16     signal Q_I: std_logic_vector(3 downto 0);
17 begin
18
19     P1: process
20     begin
21         wait until rising_edge(CLK);
22         if (RESET = '1') then
23             Q_I <= "0000";
24         else
25             Q_I <= Q_I + 1;
26         end if;
27     end process;
28
29     P2: TC <= '1' when (Q_I = "1111") else
30         '0';
31

```

```

11
12 input    clk;
13 input    reset;
14 output [3:0] q;
15 output    tc;
16
17 reg [3:0] q;
18 wire    tc;
19
20 always @ (posedge clk)

```

מוגדר במבנה process (always או initial)

reg - יוצא מalways

```

17 reg [3:0] q;
18 wire    tc;
19
20 always @ (posedge clk)
21     if (reset)
22         q <= 4'b0;
23     else
24         q <= q + 1;
25
26 assign tc = (q == 4'hf) ? 1'b1 : 1'b0;
27
28 endmodule

```

מוגדר אך ורק מחוץ ל process

wire - סיגנל שלא מוצב בprocess.

- בהצבה מחוץ לprocess יש להשתמש בalways.
- בVerilog 2001 כל סיגנל שמוגדר ככניסה ויציאה, באופן אוטומטי מוגדר כwire.


```

27 // Connect DUT to test bench
28 cntr DUT
29 (
30 .clk    (tb_clk  ),
31 .reset  (tb_reset),
32 .q      (tb_q    ),
33 .tc     (tb_tc   )
34 );

```

by name

פה אין ENTITY וארכיטקטורה, אלא דבר אחד שהוא module.

ה module מחליף את ENTITY.

always מחליף את process.

דוגמא למוקס:

```

////////////////////////////////////
// mux2_1.v
//
// 2:1 multiplexer
////////////////////////////////////
module mux2_1 (y, in0, in1, sel);
    output y;
    input in0, in1, sel;
    reg y;

    always @(in0 or in1 or sel)
    begin
        if (!sel)
            y = in0;
        else
            y = in1;
        end
    endmodule
////////////////////////////////////

```

סיגנלים שמוצבים בתוך always או initial (או PROCESS אחר) צריכים להגדיר אותם בתור REG. כאלה שמוצבים בשורה נפרדת, צריכים להגדיר אותם בתור WIRE. ובשורה עצמה להשתמש ב assign.

הזמנים פה הם ב PS. אם רוצים לקבוע משהו אחר, אז מגדירים בהרצה במודלסים:
vsim -t ns tb_cnrt

או דרך:

```
timescale 1ns/1ps
```

שזה מגדיר לי גם את הרזולוציה בנקודה העשרונית.

כמו \$finish – מסיים את הסימולציה ויוצא מהסימולטר.
יש גם \$stop – עוצר את הסימולציה, וניתן להמשיך אח"כ אם רוצים. לא יוצא מהסימולטר.

```
// verilog testbench, first example
////////////////////////////////////
module tb_cntr
(
);

reg      tb_clk, tb_reset;
wire [3:0] tb_q ;
wire      tb_tc ;

initial begin
    $display ("Hello world.....");
    $display ("time\t\t\t clk reset q tc");
    $monitor ("%8g\t\t\t %b  %b      %h  %b",
              $time, tb_clk, tb_reset, tb_q, tb_tc);
    tb_clk      = 1;
    tb_reset    = 1;
    #700        tb_reset = 0;
    #5000        $finish;
end

always begin
    #100        tb_clk = ~tb_clk;
end

// Connect DUT to test bench
cntr DUT
(
```

השרשור בין TB לרכיב:

```
5 end
6
7 // Connect DUT to test bench
8 cntr DUT
9 (
10 .clk      (tb_clk ),
11 .reset    (tb_reset),
12 .q        (tb_q   ),
13 .tc       (tb_tc  )
14 );
```

לא צריך begin end אם יש רק פקודה אחת בחלק הזה.

שיעור 2

How Does ModelSim Give Back the Signals Visibility? (Though maybe running slowly...)

Older versions

- vcom -novopt design.vhd
- vcom -novopt tb_design.vhd
- vsim -novopt tb_design

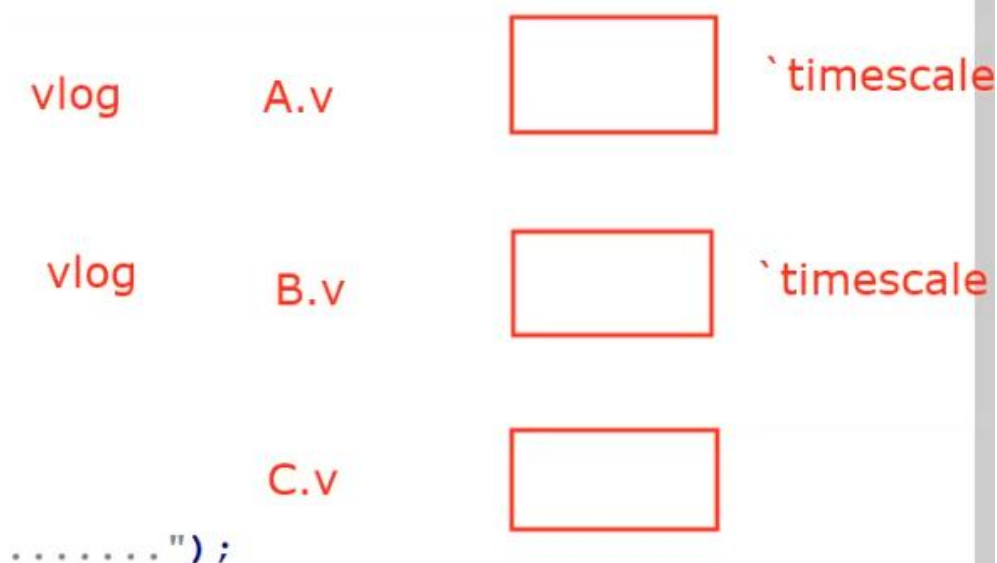
Newer versions (10.7 and later)

- vcom design.vhd
- vcom tb_design.vhd
- vsim -voptargs=+acc=1prn tb_design

שנוי בין גרסאות.

``timescale 1ns/1ps`

משמאל לקו הנתוי – יחידות הזמן
מימין לקו הנתוי – רזולוציה אחרי הנקודה, עד איזה ערך נראה אחרי הנקודה.



C יקבל את timescale האחרון, כלומר של B.

vlog C.v
vlog A.v
vlog B.v

C יקבל timescale אחר, ברירת מחדל של המנוע.

במקרה הזה



במקרה הזה
C יקבל timescale כמו A.
כשלא כתוב, יקבל את timescale הקודם.

אם יש
timescale
בוורילוג

או שיש לך בכל הקבצים
!או שאין באף קובץ

ע"מ להתגבר על בעיה זו אמרו כלל:

בוורילוג
לא שמים
timescale
בקבצי תכנון
אלא רק
ב
testbench

שהרי בתכנון לא רושמים השהיות

יש כלל קצת סותר:
אם יש כמה קבצים יש לשים בכולם, ומנגד, בקבצי תכנון לא לשים כלל.

פרקטיקה
קובץ
timescale

פתרון מומלץ: יחדי, שאותו מקמפלים ראשון
על כל הקבצים.

אותיות גדולות או קטנות:

```
);
reg [3:0] q_i ;
always @ (posedge clk)
  if (reset)
    q_i <= 4'b0;
  else
    q_i <= q_i + 1;
assign q = Q_I;
assign tc = (q_i == 4'hf) ? 1'b1 : 1'b0;
endmodule
```

זה יתן שגיאה כי השינוי הוא בצד ימין וסיגנל זה לא מוגדר.

```
0 assign KUKU = q_i + 1;
```

מצד ימין זו אינה טעות

מצד ימין של המשוואה חובה להגדיר את הסיגנל.

מצד שמאל של המשוואה אין חובה, ואם לא נגדיר נקבל כברירת מחדל שאותו סיגנל מוצהר
wires עם ביט 1. כפי שניתן לראות:

```
assign KUKU = q_i + 1;
```

[illegible]

ניתן לראות שקיבלנו סיגנל בינארי ולא וקטורי משום שלא הגדרנו את KUKU כלל.

הודעת שגיאה לא קיבלנו למרות שלא הגדרנו (וזו קצת חסרון כי נקבל סיגנל שלא הגדרנו ובvhdל נקבל שגיאה כח לא הגדרנו).

vhdl – לא הגדרת – אין. "אמא יודעת מה הכי טוב עבור הילד".

verilog – אתה המתכנן, אתה יודע הכי טוב, ואם כתבת, זה מה שיש. "תאכל את מה שבישלת".

כ"כ, vhdI – וקטור שמקבל מוקטור צריכים להיות באותו אורך.

verilog – אין חובה.

- לשם כך, כדי למנוע טעויות בverilog ישנה פקודה שדורשת להגדיר את הסיגנלים באופן

```
`default_nettype none
```

מפורש. ויש להשתמש בפקודה

זאת תמיד כדי למנוע טעויות.

```
);  
  
reg      [3:0] q_i    ;  
  
always @ (posedge clk)  
    if (reset)  
        q_i <= 4'b0;  
    else  
        q_i <= q_i + 1;  
  
wire     [7:0] KUKU;  
assign KUKU = q_i + 1;
```

לא קיבלנו שגיאה וניתן לראות שבverilog ניתן לבצע את ההצהרה לפני השימוש בפקודה ואין צורך במקום של הצהרות כמו ב.vhdl.

מה יותר טוב? הגמישות של Verilog או הנוקשות של vhdl?

לכל אחד יתרונות וחסרונות, לגמישות יש יתרון שלניתן לתת הצהרות עבור אזורים שונים בקוד.

XMR •

tb

```
assign xxx = DUT.KUKU;
```

מההיררכיה אחת ניתן לגשת

לסיגנלים מארכיה אחרת.

בvhdl יכולנו לגשת לסיגנלים פנימיים מההיררכיה אחרת?

אך ורק ע"י הסימולציה.

• Mux בשני השפות:

```
architecture BEHAVIORAL_ARC of MUX2_1 is
begin

  A_LABEL: process(SEL, IN0, IN1)
  _begin
    if (SEL = '0') then
      Y <= IN0;
    else
      Y <= IN1;
    end if;
  end process;

end BEHAVIORAL_ARC;
```

vhdl

```
1 // mux2_1 : By an always statement ("Behavioral")
2 module mux2_1
3 (
4   input  in0, in1, sel,
5   output y
6 );
7
8   reg yi;
9
10  //////////////////////////////////////
11  // Just for a note...
12  // Verilog 1995 wrote such things assign
13  // always @(in0 or in1 or sel)
14  //////////////////////////////////////
15  always @(in0, in1, sel)
16  if (!sel)
17    yi = in0;
18  else
19    yi = in1;
20
21  assign y = yi;
22
23  endmodule
```

verilog

• שיפור של Verilog:

////////////////////////////////////
always @(*)

אין צורך לרשום את כל הסיגנלים, אפשר להתייחס לכולם.

• מבנה האטום:

<pre>1 module and2 2 (3 output y, 4 input a, b 5); 6 7 assign y = a & b; 8 9 endmodule</pre>	<pre>1 module inv1 2 (3 output y, 4 input a 5); 6 7 assign y = ~a; 8 9 endmodule 10</pre>	<pre>1 module or2 2 (3 output y, 4 input a, b 5); 6 7 assign y = a b; 8 9 endmodule</pre>
---	--	--

```
// mux2_1 : By hierarchy
module mux2_1
(
  input  in0, in1, sel,
  output y
);

  wire inv_sel, x0, x1;

  inv1 U0 (inv_sel, sel);
  and2 U1 (x0, in0, inv_sel);
  and2 U2 (x1, in1, sel);
  or2  U3 (y, x0, x1);

endmodule
```

היציאה היא הראשונה ואח"כ הכניסות.

אולם, ב verilog יש מילים שמורות עבור השערים הפשוטים ועבור עוד כמה, ואין צורך ליצור מחדש קומפוננטה. מה שאין ב vhdل.

שפת ורילוג היא Case sensitive, כלומר רגישה לאותיות גדולות או קטנות, בניגוד ל VHDL ששם לא מקובל שמשתנים וסיגנלים יהיו בקטנות, וקבועים (בהמשך) בגדולות.

אם טעיתי והכנסתי איפשהו משתנה שלא הצהרתי עליו, נניח שכתבתי באותיות גדולות במקום קטנות, אז ורילוג יוצר לי אחד כזה, אבל בתור wire בודד. כלומר זה מתכון לבעיות!

פתרון –

```
`default_nettype none
```

זה אומר לכלי להיות קשוח ולא לאפשר סיגנל ללא הצהרה.

כדי לגשת ב TB לסיגנלים פנימיים ברכיב הנבדק, נכתוב DUT.KUKU למשל. (XMR).

מרבב בורילוג:


```

1 // mux2_1 : Continuous statement ("dataflow")
2 module mux2_1
3 (
4   input  in0, in1, sel,
5   output y
6 );
7   assign y = (in0 & ~sel) | (sel & in1);
8
9 endmodule

```

~ --> not, bitwise
! --> not, logical
& (one time) --> bitwise operation
&& (twice) --> logical operation
if ((a == 1) && (b == 2))

במקום להכניס רשימת רגישויות, בגרסת 2001 אפשר פשוט לשים:

always@(*)

אפשרות נוספת למררב:

```

1 // mux2_1 : Conditional assignment ("dataflow")
2 module mux2_1
3 (
4   input  in0, in1, sel,
5   output y
6 );
7   assign y = (!sel)? in0 : in1;
8
9 endmodule

```

תכנון הירארכי: בורילוג אין צורך להגדיר קוד ארוך בשביל תכנון הירארכי. פשוט כותבים:

```

1 // mux2_1 : By hierarchy
2
3 module mux2_1
4 (
5     input  in0, in1, sel,
6     output y
7 );
8
9     wire inv_sel, x0, x1;
10
11     inv1 U0 (inv_sel, sel);
12     and2 U1 (x0, in0, inv_sel);
13     and2 U2 (x1, in1, sel);
14     or2  U3 (y, x0, x1);
15
16 endmodule

```

לוגיקה קומבינטורית:

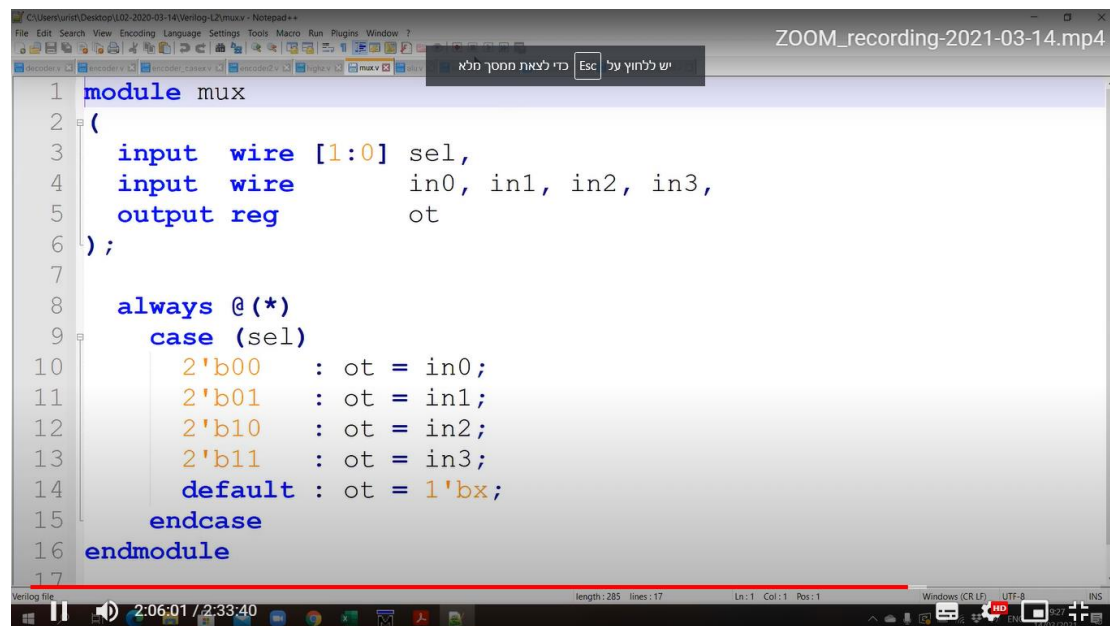
מימוש של מפלג:

```

1 module decoder
2 (
3     input [1:0] adrs,
4     output      sel0, sel1, sel2, sel3
5 );
6
7     assign sel0 = (adrs == 2'b00) ? 1'b1 : 1'b0;
8     assign sel1 = (adrs == 2'b01) ? 1'b1 : 1'b0;
9     assign sel2 = (adrs == 2'b10) ? 1'b1 : 1'b0;
10    assign sel3 = (adrs == 2'b11) ? 1'b1 : 1'b0;
11
12 endmodule
13

```

עוד אפשרות:

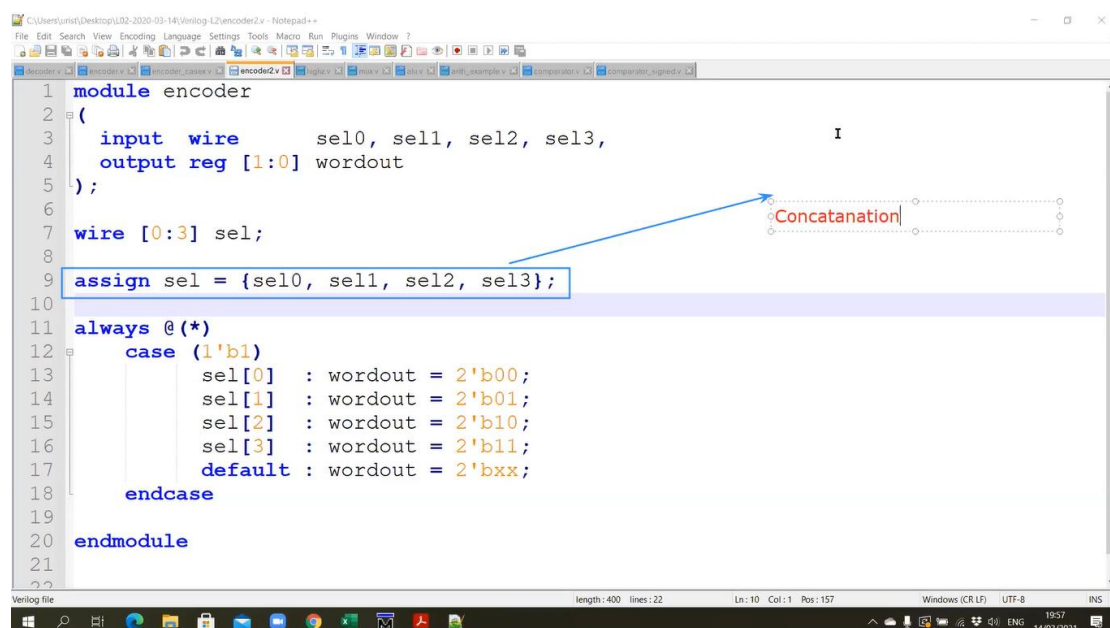


```

1 module mux
2 (
3     input wire [1:0] sel,
4     input wire      in0, in1, in2, in3,
5     output reg      ot
6 );
7
8 always @(*)
9     case (sel)
10        2'b00 : ot = in0;
11        2'b01 : ot = in1;
12        2'b10 : ot = in2;
13        2'b11 : ot = in3;
14        default : ot = 1'bx;
15    endcase
16 endmodule
17

```

שרשור, ו case:



```

1 module encoder
2 (
3     input wire      sel0, sel1, sel2, sel3,
4     output reg [1:0] wordout
5 );
6
7 wire [0:3] sel;
8
9 assign sel = {sel0, sel1, sel2, sel3};
10
11 always @(*)
12     case (1'b1)
13        sel[0] : wordout = 2'b00;
14        sel[1] : wordout = 2'b01;
15        sel[2] : wordout = 2'b10;
16        sel[3] : wordout = 2'b11;
17        default : wordout = 2'bxx;
18    endcase
19
20 endmodule
21

```

אפשרות נוספת לקיים:

```

1 module enc (
2     input wire [7:0] a,
3     output reg [2:0] y
4 );
5
6 always @* begin
7     casex (a)
8         8'b1xxxxxxx : y = 3'd7;
9         8'b01xxxxxx : y = 3'd6;
10        8'b001xxxxx : y = 3'd5;
11        8'b0001xxxx : y = 3'd4;
12        8'b00001xxx : y = 3'd3;
13        8'b000001xx : y = 3'd2;
14        8'b0000001x : y = 3'd1;
15        8'b00000001 : y = 3'd0;
16        default      : y = 3'd0;
17    endcase
18 end
19 endmodule

```

שיעור 3

דרך לכתיבת encoder:

```

1 module encoder
2 (
3     input wire    sel0, sel1, sel2, sel3,
4     output reg [1:0] wordout
5 );
6
7 wire [0:3] sel;
8
9 assign sel = {sel0, sel1, sel2, sel3};
10
11 always @(*)
12     case (1'b1)
13         sel[0] : wordout = 2'b00;
14         sel[1] : wordout = 2'b01;
15         sel[2] : wordout = 2'b10;
16         sel[3] : wordout = 2'b11;
17         default : wordout = 2'bxx;
18     endcase
19
20 endmodule

```

דרך נוספת:

```

module enc (
    input wire [7:0] a,
    output reg [2:0] y
);

always @* begin
    casex (a)
        8'b1xxxxxxx : y = 3'd7;
        8'b01xxxxxx : y = 3'd6;
        8'b001xxxxx : y = 3'd5;
        8'b0001xxxx : y = 3'd4;
        8'b00001xxx : y = 3'd3;
        8'b000001xx : y = 3'd2;
        8'b0000001x : y = 3'd1;
        8'b00000001 : y = 3'd0;
        default      : y = 3'd0;
    endcase
end
endmodule

```

:compartor

```

1 module comparator
2 (
3     input [3:0] a, b,
4     output      eq,
5     output      gt,
6     output      lt
7 );
8
9 wire signed [3:0] as, bs;
10 assign as = a;
11 assign bs = b;
12
13 // For example:
14 // 4'sb1001 is signed
15 // 4'b1001  is not signed
16
17 assign eq = (as == bs) ? 1'b1 : 1'b0 ;
18 assign gt = (as > bs) ? 1'b1 : 1'b0 ;
19 assign lt = (as < bs) ? 1'b1 : 1'b0 ;
20
21 endmodule
22
23

```

יש לשים לב לsigned או לnot signed.

מחשוב:

```

1 module alu
2 (
3     input  [3:0] a,
4     input  [3:0] b,
5     input  [1:0] sel,
6     output [3:0] y
7 );
8
9 reg [3:0] y_i;
10
11 always @(*)
12     case(sel)
13         2'b00 : y_i = a + b;
14         2'b01 : y_i = a - b;
15         2'b10 : y_i = ~a;
16         2'b11 : y_i = a | b;
17         default : y_i = 4'bx;
18     endcase
19
20 assign y = y_i;
21
22 endmodule
23

```

בשורה המסומנת מדובר ב־or.
עבור פעולה לוגית או לא נפעל כך:

Operation Logic operation		Bitwise operation
OR		
AND	&&	&
NOT	!	~

(לוגי – True/Fals)

if (a == 7) || ((a > 19) && (a < 21) && !(a == 20))

עבור בוליאני

```

y = a & b;
Y = a | b;
I

```

עבור קומבינטורי:

עבור xor:

```

1 //-----
2 // Design Name : parity_using_assign
3 // File Name : parity_using_assign.v
4 // Function : Parity using assign
5 // Coder : Deepak Kumar Tala
6 //-----
7 module parity_using_assign (
8 data_in , // 8 bit data in
9 parity_out // 1 bit parity out
10 );
11 output parity_out ;
12 input [7:0] data_in ;
13
14 wire parity_out ;
15
16 assign parity_out = (data_in[0] ^ data_in[1]) ^
17 (data_in[2] ^ data_in[3]) ^
18 (data_in[4] ^ data_in[5]) ^
19 (data_in[6] ^ data_in[7]);
20
21 endmodule

```

You could download file parity_using_assign.v [here](#)

בכתיבה מקוצרת:

```

1 //-----
2 // Design Name : parity_using_bitwise
3 // File Name : parity_using_bitwise.v
4 // Function : Parity using bitwise xor
5 // Coder : Deepak Kumar Tala
6 //-----
7 module parity_using_bitwise (
8 data_in , // 8 bit data in
9 parity_out // 1 bit parity out
10 );
11 output parity_out ;
12 input [7:0] data_in ;
13
14 assign parity_out = ^data_in;
15
16 endmodule

```

You could download file parity_using_bitwise.v [here](#)

גם בפקודות אחרות:

```

assign y = &data_in;

==> assign y = data_in[0] &
data_in[1] & data_in[2]
וכו'

assign y |data_in;

```


:Hight-z

```
module highz
(
  input  en0, en1,
  input  a0 , a1 ,
  output y
);

assign y = (en0) ? a0 : 1'bz;
assign y = (en1) ? a1 : 1'bz;

endmodule
```

```
# Loading work.tb_highz
VSIM 14> run 800 ns
# time    a0 a1 en0 en1
#         0 0 0 0 0 z
#         100 0 0 0 1 0
#         200 0 0 1 0 0
#         300 0 0 1 1 0
#         400 0 1 0 0 z
#         500 0 1 0 1 1
#         600 0 1 1 0 0
#         700 0 1 1 1 z
VSIM 15> run 800 ns
#         800 1 0 0 0 z
#         900 1 0 0 1 0
#        1000 1 0 1 0 1
#        1100 1 0 1 1 x
#        1200 1 1 0 0 z
#        1300 1 1 0 1 1
#        1400 1 1 1 0 1
#        1500 1 1 1 1 1
```

כאשר ישנה העברה לץ משתי הכניסות ויש שני האיניבילים (a) מנוגדים, נקבל x.
 כאשר אין מעבר של שני האיניבילים (a) נקבל z.
 כאשר יש מעבר של איניביל ומעבר של z, האיניביל גובר.

בכתיבה אחרת, יש טעות:

```

1 module highz
2 (
3   input  en0, en1,
4   input  a0 , a1 ,
5   output reg y
6 );
7
8 // This is WRONG: there is no strength in "reg"
9 // People who come from VHDL do this kind of mistake
10 always @(*)
11   y = (en0) ? a0 : 1'bz;
12
13 always @(*)
14   y = (en1) ? a1 : 1'bz;
15
16 endmodule
17

```

```

VSIM 18> run 800 ns
# time    a0 a1 en0 en1 y
#         0 0 0 0 0 z
#        100 0 0 0 1 0
#        200 0 0 1 0 z
#        300 0 0 1 1 0
#        400 0 1 0 0 z
#        500 0 1 0 1 1
#        600 0 1 1 0 z
#        700 0 1 1 1 1
VSIM 19> run 800 ns
#         800 1 0 0 0 z
#         900 1 0 0 1 0
#        1000 1 0 1 0 z
#        1100 1 0 1 1 0
#        1200 1 1 0 0 z
#        1300 1 1 0 1 1
#        1400 1 1 1 0 z
#        1500 1 1 1 1 1

```



לדוגמה במצב הזה (איפה שהקו הצהוב), צריך לצאת x.
כאן יש עניין של חוזק של סיגנל.

חוזק של סיגנל קשור דווקא ל-wire.

אין חוזק של סיגנל ב-reg.

וגם לא נדע מי מה-processים ינצח.

בvhdl ראינו את הסכנה שכמה process-ים ידחפו את אותו סיגנל, וכאן גם נוסף שלפי כתיבה זו לא נדע מי ינצח.

• מתי נשתמש ב = ומתי ב <= ?

- **Blocking assignment:** evaluation and assignment are immediate

```
always @ (a or b or c)
begin
  x = a | b;           1. Evaluate a | b, assign result to x
  y = a ^ b ^ c;       2. Evaluate a^b^c, assign result to y
  z = b & ~c;           3. Evaluate b&(~c), assign result to z
end
```

- **Nonblocking assignment:** all assignments deferred until all right-hand sides have been evaluated (end of simulation timestep)

```
always @ (a or b or c)
begin
  x <= a | b;           1. Evaluate a | b but defer assignment of x
  y <= a ^ b ^ c;       2. Evaluate a^b^c but defer assignment of y
  z <= b & ~c;           3. Evaluate b&(~c) but defer assignment of z
end                      4. Assign x, y, and z with their new values
```

ב = - הפעולה נעשת במקום.

ב <= - הפעולה נעשת אך ההצבה נעשת בסוף.

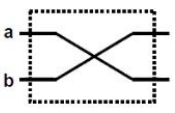
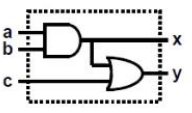
ישנם מקרים בהם נקבל סתירה בתוצאות.

=> non-blocking - נשתמש כשיש פליפלופים.

=> blocking - נשתמש כשיש לוגיקה קומבינטורית.

Why two ways of assigning values?

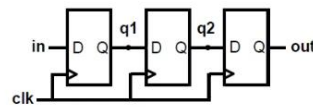
Conceptual need for two kinds of assignment (in always blocks):

		
Blocking: Evaluation and assignment are immediate	$a = b$ $b = a$	$x = a \& b$ $y = x \mid c$
Non-Blocking: Assignment is postponed until all r.h.s. evaluations are done	$a <= b$ $b <= a$	$x <= a \& b$ $y <= x \mid c$
When to use: (only in always blocks!)	Sequential Circuits	Combinational Circuits

Use Nonblocking for Sequential Logic

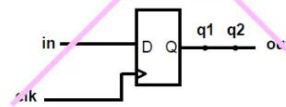
```
always @ (posedge clk)
begin
    q1 <= in;
    q2 <= q1;
    out <= q2;
end
```

"At each rising clock edge, q1, q2, and out **simultaneously receive the old values** of in, q1, and q2."



```
always @ (posedge clk)
begin
    q1 = in;
    q2 = q1;
    out = q2;
end
```

"At each rising clock edge, q1 = in. After that, q2 = q1 = in; After that, out = q2 = q1 = in; Finally out = in."



- Blocking assignments do not reflect the intrinsic behavior of multi-stage sequential logic
- **Guideline: use nonblocking assignments for sequential always blocks**

6.111 Fall 2007

Lecture 6, Slide 4

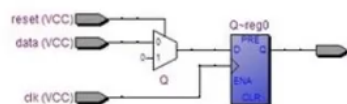
סינכרוני: Reset

Synchronous Reset

dff_sync_reset.v

```
1 module dff_sync_reset(
2     clk,
3     reset,
4     data,
5     Q
6 );
7
8 //input
9 input clk, reset, data;
10
11 //output
12 output reg Q;
13
14 //synchronous
15 always @(posedge clk) begin
16     if(reset) begin
17         Q <= 0;
18     end
19     else begin
20         Q <= data;
21     end
22 end
23
24 endmodule
```

For synchronous reset, synthesis tool models the reset as another input, selecting either data signal or zero using a mux based on the reset signal.



סינכרוני: Reset

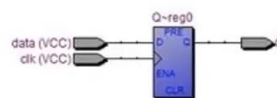
dff_async_reset.v

```

1 module dff_async_reset(
2   clk,
3   reset,
4   data,
5   Q
6 );
7
8 //input
9 input clk, reset, data;
10
11 //output
12 output reg Q;
13
14 always @(posedge clk, posedge reset) begin
15   if(reset) begin
16     Q <= 0;
17   end
18   else begin
19     Q <= data;
20   end
21 end
22
23 endmodule

```

FPGA provides asynchronous reset and set options for every register(D flip-flop). Hence, asynchronous resets are implemented by synthesis tools using this reset pin.



מכונת מצבים:

```

8
9 parameter WIDTH = 2;
10 parameter IDLE = 2'b00;
11 parameter COIN1 = 2'b01;
12 parameter COIN2 = 2'b10;
13
קבועים

```

קביעת פרמטרים מבחוץ:

```

1
2 // Note for the parameter change "by order"
3 // The change by name would be:
4 // SM #(.WIDTH(3), .IDLE(3'b001), .COIN1(3'b010), .COIN2(3'b100)) DUT
5 SM #(3, 3'b001, 3'b010, 3'b100) DUT
6 (
7   .clk      (clk      ),
8   .reset    (reset    ),
9   .coin      (coin      ),
10  .coffee   (coffee   )
11 );
12
13 endmodule

```

מכיון שניתן לדרוס פרמטרים מבחוץ נגדיר בlocalparam:

```

8
9 localparam WIDTH = 2;
10 localparam IDLE = 2'b00;
11 localparam COIN1 = 2'b01;
12 localparam COIN2 = 2'b10;

```

נשתמש בדוגמא של בדיקת זוגיות כדי להסביר כמה אפשרויות:

1. פונקציות:

```

15
16 function parity;
17 input [31:0] data;
18 begin
19     parity = (data_in[0] ^ data_in[1]) ^
20              (data_in[2] ^ data_in[3]) ^
21              (data_in[4] ^ data_in[5]) ^
22              (data_in[6] ^ data_in[7]);
23 end
24 endfunction
25
26
27 assign parity_out = parity(data_in);

```

2. לולאה:

```

15 function parity;
16 input [31:0] data;
17 integer i;
18 begin
19     parity = 0;
20     for (i = 0; i < 32; i = i + 1) begin
21         parity = parity ^ data[i];
22     end
23 end
24 endfunction
25

```

3. פירמידה:

```

13
14 assign parity_out = ^data_in;
15

```

דרך זו היא היעילה ביותר. חוסכת הרבה שורות קוד. אפשרי לבצע אותו גם עם AND/OR.

מספרים מסומנים (משלים ל2):

ב VHDL היינו מגדירים בספריה האם המספרים מסומנים או לא. אבל בורילוג ראינו שלא צריך. אז מה נעשה אם רוצים לומר לו?

נכתוב:

```
wire signed [3:0] as, bs;
```

גם מה העניין של רשימת רגישויות?

:Tree-state

יציאה שמחוברת ל2 סיגנלים מקוצרים, שתלויים בenable שלהם. אם שניהם מופעלים, והם שונים, אז ביציאה יהיה X ואילו אשר שניהם סגורים יהיה Z. הנקודה היא שבשונה ב VHDL, כאן הקידוד אסור שיעשה ע"י פרוססים. כשניסינו ראינו שהתוצאות לא יוצאות נכון (וגם אין שם X). הסיבה נעוצה בהבדלים בין WIRE ל REG, כי STRENGTH (של טרי-סטייט) עובד רק עם WIRE. כמו כן, בורילוג בפרוסס אין סדר לפעולות, כלומר אם יהיה 2 פרוססים אין חוקיות מה יפעל קודם, וזה יגרום לבעיות. לכן מותר להגדיר TB את CLK ההתחלתי שווה 1 בתוך פרוסס INITIAL, מה שאסור ב VHDL.

הביטוי שווה:

מתי משתמשים ב ($=$) ומתי ב ($<=$)?

הראשון נקרא blocking, והשני nonblocking.

בראשון זה קורה מיד, בצורה הירארכית, לכן בלוגיקה קומבינטורית נשתמש בו, אבל כאשר יש לנו FF נצטרך את השני.

[ממילא מחוץ ל FF נשתמש רק ב ($=$) ?]

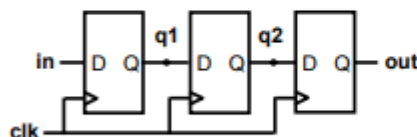
דוגמאות:

Blocking: Evaluation and assignment are immediate	 $a = b$ $b = a$ 	$x = a \& b$ $y = x \mid c$
Non-Blocking: Assignment is postponed until all r.h.s. evaluations are done	$a <= b$ $b <= a$	 $x <= a \& b$ $y <= x \mid c$
When to use: (only in always blocks!)	Sequential Circuits	Combinational Circuits

Use Nonblocking for Sequential Logic

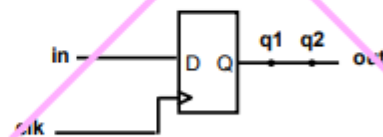
```
always @ (posedge clk)
begin
  q1 <= in;
  q2 <= q1;
  out <= q2;
end
```

"At each rising clock edge, q1, q2, and out simultaneously receive the old values of in, q1, and q2."



```
always @ (posedge clk)
begin
  q1 = in;
  q2 = q1;
  out = q2;
end
```

"At each rising clock edge, q1 = in. After that, q2 = q1 = in; After that, out = q2 = in; Finally out = in."



- Blocking assignments do not reflect the intrinsic behavior of multi-stage sequential logic
- **Guideline: use nonblocking assignments for sequential always blocks**

מכונת מצבים:

כדי להגדיר תכנון בתור א-סינכרוני, נכניס גם את RST לרשימת רגישויות.

כדי להגדיר לתכנון האם ה RST הוא פועל ב 1 או 0, נכתוב ברשימת רגישויות posedge או negedge

בהתאמה. ואז בתכנון עצמו נכתוב את התנאי בהתאם כמובן.
אבל אם נעשה סלט, כלומר ברשימת רגישויות נכניס שלילי ובתנאי חיובי, יהיה בלאגן.

כשמגדירים משתת מצב חדש, לכל מצב גם צריך להגדיר את הקידוד לו מבחינת ביטים, בשונה מ
.VHDL

למרות שהם קבועים, ניתן לשנות אותם בTB (אלא אם כן הם לוקאלפרם):

```
// Note for the parameter change "by order"
// The change by name would be:
SM #(.WIDTH(3), .IDLE(3'b001), .COIN1(3'b010), .COIN2(3'b100)) DUT
//SM #(3, 3'b001, 3'b010, 3'b100) DUT
(
.clk      (clk      ),
.reset    (reset    ),
.coin     (coin     )
```

חשוב להקפיד על הסדר של המיקום שלהם.

שיעור 4

<pre> 1 module rom 2 (3 input [1:0] adrs, 4 output reg [7:0] data 5); 6 7 always @(*) 8 case (adrs) 9 2'd0: data = 8'haa; 10 2'd1: data = 8'h55; 11 2'd2: data = 8'h99; 12 2'd3: data = 8'h00; 13 endcase 14 15 endmodule 16 17 </pre>	<pre> module rom (input [1:0] adrs, output reg [7:0] data); always @(*) case (adrs) 0: data = 8'haa; 1: data = 8'h55; 2: data = 8'h99; 3: data = 8'h00; endcase endmodule </pre>
---	---

מערך דו-ממדי:

```

1 module rom
2 (
3     input  [1:0]  adrs,
4     output [7:0]  data
5 );
6
7 reg [7:0] rom_array [0:3];
8
9 initial
10 begin
11     rom_array[0] = 8'haa;
12     rom_array[1] = 8'h55;
13     rom_array[2] = 8'h99;
14     rom_array[3] = 8'h00;
15 end
16
17 assign data = rom_array[adrs];
18
19 endmodule
20
21

```

המימד הראשון נותן את גודל הוקטורים במערך- טורים
המימד השני נותן את הגודל של המערך - שורות
verilog הממדים באים בהתחלה, אך במערך באים בסוף.

אותו מערך אך עם קריאה מקובץ:

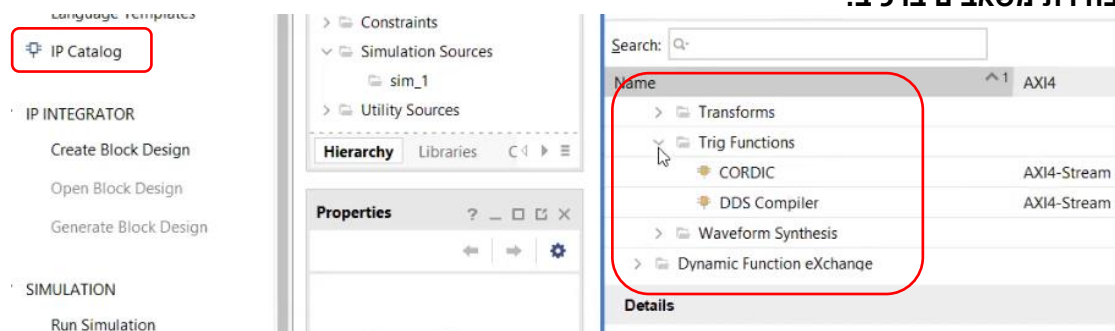
```

1 module rom
2 (
3     input  [1:0]  adrs,
4     output [7:0]  data
5 );
6
7 reg [7:0] rom_array [0:3];
8
9 initial
10     $readmemh("rom.txt", rom_array, 0, 3);
11
12 assign data = rom_array[adrs];
13
14 endmodule
15
16

```

:Vivado

בחירת משאבים ברכיב:



Customize IP

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☒ Show disabled ports

Component Name: blk_mem_gen_0

Basic Port A Options Other Options Summary

Pipeline Stages within Mux: 0 Mux Size: 1x1

Memory Initialization

☒ Load Init File

Coe File: no_coe_file_loaded Browse Edit

☐ Fill Remaining Memory Locations

Remaining Memory Locations (Hex): 0

Structural/UniSim Simulation Model Options

Defines the type of warnings and outputs are generated when a read-write or write-write collision occurs.

Collision Warnings: All

Behavioral Simulation Model Options

☐ Disable Collision Warnings ☐ Disable Out of Range Warnings

OK Cancel

Customize IP

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

Save Location For COE File

Save In: ROM

project_1
project_2
work

Recent Directories

C:/Users/urist/Desktop

File Preview

Select a file to preview.

File name: KUKU

Files of type: COE Files (*.coe)

COE File Editor - KUKU.coe

Key	Value
memory_initialization_radix	16
memory_initialization_vector	AA 55 99 00

Validate Save Save As... Close

```

1 memory_initialization_radix=16;
2 memory_initialization_vector=AA 55 99 00;
3

```

Customize IP

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☒ Show disabled ports

Component Name: blk_mem_gen_0

Pipeline Stages within Mux: 0 Mux Size: 1x1

Memory Initialization

☒ Load Init File

Coe File: "Users/urist/Desktop/LECTURE/Verilog-L4/ROM/KUKU.coe" Browse Edit

☐ Fill Remaining Memory Locations

Remaining Memory Locations (Hex): 0 Fill Remaining Memory Locations

Structural/UniSim Simulation Model Options

Defines the type of warnings and outputs are generated when a read-write or write-write collision occurs.

Collision Warnings: All

Behavioral Simulation Model Options

☐ Disable Collision Warnings ☐ Disable Out of Range Warnings

OK Cancel

Generate Output Products

The following output products will be generated.

Preview

- blk_mem_gen_0.xci (OOC per IP)
 - Instantiation Template
 - Synthesized Checkpoint (.dcp)
 - Structural Simulation
 - Change Log

Synthesis Options

Global ☒ Out of context per IP ☐

Run Settings

Number of jobs: 4

Apply Generate Skip

לאחר שהרצנו נקבל Vivado ייצר מספר קבצים.

xdc – רשום בו האילוצים (שקשורים לclk ולמיקומים).

```

47 # PART OF THIS FILE AT ALL TIMES.
48 #
49 #####
50
51 # Core Period Constraint. This constraint can be modified, and is
52 # valid as long as it is met after place and route.
53 create_clock -name "TS_CLKA" -period 20.0 [ get_ports clka ]
54 set_property HD.CLK_SRC BUFGCTRL_X0Y0 [ get_ports clka ]
55 #####
56

```

mif – רשומים בו הערכים שבתכולת הזיכרון. קובץ שמקובל בתעשייה.

1	10101010
2	01010101
3	10011001
4	00000000
5	

xml – מגדיר את המאפיינים של המודול.

```

14 <spirit:slave/>
15 <spirit:portMaps>
16 <spirit:portMap>
17 <spirit:logicalPort>
18 <spirit:name>CLK</spirit:name>
19 </spirit:logicalPort>
20 <spirit:physicalPort>
21 <spirit:name>s_aclk</spirit:name>
22 </spirit:physicalPort>
23 </spirit:portMap>
24 </spirit:portMaps>
25 <spirit:parameters>
26 <spirit:parameter>
27 <spirit:name>ASSOCIATED_BUSIF</spirit:name>
28 <spirit:value>spirit:id="BUSIFPARAM_VALUE.CLI
29 </spirit:parameter>
30 <spirit:parameter>
31 <spirit:name>ASSOCIATED_RESET</spirit:name>

```

veo – איך לעשות אינסטאציה למודול הזה. Verilog

```

50 // IP Revision: 4
51
52 // The following must be inserted into your Verilog file for this
53 // core to be instantiated. Change the instance name and port connections
54 // (in parentheses) to your own signal names.
55
56 //----- Begin Cut here for INSTANTIATION Template ---// INST_TAG
57 blk_mem_gen_0 your_instance_name (
58     .clka(clka), // input wire clka
59     .addra(addra), // input wire [1 : 0] addra
60     .douta(douta) // output wire [7 : 0] douta
61 );
62 // INST_TAG_END ----- End INSTANTIATION Template -----
63

```


who – איך לעשות אינסטאציה למודול הזה. בלחצו

```

65 -- body. Substitute your own instance name and net names.
66
67 ----- Begin Cut here for INSTANTIATION Template ----- INST_TAG
68 your_instance_name : blk_mem_gen_0
69     PORT MAP (
70         clka => clka,
71         addra => addra,
72         douta => douta
73     );
74 -- INST_TAG_END ----- End INSTANTIATION Template -----
75

```

יש גם כמה מודלים:

מודל VHDL לסינטזה:

```

50 -- IP Revision: 4
51
52 LIBRARY ieee;
53 USE ieee.std_logic_1164.ALL;
54 USE ieee.numeric_std.ALL;
55
56 LIBRARY blk_mem_gen_v8_4_4;
57 USE blk_mem_gen_v8_4_4.blk_mem_gen_v8_4_4;
58
59 ENTITY blk_mem_gen_0 IS
60     PORT (
61         clka : IN STD_LOGIC;
62         addra : IN STD_LOGIC_VECTOR(1 DOWNTO 0);
63         douta : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
64     );
65 END blk_mem_gen_0;
66
67 ARCHITECTURE blk_mem_gen_0_arch OF blk_mem_gen_0 IS
68     ATTRIBUTE DowngradeIPIdentifiedWarnings : STRING;
69     ATTRIBUTE DowngradeIPIdentifiedWarnings OF blk_mem_gen_0_arch :
70     COMPONENT blk_mem_gen_v8_4_4 IS
71         GENERIC (
72             C_FAMILY : STRING;
73             C_DEVICEFAMILY : STRING;

```


מבוא למושגים קריפטוגרפיה:

קריפטו (מיוונית – κρυπτός): הסתרה, הצפנה
גרפיה (מיוונית – γραφειν): כתיבה
לוגיה (מיוונית – λογία): לימוד

- קריפטוגרפיה (או קריפטולוגיה) היא מדע העוסק באבטחת מידע
- התרגום הישיר של המושג "קריפטוגרפיה/קריפטולוגיה" עשוי להיות "תורת ההצפנה", אבל לא מדובר רק על הצפנה
- תחום אבטחת המידע עוסק באתגרים רבים ושונים בהגנה על מידע, הכוללים: הסתרת מידע (הצפנה), שלמות מידע, זיהוי, הרשאות גישה, שיתוף סודות, מניעת הכחשה ועוד

ותהר האשה, ותלד בן; ותרא אותו כי-טוב הוא, ותצפנהו שלשה ירחים. ולא-יכלה עוד הצפינו, ותקח-לו תבת גמא, ותחמרה בחמר ובצפת; ותשם בה את-הילד, ותשם בסוף על-שפת היאר

(שמות ב', ב')

הבטחת מידע



האופן המוסכם להצגת משתתפים בתיאור תהליך קריפטוגרפי

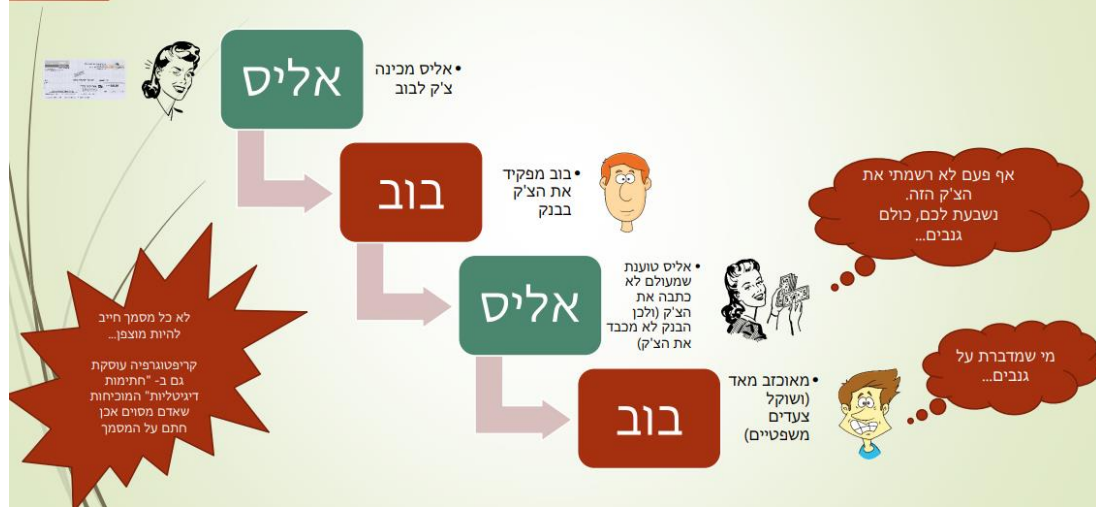
A	B	C	D	E
Alice בעברית: איה	Bob בעברית: בועז	Carol, Carlos, Charlie Chuck: a malicious person Craig: a passwords cracker	Dan, Dave, David	Eve: an eavesdropper (a passive attacker) Erin: a 5 th person



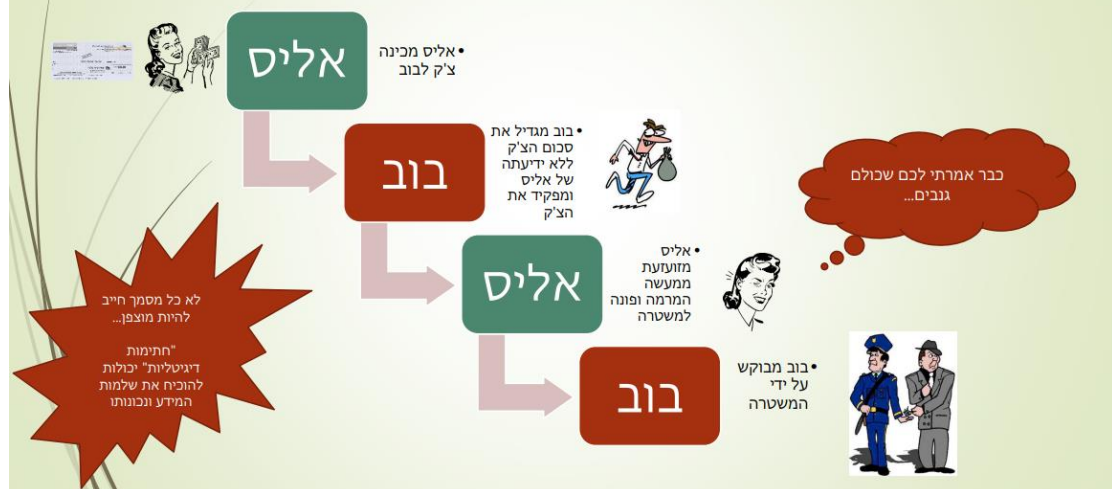
- הוצג לראשונה על ידי Ron Rivest בכמה מאמרים מאז שנת 1977 ומאז הפך לסטנדרט בתיאור תהליכים קריפטוגרפיים
- הפירוט, (כולל שמות משתתפים נוספים) מובא בספר Applied Cryptography של Bruce Schneier
- השמות העבריים איה ובעז הוצעו על ידי פרופ' דוד הראל (מכון ויצמן, חתן פרס ישראל)

נשתמש בשמות המתחילות באותן אותיות.

דוגמא 2: התכחשות למסמך חתום...



דוגמא 3: שינוי מסמך במזיד



דוגמא 4: וודאות לתקינות ושלמות של מידע

אליס

בוב

...111011010001001...01010101001...01011110..

- בוב (המתגורר בירושלים) מוריד קובץ גדול מאד, בפרוטוקול FTP, מתוך שרת הקבצים של אליס בארה"ב
- בוב חושש (ובצדק) שבגלל גודל הקובץ והמרחק הרב עלולים להיפגע ביטים בודדים בקובץ, ויגרמו בעיות בלתי צפויות תוך שימוש בקובץ בלתי תקין
- בוב מעוניין לבדוק את התקינות של הקובץ לפני השימוש בו
- מכיוון שאליס הכינה קובץ נפרד של חתימה דיגיטלית על הקובץ, בוב רק צריך לבדוק אם החתימה שחישב מהקובץ שווה לחתימה שהציגה אליס

דוגמה לקובץ עם חתימה:

Xilinx Unified Installer 2021.2: Windows Self Extracting Web Installer (EXE - 212.41 MB)

MD5 SUM Value : 76b60fc6a7433e066f4e4dd0a855ec93

Download Verification ⓘ

Digests Signature Public Key

אם המספר יוצא אחרת – סימן שהקובץ הגיע שגוי.
לכן, לפני התקנת הקובץ מומלץ לבדוק שהחתימות שוות ע"י אפליקציות מתאימות.
לא מדובר בהצפנה אלא באמצעי הבדוק את שלימות המידע.

דוגמא 5: הרשאות גישה \ בקרת גישה

אקטואליה

מומחה סייבר: "הפריצה לטלפון של גנץ יכלה לקרות גם כשהיה רמטכ"ל"

יצחק בן ישראל, ראש מרכז הסייבר באוניברסיטת ת"א, התייחס לפריצה האיראנית לטלפון של יו"ר כחול-לבן: "זה לא מסובך, כנראה שהוא הפך למטרה כשנכנס לפוליטיקה"

מערב אונליין | 11:56 15/03/2019 | דק' קריאה

חשיפת "עובדה": מידע מהפלאפון של אהוד ברק מוחזק בידי האיראנים

על רקע פרשת הפריצה האיראנית לטלפון הנייד של גנץ: השב"כ עדכן גם את ברק כי הטלפון שלו נפרץ - ויש חשש שהמידע שהיה בו מוחזק בידי האיראנים. גם השב"כ וגם ברק סירבו להתייחס לידיעה

שחר אלתרמן | עובדה | פורסם 17/03/19 22:30

המידע אליו מתבצעת הגישה לא בהכרח מוצפן
מדובר על ההגנה לגישה למידע

דוגמא 6: הרשאות גישה \ בקרת גישה עם הצפנה

בורסת קריפטו ללקוחותיה: לא נוכל להחזיר כסף כי המייסד מת עם הסיסמה למערכת

חברת QuadrigaCX חייבת ללקוחות שלה כ-190 מיליון דולר במטבעות וירטואליים. הסבר: המייסד ג'רלד קוטן מת במפתיע בגיל 30 והיה היחיד שברשותו סיסמת ההפעלה למערכת

שירות כלכליסט 04.02.19 11:58



לפעמים ההגנה
חזקה מדי...

אפילו למי שצריך
לגשת למידע באופן
חוקי

אנו נלמד בקורס על אבני היסוד המרכיבות את האלגוריתמים בקריפטוגרפיה

על כמה אלגוריתמים מרכזיים

ועל המימוש בחומרה של אותן אבני יסוד ושל האלגוריתמים המדוברים

מבוא למושגים בקריפטוגרפיה

אלגוריתמים בקריפטוגרפיה ומימוש ב-Verilog (מספר קורס – 130330)

פרק ב': פעולות הפיכות ופעולות בלתי הפיכות

שם המרצה: אורי שטרו

נושאי הפרק

- פונקציה חד-כיוונית – One Way Function
- פונקציות הפיכות
 - XOR
 - Permutation
 - SBOX
- מושגי יסוד בתיאור מרכיבי מערכת הצפנה
- הצפנת One Time Pad
- צופן זרם – Stream Cipher

פונקציה חד-כיוונית One Way Function

נתונה הפונקציה: $y = f(x)$

אנו אומרים שהפונקציה f היא One Way Function (חד-כיוונית) אם

$$x = f^{-1}(y)$$

בלתי אפשרית (או קשה מאוד) לביצוע

פונקציות חד-כיווניות
משתתפות באלגוריתמים
רבים בתחום
הקריפטוגרפיה.
נראה בהמשך הקורס...

אתה יכול להצפין פונקציה חד כיוונית אך לא לפענח, אז מה עושים?
התשובה היא שפונק' חד כיוונית משתתפת באלגוריתמים. אתה לא אמור ללכת אחורה במידע ולפענח אותו.

פונקציות הפיכות

פונקציית XOR

הפונקציה ההפוכה

$$X = f^{-1}(y) = Y \text{ xor } A \rightarrow$$

שהרי:

$$Y = X \text{ xor } A \rightarrow$$

$$Y \text{ xor } A = X \text{ xor } A \text{ xor } A \rightarrow$$

$$Y \text{ xor } A = X \rightarrow$$

מ.ש.ל

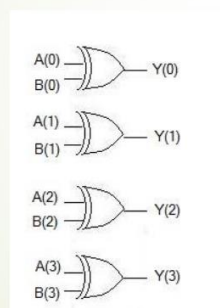
הגדרת הפונקציה

$$Y = f(x) = X \text{ xor } A \rightarrow$$

XOR

מימוש בחומרה

תוצאת הסינטזה



Verilog – מימוש בחומרה

```

wire [3:0] A;
wire [3:0] B;
wire [3:0] Y;
  
```

...

```

assign Y = A ^ B;
  
```

ניתן להציב ביט-ביט:

```

assign Y[0] = A[0] ^ B[0];
assign Y[1] = A[1] ^ B[1];
assign Y[2] = A[2] ^ B[2];
assign Y[3] = A[3] ^ B[3];
  
```

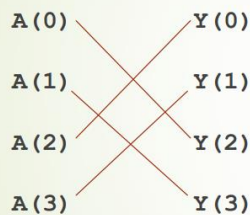
Area: שטח שערי ה-XOR
Timing: זמן ההשהיה של השער הלוגי (Propagation Delay)

פרמוטציה:

החלפה ברמה של הביט.

Permutation ידועה גם בשם P-Box

תוצאת הסינטזה



מימוש בחומרה – Verilog

```

wire [3:0] A;
wire [3:0] Y;

...
assign Y[0] = A[2];
assign Y[1] = A[3];
assign Y[2] = A[0];
assign Y[3] = A[1];
  
```

הפונקציה ההפוכה:

```

assign A[0] = Y[2];
assign A[1] = Y[3];
assign A[2] = Y[0];
assign A[3] = Y[1];
  
```

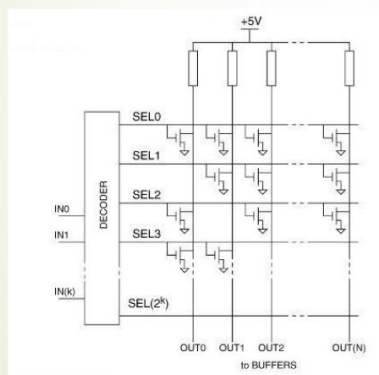
Area: אין תאים לוגיים או טרמיסטורים, מדובר על Wire בלבד (לכל היותר שטח ה-Wiring)
Timing: לכל היותר Wire Delay (לפעמים גם זה לא, כי מדובר על "שינוי שם" של סיגל... כנ"ל ב-Area)

SBOX

כאן רואים החלפה של 4 ביטים ב-4 ביטים. החלפה ברמה של הביט.
נעשה כאן שימוש ב-rom. השטח של ה-rom הוא השטח של הלוגיקה.

SBOX ידועה גם בשם Substitution Box

מימוש בחומרה ברמת הטרנזיסטור



מימוש בחומרה – Verilog

```

wire [3:0] adrs;
reg [3:0] data;

always @(*)
  case (adrs)
    0: data = 4'hf;
    1: data = 4'h7;
    . . .
    14: data = 4'h2;
    15: data = 4'h0;
  endcase
  
```

זוהי אחת מכמה אפשרויות קידוד ומימוש

Area: שטח ה-ROM או שטח הלוגיקה אם מדובר על Synthesized ROM
Timing: זמן ההשהיה של ה-ROM או של הלוגיקה (Propagation Delay) או Address to Data_out

מושגי יסוד בתיאור מערכת הצפנה

המושג	סימון	הסבר
Plain Text	P	מידע לא מוצפן
Cipher Text	C	מידע מוצפן
Message	M	הודעה (במקרים רבים מדובר על ה-Plain Text)
Key	K	מפתח ההצפנה

דוגמה 1:

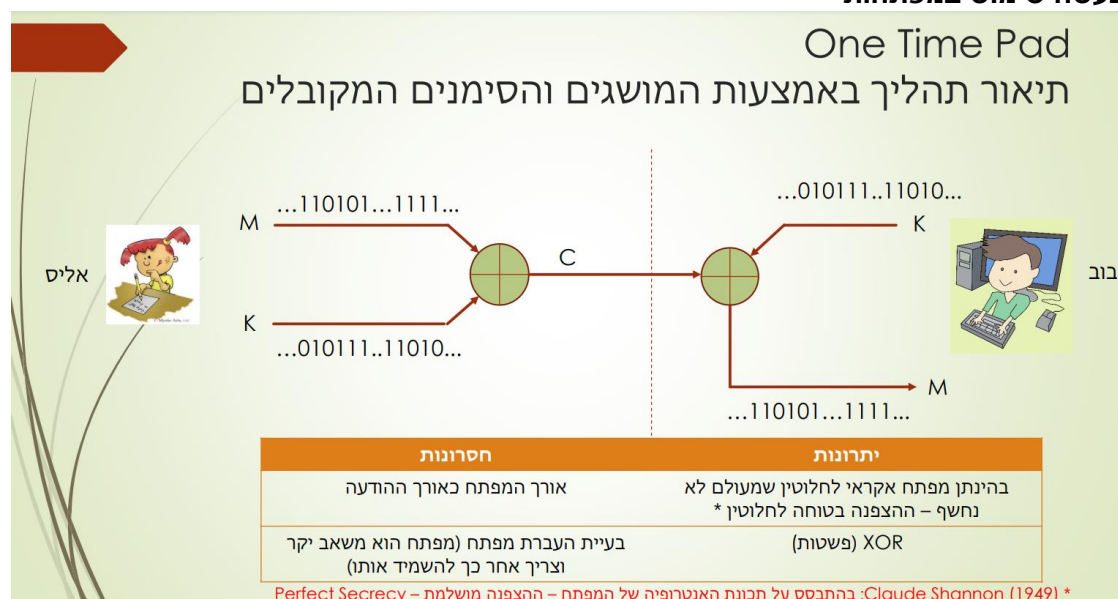
אלגוריתם ההצפנה הבטוח ביותר בעולם

One Time Pad "פנקס חד פעמי"

- נתונה הודעה בגודל N ביטים
- יש לבחור מפתח בגודל N ביטים אקראי (שמעולם לא השתמשו בו) - מפתח זה נמצא הן בידי השולח והן בידי המקבל
- מבצעים XOR ביט ביט, בין ביט מההודעה וביט מהמפתח, בהתאמה
- שולחים את תוצאת ה-XOR
- המקבל, מפענח את ההודעה על ידי ביצוע XOR ביט ביט, בין ההודעה לבין המפתח בהתאמה
- אסור באיסור מוחלט להשתמש שוב במפתח

נעשה שימוש במפתחות

One Time Pad תיאור תהליך באמצעות המושגים והסימנים המקובלים



לפעמים וכך לרוב, האלגוריתם ידוע, והסוד זה המפתח.

One Time Pad היסטוריה

- שיטת One Time Pad הומצאה בשנת 1917 על ידי Gilbert Sandford Vernam במעבדות Bell (AT&T)
- המנגנון נקרא "צופן Vernam Cipher – Vernam Cipher"
- מאז ועד היום נעשה שימוש רב ב-One Time Pad (בפרט על ידי ארה"ב ובריה"מ בתקופת מלחמת העולם השנייה ואחריה, בתקופת המלחמה הקרה)
- הסובייטים "הרשו לעצמם" לעשות שימוש חוזר במפתחות
- התוצאה: במסגרת Venona Project (בין 1943 ל-1980, פורסם רק ב-1995) חשפו שירותי הביון של ארה"ב תשדורות סובייטיות סודיות, כתוצאה משימוש לא נכון (שימוש חוזר במפתחות) והפילו את מרגלי האטום הסובייטים בארה"ב

הצפנה ביט ביט.

כאשר משתמשים באופטימיזציה במודלים, אז יכול להיות שלא נראה את הסיגנלים בwaveform, כי התכנון יראה אחרת שם וכו', אבל לכן נוכל לקבל את הערכים שלהם דרך המוניטור בסקריפט.

זכרונות ROM:

יש מספר דרכים לממש (במודל). נראה פה בסיכום את החידוש, שיש כזה דבר וקטור:

```

1  module rom
2  (
3      input  [1:0]  adrs,
4      output [7:0]  data
5  );
6
7      reg [7:0] rom_array [0:3];
8
9      initial
10     begin
11         rom_array[0] = 8'haa;
12         rom_array[1] = 8'h55;
13         rom_array[2] = 8'h99;
14         rom_array[3] = 8'h00;
15     end
16
17     assign data = rom_array[adrs];
18
19 endmodule

```

כלומר יש פה מערך דו מימדי – 4 מקומות, שכל אחד הוא בגודל של 4 ביטים.

[*initial לפעמים לא יעבוד בכלי סינטיזה מסוימים, באופן כללי.]

כעת נראה אפשרות של קריאה מקובץ:

```

1 module rom
2 (
3     input  [1:0]  adrs,
4     output [7:0]  data
5 );
6
7 reg [7:0] rom_array [0:3];
8
9 initial
10     $readmemh("rom.txt", rom_array, 0, 3);
11
12 assign data = rom_array[adrs];
13
14 endmodule

```

נצטרך להכניס לכלי הסינטזה את הקובץ?
 אבל צריך לוודא שאכן הוא קרא אותו כמו שצריך, כי לפעמים הכלים לא אוהבים את זה. נצטרך גייט סימוליישן.

שלושת הדוגמאות האלה הם סינטייזד ממורי. עכשיו נראה קומפיילד ממורי.
 זאת אפשרות בכלי הסינטזה. להשתמש בבלוק הזיכרון שמובנה בסיליקון במקום בFF.

קריפטוגרפיה:

תחום העוסק באבטחת מידע. מכיל הצפנות (הסתרת מידע), זיהוי משתמשים, הרשאות גישה, מניעת הכחשה ועוד.

מושגים:

א. פונקציה חד כיוונית – כאשר הפעולה $x = f^{-1}(x)$ קשה לביצוע.

ב. דו כיוונית- כאשר זה כן אפשרי. דוגמאות להפיכות:

1. XOR.
2. Permutation.
- 3.

שיעור 5

הצפנה - המשך:

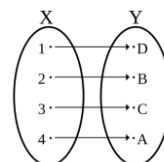
מושג – Stream Cipher "צופן זרם"

- מנגנון המצפין את המידע תו אחר תו (או ביט אחר ביט) עם שידור
- One Time Pad הוא למעשה Stream Cipher
- אנו נלמד בהמשך על Block Cipher ("צופן בלוקים" או "צופן גושים" המבצע הצפנה על מידע לאחר שנאסף לבלוק בגודל נתון (ולא מיד עם שידור)

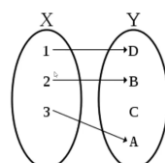
משדרים ביט אחרי ביט.

אמרנו שפונק' XOR היא הפיכה – זו פונק' דו כיוונית,

פונק' Permutation (P-Box) ו-SBox הן גם הפיכות אך לא בהכרח, וזה קשור להגדרה מתמטית.

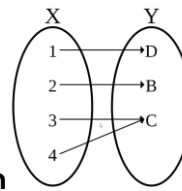


ישנה התאמה בין קבוצות:



התאמה חד-ערכית.

אולם, ישנה התאמה שלא מכסה את הכל, היא "לא על":



ישנה עוד התאמה: היא לא חד-ערכית, א"א לעשות התאמה הפוכה, C יקבל 3 או 4.

כדי שהתאמה תהיה הפיכה ונעשה f^{-1} על הפונק' להיות 'על' חד-ערכית – חד-חד-ערכית.

מימוש One Time Pad ב-Verilog

דוגמה

```
1 module one_time_pad
2 (
3     input  [7:0] data_in,
4     input  [7:0] key_in ,
5     output [7:0] data_out
6 );
7
8 assign data_out = data_in ^ key_in;
9
10 endmodule
11
```

ע"י XOR עושים כאן הצפנה, ביט ביט.

הצפנה ע"י קריאה מקבצים:

```
1 `timescale 1 ns / 1 ps
2
3 module tb_one_time_pad();
4
5     integer data_file_in;
6     integer key_file_in ;
7     integer statusD;
8     integer statusK;
9     reg [7:0] plaintext, key;
10    wire [7:0] ciphertext, resolved_plaintext;
11    reg      clk = 0;
12
13    always
14        #10 clk = ~ clk;
15
16    initial
17    begin
18        data_file_in = $fopen("data_in.txt", "r");
19        key_file_in  = $fopen("key_in.txt", "r");
20    end
21
```

הקובץ המוצפן

פותח את הקבצים במוד של קריאה:

```
16 initial
17 begin
18     data_file_in = $fopen("data_in.txt", "r");
19     key_file_in  = $fopen("key_in.txt", "r");
20 end
21
```

פעולות שבחרנו לעשות ולקרוא מהקובץ:

```

22 initial
23 begin
24   repeat (10) @ (posedge clk);
25   while ( ! $feof(data_file_in))
26   begin
27     @ (negedge clk);
28     statusD = $fscanf(data_file_in,"%h\n", plaintext);
29     statusK = $fscanf(key_file_in,"%h\n", key );
30     @ (negedge clk);
31   end
32   repeat (10) @ (posedge clk);
33   $fclose(data_file_in);
34   $fclose(key_file_in );
35   #100 $finish;
36 end

```

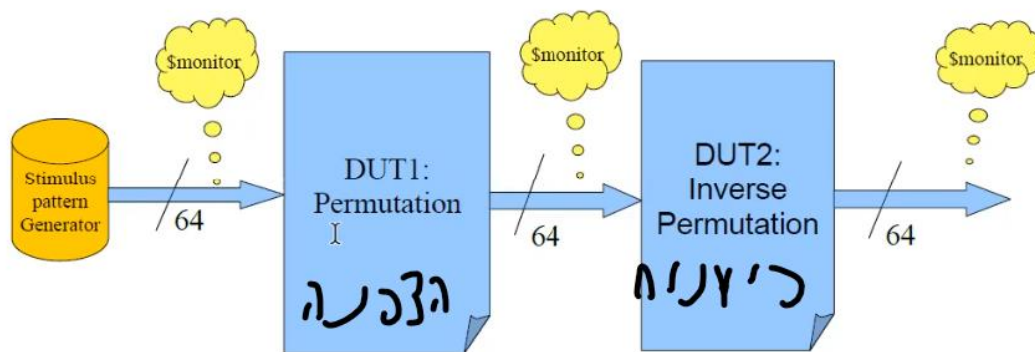
```

7
8 always @(*)
9   $display("time = %8t | K = %h | P = %2c | C = %2c | Recovered = %2c",
10            $time, key, plaintext, ciphertext, resolved_plaintext);

```

כמו \$monitor

במטלה 4 נעשת בדיקת אלגוריתמים של הצפנה.



הנתן שניכנס צריך גם לצאת.

```

1
2 one_time_pad encrypt
3 (
4   .data_in (plaintext),
5   .key_in (key),
6   .data_out (ciphertext)
7 );
8
9 one_time_pad decrypt
10 (
11   .data_in (ciphertext),
12   .key_in (key),
13   .data_out (resolved_plaintext)
14 );
15
16 endmodule

```


הקוד מדפיס כל מיני אותיות מוזרות:

אתרים לחישוב הצפנה:

<https://www.branah.com/ascii-converter>

Convert ASCII (Example: a b c)

Hello World!

Add spaces Remove spaces ☒ Convert with

Convert Hex (Example: 0x1 0x2 0x6:
48656463620576672664210a

Convert Decimal (Example: 97 96 99:
07211108108111032087111141081000330

Convert Binary (Example: 01100001: 01
0100100001100101011001100110001101
001101100011000100001000010001010

Convert Base64 (Example: YSBIQm=)

3/20/2018 12:37:50 PM GPGP Express

עבור המפתח:

<https://www.random.org/bytes>

Part 1: Required Fields

Generate random bytes (maximum 16,384).

How do you want your bytes displayed?

- ☒ Hexadecimal
- ☐ Decimal
- ☐ Octal
- ☐ Binary
- ☐ Download to file

Part 2: Go!

6d 61 8c 56

© 1998-20

נקבל מספרים רנדומלים.

שיטות הצפנה:

1. One Time Pad:

לוקחים איזהו מפתח בגודל N ביטים, שאותו שולחים בדרך אחרת, ואז עושים XOR עם המידע.

יתרונות – הצפנה בטוחה, פשוטות מימוש.

חסרונות – מפתח ארוך, בעיית העברה.

[Stream cipher]:

מנגנון שמצפין ביט אחרי ביט ברגע השידור. One time pad הוא אחד כזה.

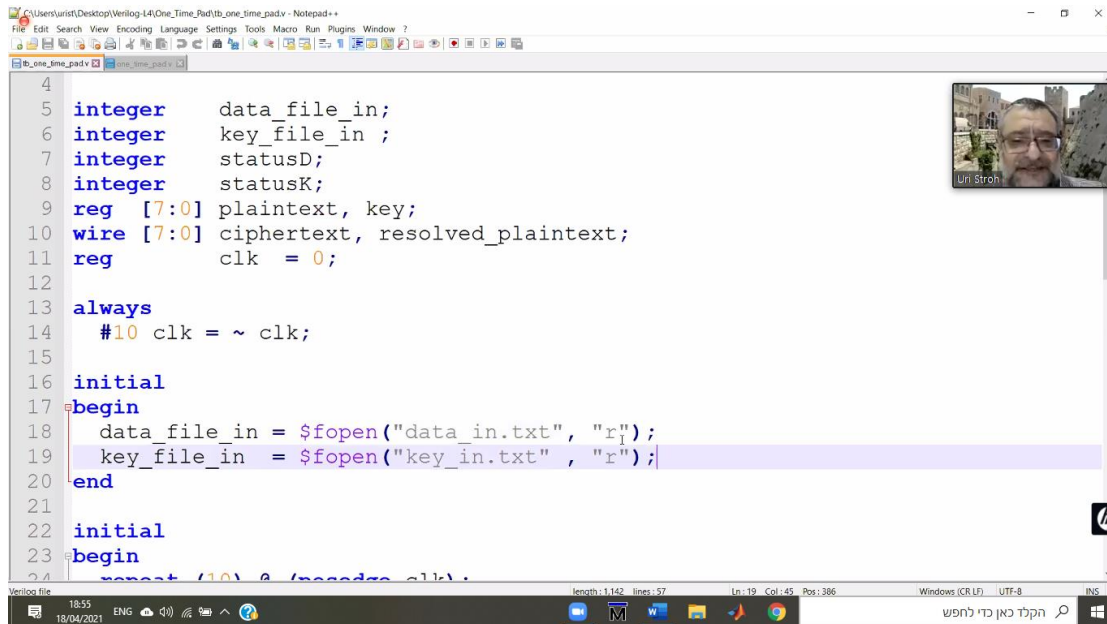
מימוש פשוט של המצפין והמפענח, שזה אותו דבר:

```
module one_time_pad
(
    input  [7:0] data_in,
    input  [7:0] key_in ,
    output [7:0] data_out
);

assign data_out = data_in ^ key_in;

endmodule
```

קריאה מקובץ בTB:



```

4
5 integer    data_file_in;
6 integer    key_file_in ;
7 integer    statusD;
8 integer    statusK;
9 reg [7:0] plaintext, key;
10 wire [7:0] ciphertext, resolved_plaintext;
11 reg        clk = 0;
12
13 always
14     #10 clk = ~ clk;
15
16 initial
17 begin
18     data_file_in = $fopen("data_in.txt", "r");
19     key_file_in = $fopen("key_in.txt", "r");
20 end
21
22 initial
23 begin
24     repeat (1000) @(posedge clk);

```

(יש עוד פרטים חשובים פה בTB הזה, ראה קובץ מלא!)

בכל אופן, אנחנו נשרשר פעמיים את הרכיב של ההצפנה בTB, כדי שפעם אחת יקודד ופעם שניה יפענח, כי זו הרי אותה פעולה.

2. פרמנטציה - שינוי מקום הביטים, נקרא גם (Pbox) -

דוגמא ספציפית לPBOX - כל מספר מופיע במקום שהוא הופיע לפי ערכו, כדלהלן:

3. SBOX – שינוי ערכים לפי החלטה מראש (case)

IP

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

That is the permuted input has bit 58 of the input as its first bit, bit 50 as its second bit, and so on with bit 7 as its last bit. The permuted input block is then the input to a complex key-dependent computation described below. The output of that computation, called the preoutput, is then subjected to the following permutation which is the inverse of the initial permutation:

IP⁻¹

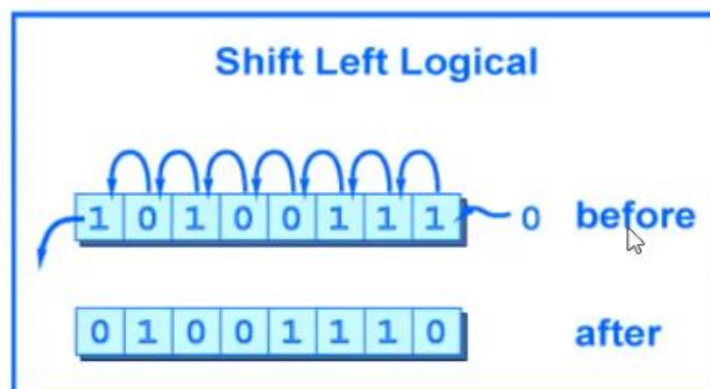
40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

That is, the output of the algorithm has bit 40 of the preoutput block as its first bit, bit 8 as its second bit, and so on, until bit 25 of the preoutput block is the last bit of the output.

שיעור 6

פעולות Shift ו-Rotate הן מקרה פרטי של פרמוטציה.

פעולת Shift Left:

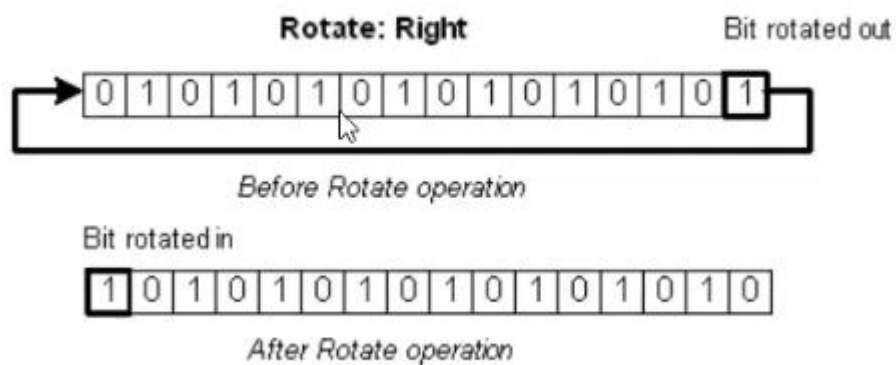


המידע נדחף שמאלה ומימין נוסף 0. – דחיפה.
פעולה זו היא בלתי הפיכה. פעולת Shift Right.
זו בעצם הכפלה ב2.

דוגמה:

BITWISE OPERATORS		
<< <u>Shift Left</u>		
<u>SYNTAX</u>	<u>BINARY FORM</u>	<u>VALUE</u>
$x = 7;$	00000111	7
$x = x << 1;$	00001110	14
$x = x << 3;$	01110000	112
$x = x << 2;$	11000000	192

פעולת Rotate Left:



דוחף שמאלה ובלולאה, הנדחף הולך הכי ימינה. – לולאה.
דוגמה אחרת:



קוד של Shift Left:

```

1 // A=11001100
2 //   76543210
3
4 // Y=01100000<--
5 //   76543210
6
7 // Y = A << 3
8
9 wire [7:0] y, a;
10
11 assign y = {a[4:0], 3'b000};
12
13 // More ways...
14 assign y[7] = a[4];
15 assign y[6] = a[3];
16 ...
17 assign y[2:0] = 3'b0;
18

```

קוד של Rotate Left:

```

////////////////////////////////////
// A=11001100
//   76543210
//
// Y=01100110
//   76543210
//
// Y = A >> 3
//
wire [7:0] y, a;

assign y = {a[4:0], a[7:5]};

// More ways...
assign y[7] = a[4];
assign y[6] = a[3];
...
assign y[2:0] = a[7:5];

////////////////////////////////////

```

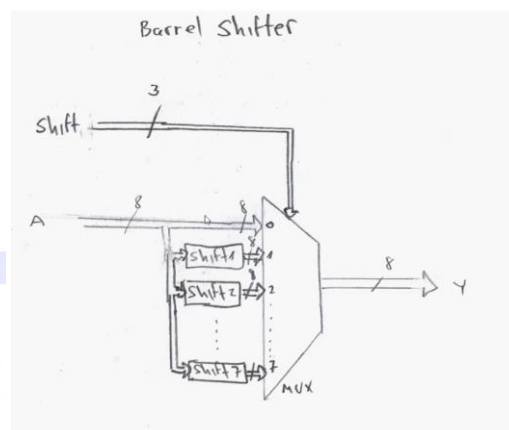
מבחינת Area מדובר רק בעניין של חיוט. ומבחינת זמן זה אפ' אפס time.

החומרה הפשוטה ביותר לממש פעולה זו היא Barrel Shifter:

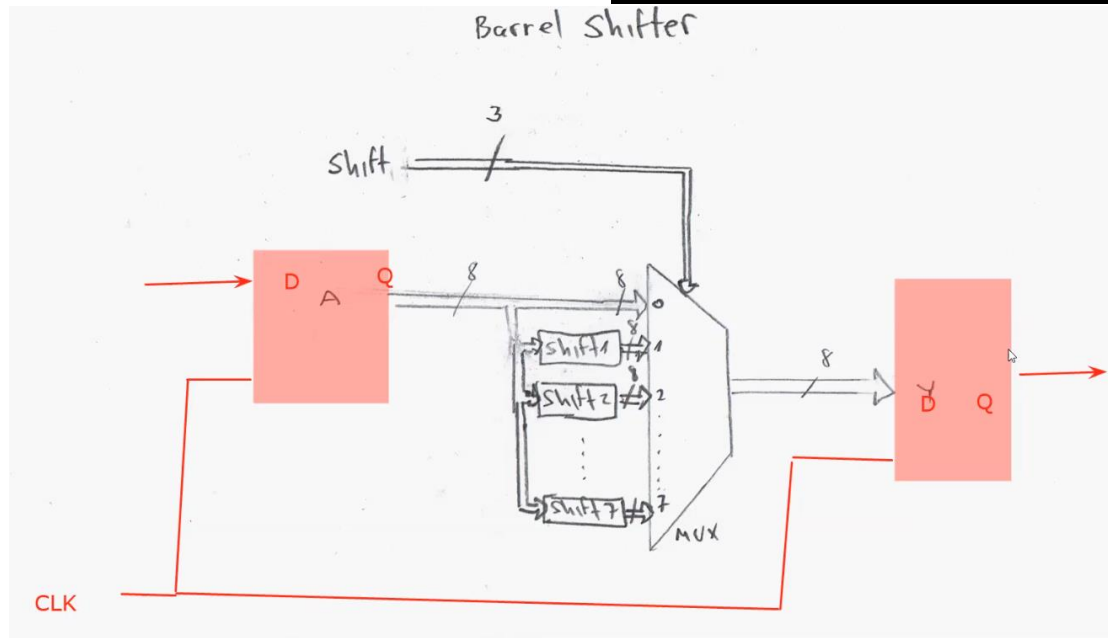
```

always @(*)
case (shift)
0: y = a;
1: y = {a[6:0], 1'b0};
2: y = {a[5:0], 2'b00};
...
7: y = {a[0], 6'b000000};
endcase

```



ע"מ להעריך את הזמן נוסף פליפלופים:



הזמן של Shift בגודל 0 הזזות הוא

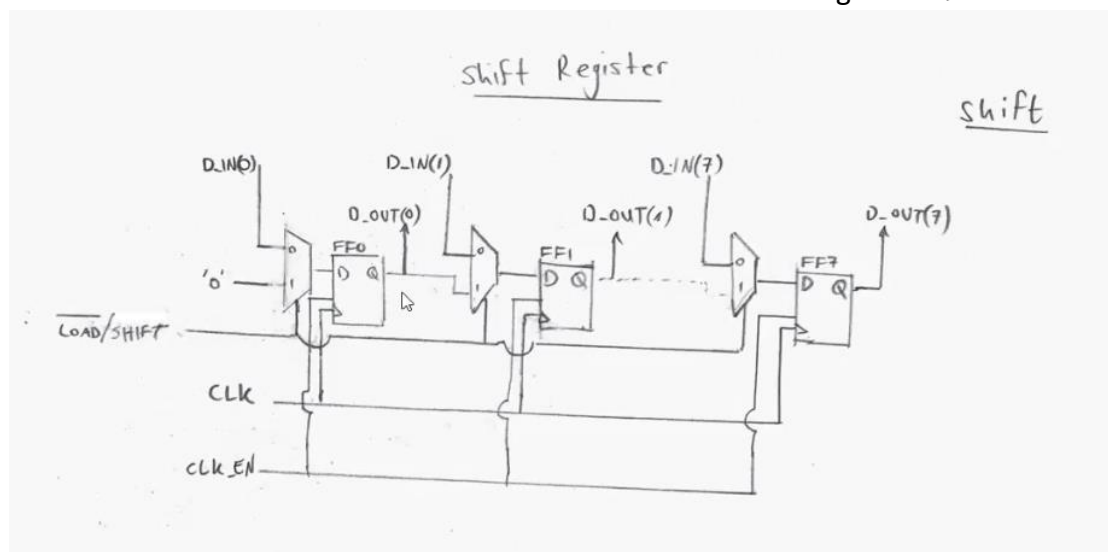
מדדים:

Area (Mux area)

Timing (Tpd of MUX)

Performance (1 cycles no matter the amount of shift)

מימוש באמצעות Shift Register:



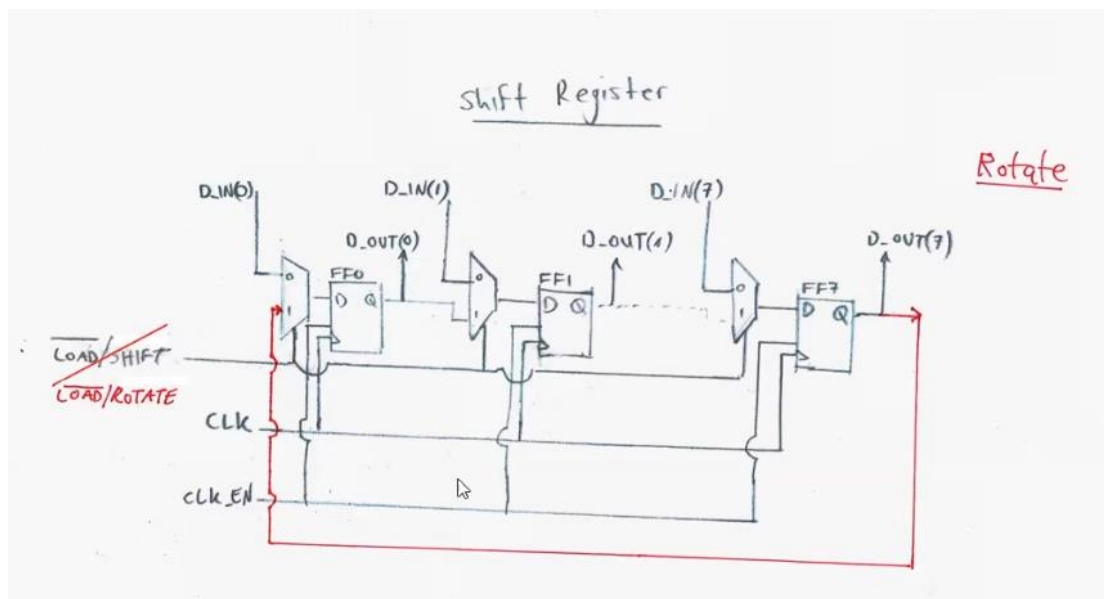
ישנן בעיות שפותר וישנן שלא:

- מכיון שמשתמשים בריגסטרים במימוש זה, אין צורך בריגסטרים בכניסה וביציאה וחסכו אותם.
- מבחינת time area יש שכאן תלות במספר האמורים והפליפלופים.
- מכיון שהאמורים רזים יותר, אז המסלולים הקריטיים בין פליפלופ לפליפלופ קטן יותר ומבחינת time זה טוב יותר.

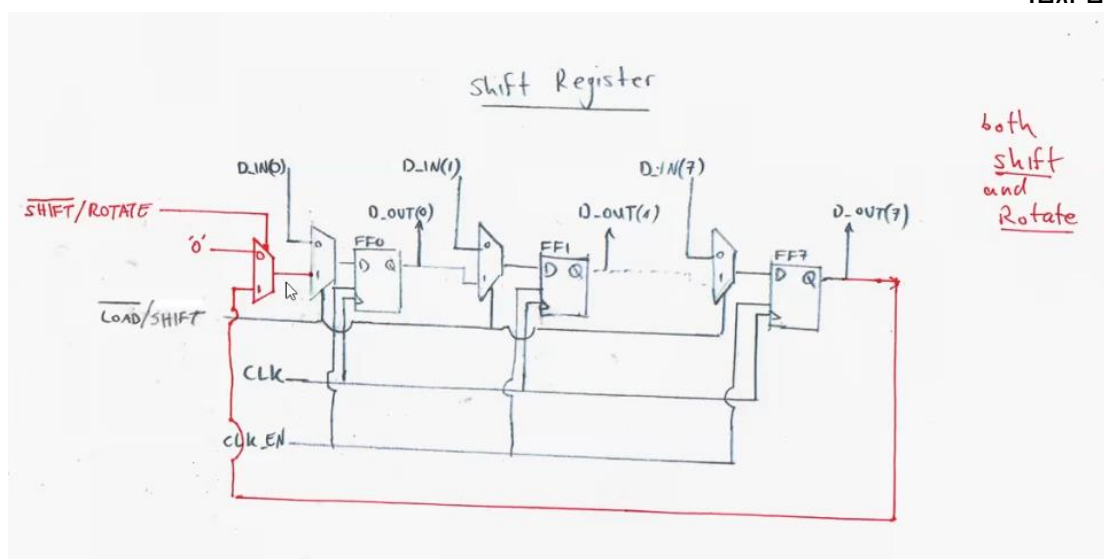
- Barrel Shifter לוקח זמן מחזור 1, Shift Register לוקח זמן מחזור * מספר ההזזות. לכן בביצועים יכול להיות שפחות טוב.

פעולות Shift ו-Rotate נפוצות בהצפנה ולכן נלמד איך הן ממומשות בחומרה.

עבור Rotate Left:



:DL DL



בקוד עבור Shift Left:

```
always @(*)
    data[0] = a;

always @(posedge clk or posedge rst)
    if (rst)
        data[1] <= 1'b0;
    else
        data[1] <= data[0];

always @(posedge clk or posedge rst)
    if (rst)
        data[2] <= 1'b0;
    else
        data[2] <= data[1];

always @(posedge clk or posedge rst)
    if (rst)
        data[3] <= 1'b0;
    else
        data[3] <= data[2];

assign y = data[3];

endmodule
```

זאת כתיבה שעבור מספר רב של ביטים יוצרת עבודה קשה, לכן נשתמש באינדוקציה:

```
3     input  a ,
4     input  clk,
5     input  rst,
6     output y
7 );
8
9     reg [3:0] data;
10
11     always @(*)
12         data[0] = a;
13
14     genvar i;
15
16     for (i = 0; i < 3; i = i + 1)
17     begin
18         always @(posedge clk or posedge rst)
19             if (rst)
20                 data[i + 1] <= 1'b0;
21             else
22                 data[i + 1] <= data[i];
23     end
24
25     assign y = data[3];
```

מגדיר את כמות
השיכפולים
i - אינדקס השיכפול

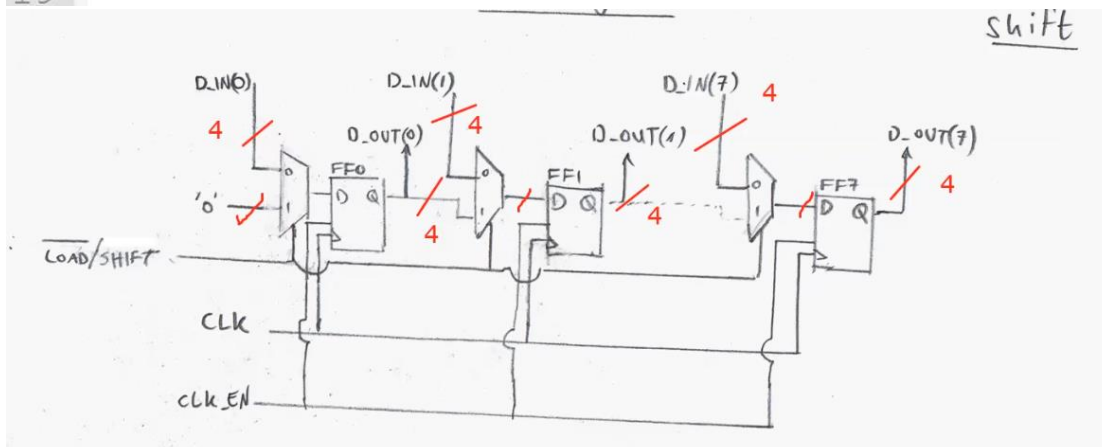
קוד משוכפל ידנית. זה נותן יתרון של פחות עבודה וללא טעויות בשינוי ערכים.

כאשר אין מספיק מקום, אפשר להגדיר כל בית כ4 ביטים וכך הקוד:

```

1 module shiftreg2
2 (
3     input    [3:0] a    ,
4     input    clk,
5     input    rst,
6     output   [3:0] y
7 );
8
9 reg [15:0] data;
10
11 always @(*)
12     data[3:0] = a;
13
14 always @(posedge clk or posedge rst)
15 if (rst)
16     data[7:4] <= 4'b0;
17 else
18     data[7:4] <= data[3:0];
19

```



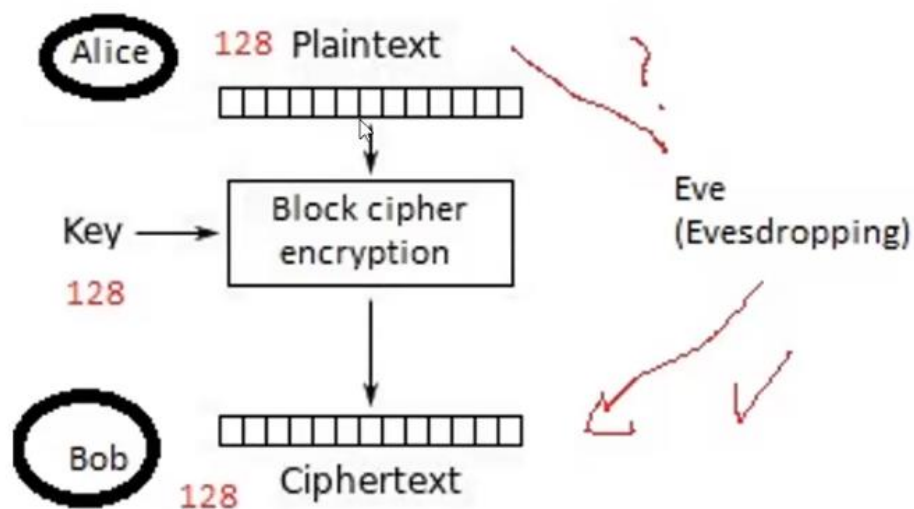
ובקוד עם שכפול:

```

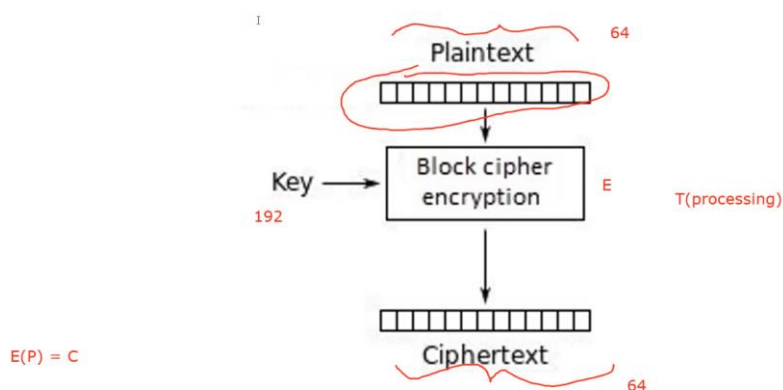
9  reg [15:0] data;
10
11  always @(*)
12      data[3:0] = a;
13
14  genvar i;
15
16  for (i = 0; i < 3; i = i + 1)
17  begin
18      always @(posedge clk or posedge rst)
19          if (rst)
20              data[4*(i+1)+3 -: 4] <= 4'b0;
21          else
22              data[4*(i+1)+3 -: 4] <= data[4*(i+1)-1 -: 4];
23      end
24
25  assign y = data[15:12];

```

הצפנת Block Cipher:
דיברנו על One Time Pad.



אליס שלחה מידע לבוב.

עבור encryption (הצפנה):

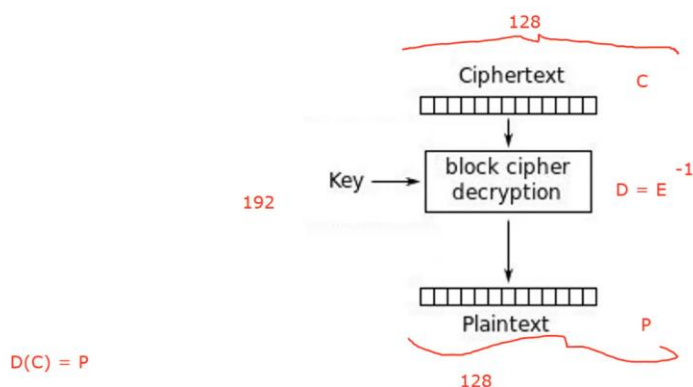
כאשר מצפינים,

מעבדים את כל המידע, כל 128 ביטים מצטברים, ואח"כ מצפינים. א"א לשלוח ביט אחד ולהצפין, וכך ביט ביט.

הגדרת גודל בלוק סייפר הוא חלק מהאלגוריתם.

עבור decryption (פענוח):

הפעולה ההפוכה.

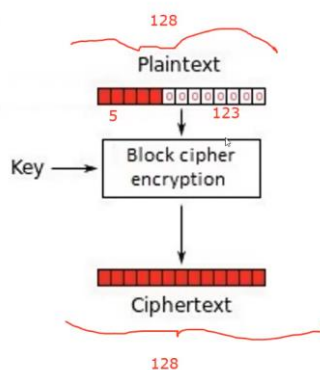


מכאן עולות שתי שאלות:

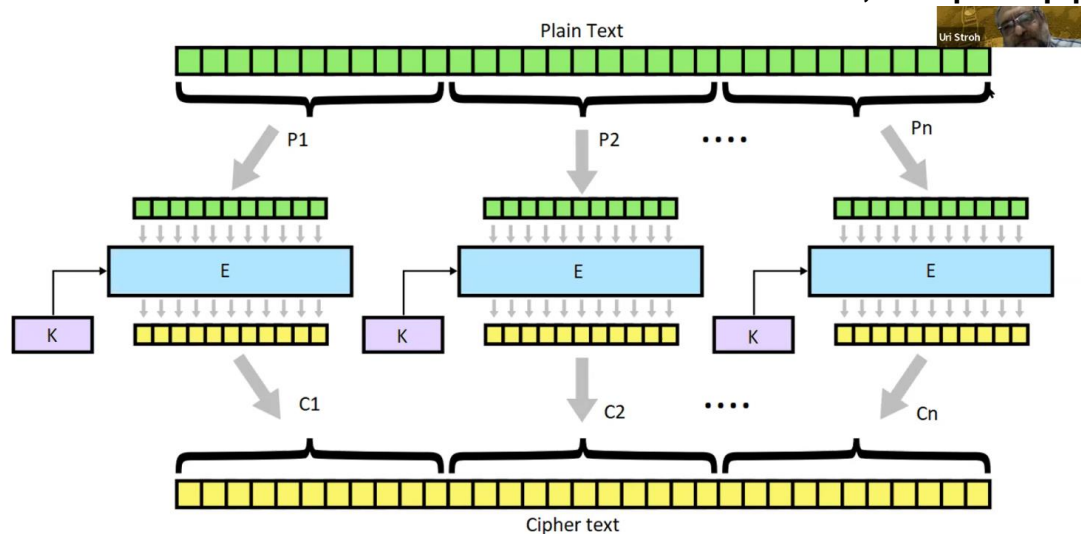
1. מה קורה כאשר גודל המידע קטן מגודל הבלוק סייפר?
2. מה קורה כאשר גודל המידע גדול מגודל הבלוק סייפר?

פתרונות:

1. עבור גודל קטן - ריפוד באפסים



2. עבור גודל גדול - נחלק את המידע בכפולות של הבלוק סייפר. אם הבלוק האחרון קטן מבלוק סייפר, נרפד באפסים.



כדי שפונקציה תהיה הפיכה, יש צורך שהיא תהיה:

1. על.

2. חד ערכית.

אחרת לא נוכל לשחזר את הערכים שהוצפנו.

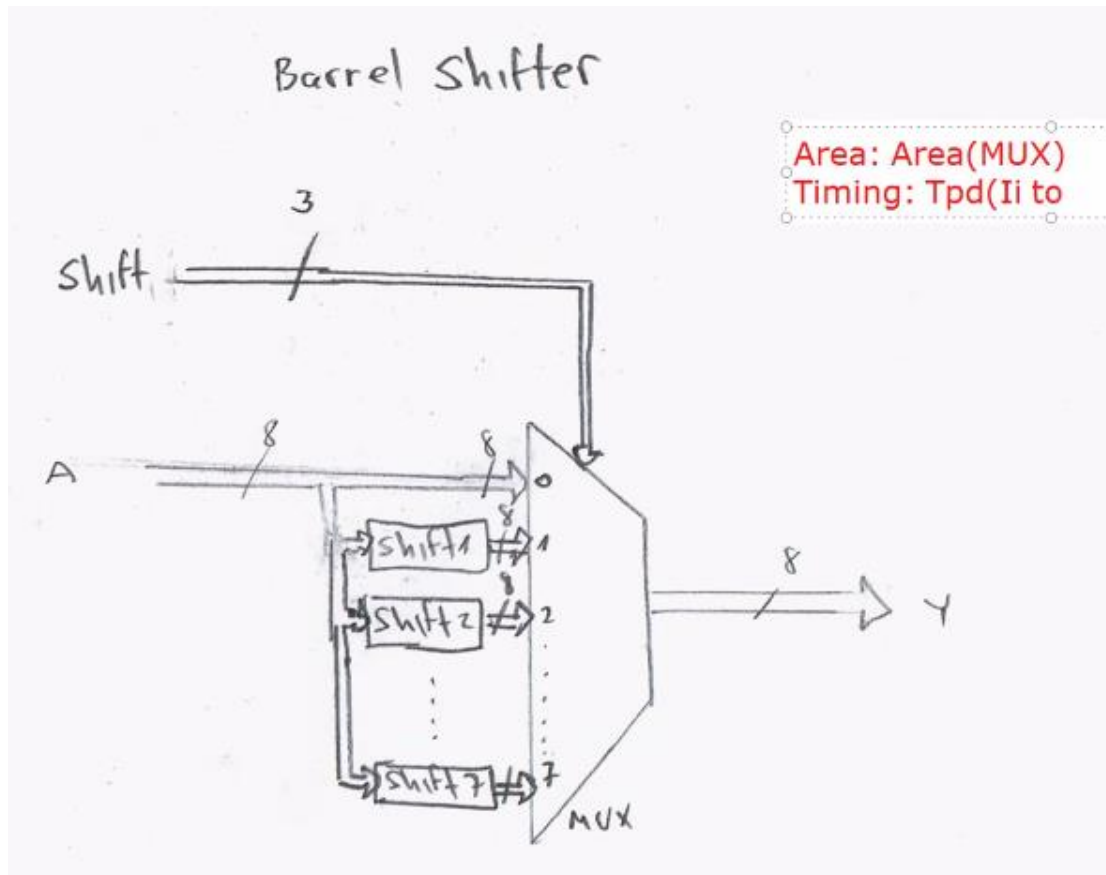
לכן אם נרצה להשתמש בשיטת הצמנה של הזזה של הערכים (SHIFT) נצטרך לעשות ROTATE, כדי לשמור על כל הזיכרון בפנים. אפשרויות למימוש:

$$Y=A<<1$$

או

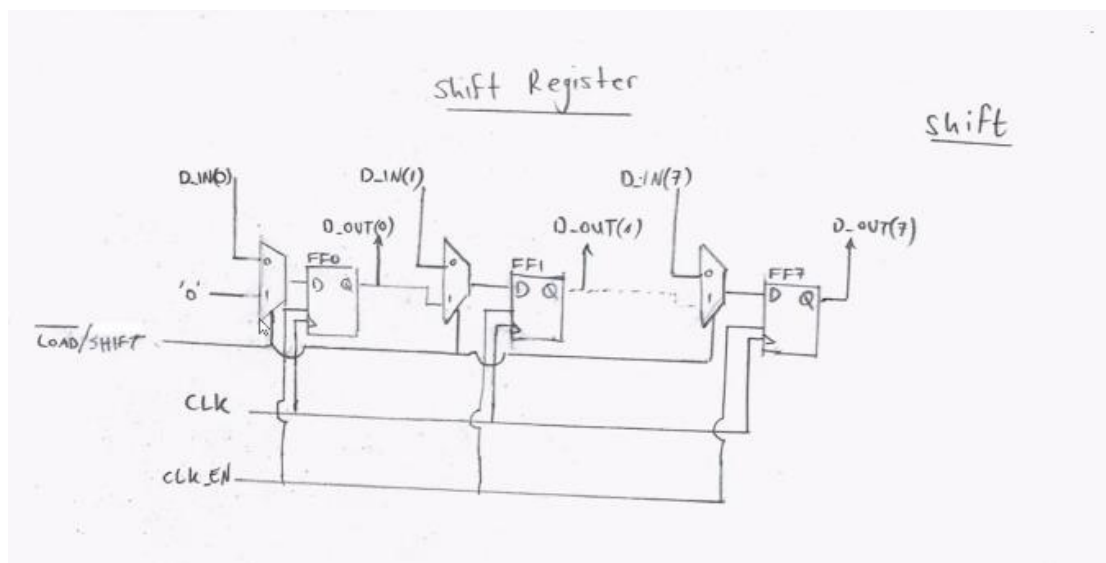
$$Y=\{a[0],a[4:1]\}$$

יש אפשרות לתכנן Barrel shifter, כלומר שלפי הכניסה של המוקס נקבע כמה הזזות נעשה:



ברכיב כזה יש דיליי שווה עבור כל ההזזות.

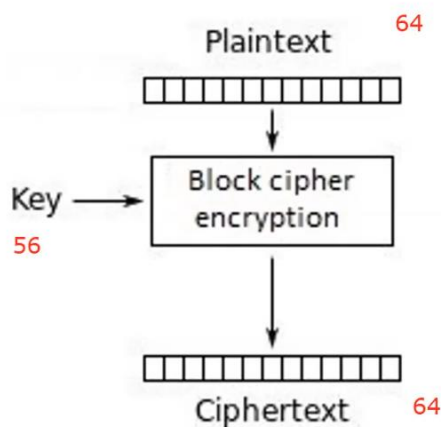
לעומת זאת אם משתמשים בשיפט רגיסטר:



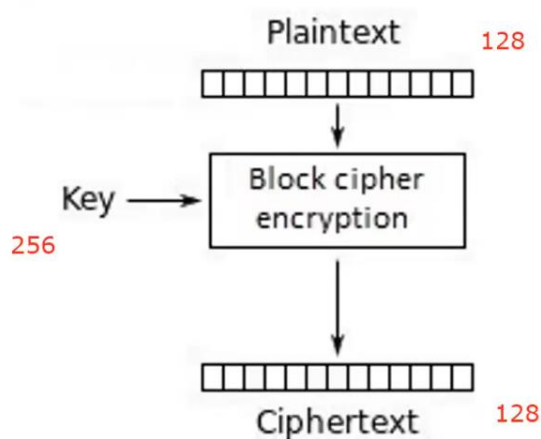
הוא לוקח הרבה יותר זמן (בהתאם לכל מספר הזזות) ויותר זכרונות (של FF).

שיעור 7

כך היה פעם:



כיום:



אלו הם גדלים טיפוסיים.
לא חייבים שהמפתח יהיה בגודל המידע, זה תלוי באלגוריתם.

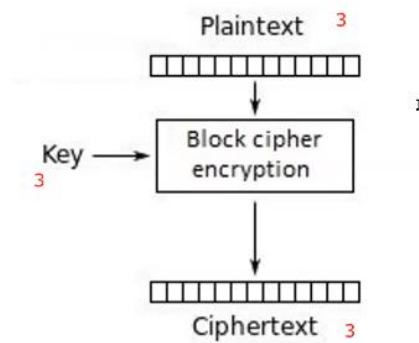
שאלה: למה א"א לעשות גדלים של 8, גדלים קטנים?

- איך מנחשים את P על סמך C?
כאשר יודעים את C ואת האלגוריתם אך לא יודעים את המפתח, מה עשים?

יש פתרון ע"י הרצת כל האפשרויות במפתח ובP - התקפת Brute Force "כוח גס"

	A	B	C	D	E	F	G	H	I	J	K	L	M
			Plaintext										
			0	1	2	3	4	5	6	7	...	255	
Key	0												
	1		6	7						6			
	2												
	3			6									
	4												
	5												
	6									7			
	7			7									
	...												
	255											6	7

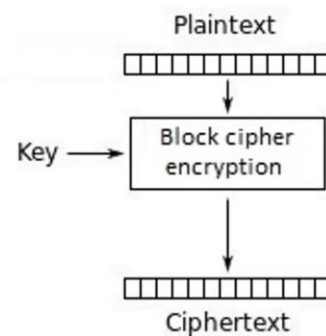
כדוגמה ניקח 3 ביט.



נבנה טבלה ונסתכל בC, נסכם את מספר האפשרויות שנקבל, לדוגמה 6, וכך נצמצם את האפשרויות לחיפוש. ונעשה ניסיונות התאמה ע"מ למצוא את המפתח. ככל שהטבלה גדלה, הסיבוכיות עולה וקשה לחשב. ככל שהטבלה קטנה יותר נוכל למצוא את המפתח המתאים.

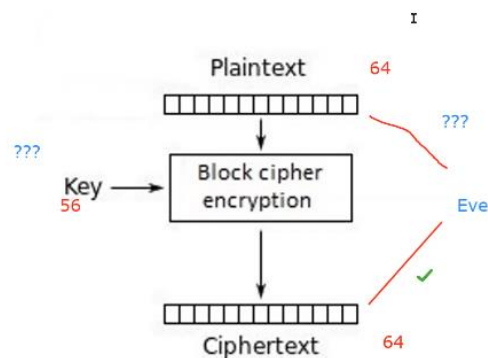
אלגוריתם DES:

DES = Data Encryption Standard

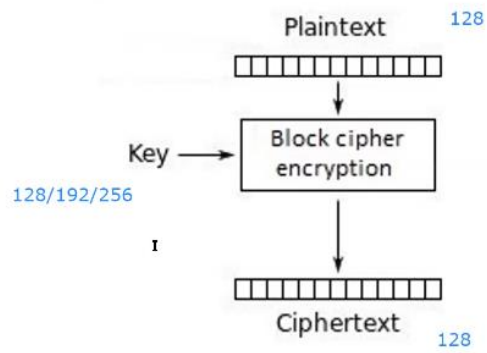


התוצאה: במסגרת Venona Project (בין 1943 ל-1980, פורסם רק ב-1995) חשפו שירותי הביון של ארה"ב תשדורות סובייטיות סודיות, כתוצאה משימוש לא נכון (שימוש חוזר במפתחות) והפילו את מרגלי האטום הסובייטים בארה"ב

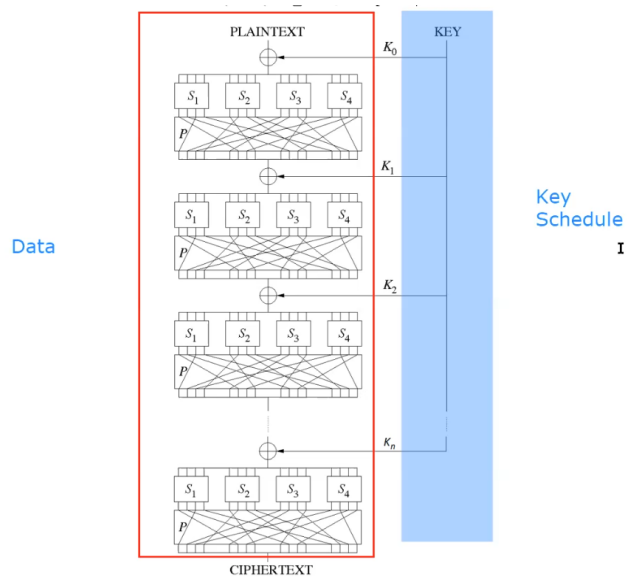
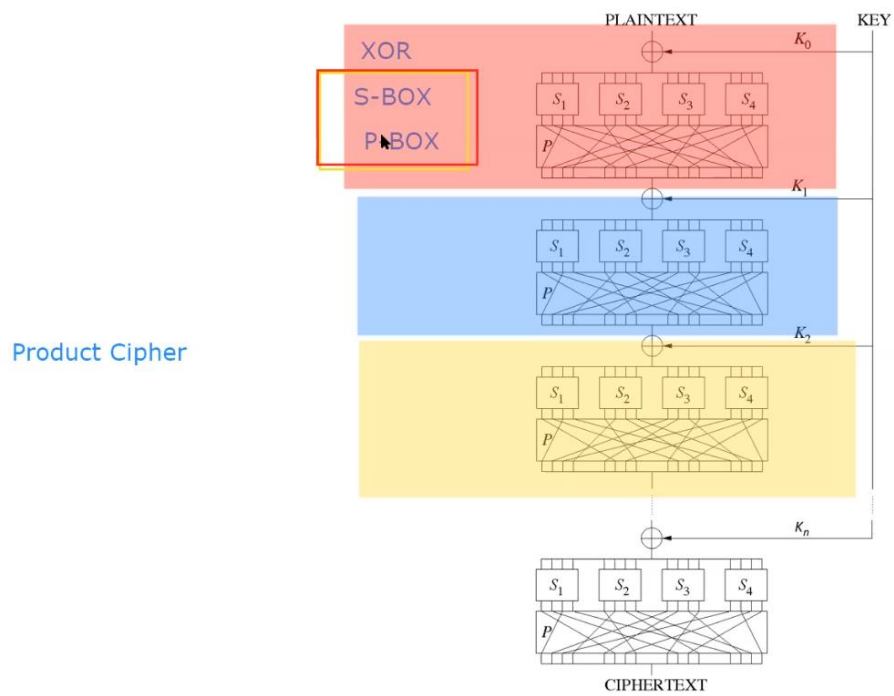
AES
Advanced
Encryption
Standard



כאן גודל הטבלה הוא $2^{54} \times 2^{64}$

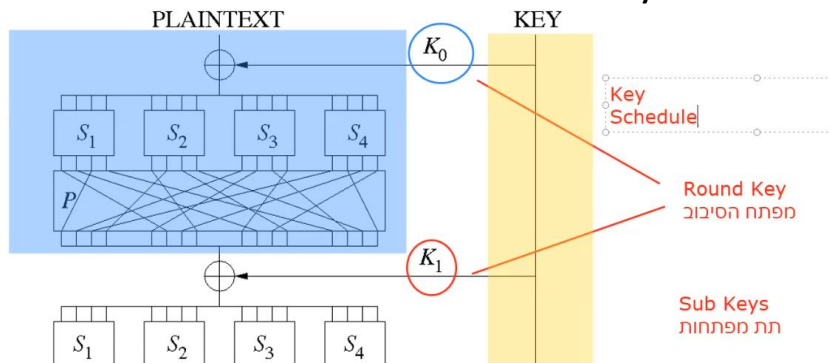


היום מפתח של 128 הוא קצת גבולי.



כל אלגוריתם והקדמה שלו.

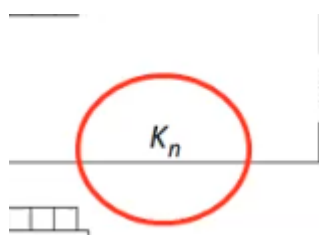
Round
סיבוב



יש n סיבובים.

ובכל סיבוב יש K אחר.

לכל סיבוב יש Key משלו.



Number of rounds
מספר סיבובים

חלק ממערכת הבטיחות של האלגוריתם.

לדוגמה באלגוריתם DES החליטו שיש לעשות 16 סיבובים (לפחות) ע"מ לשמור על התוצאה

מפני התקפות.

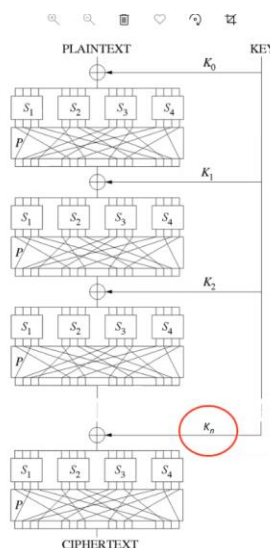
באלגוריתם GUST יש לעשות 32 סיבובים.

אפקט המפולת:

0000....0000

Number of rounds
מספר סיבובים

Af37de90B...



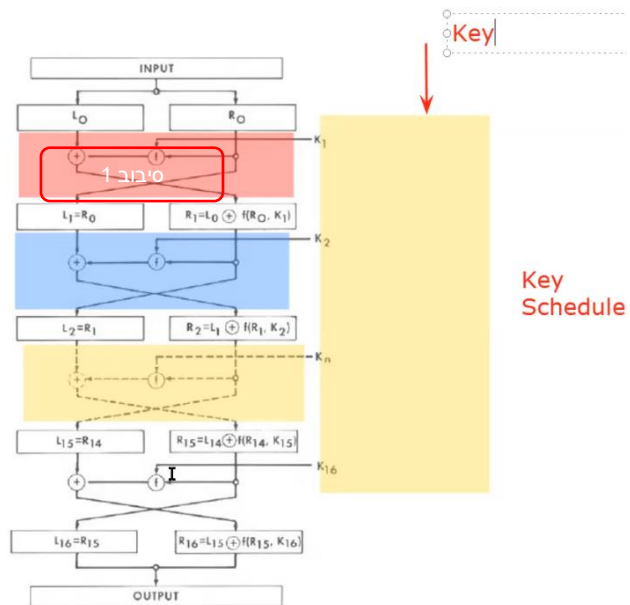
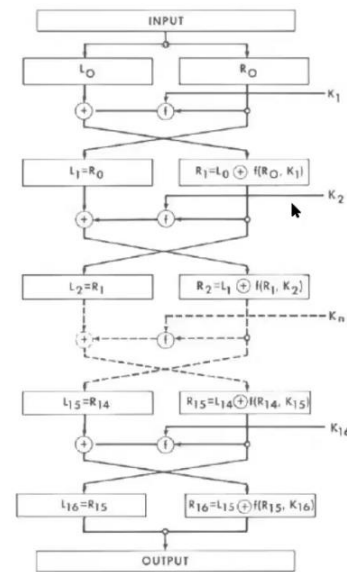
אפקט
המפולת

אפקט
כדור השלג

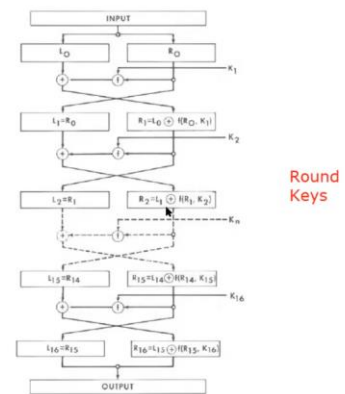
רוצים שהשינוי בין P ל C יהיה מקסימלי עבור שינוי מינימלי ב Key.

בפיענוח יש לעשות פעולת פרמוטציה הפוכה, S-BOX הפוכה, XOR הפוכה וכן הלאה.

:Feistel Network



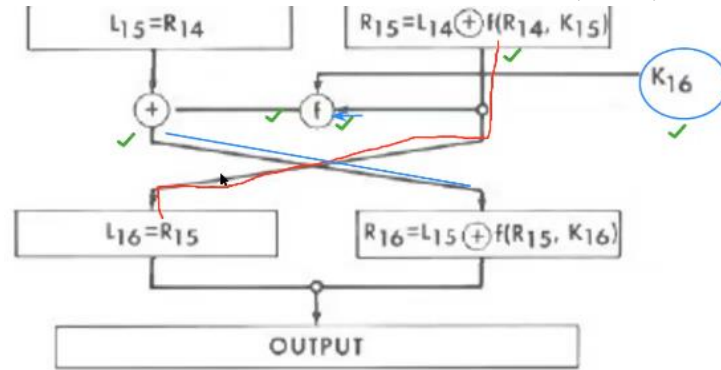
מידע פתוח
Plaintext



מידע מוצפן
Ciphertext

סיבוב אחד ברשת פיסטל לא מספיק כדי להצפין את המידע כי נעביר את אותו. ולכן צריך לפחות שני סיבובים.

- איך מבצעים פיענוח של רשת פיסטל?

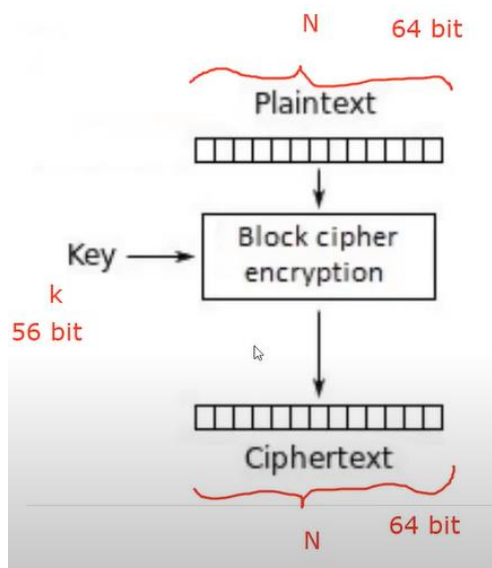


f היא לא הפיכה, אך נוכל לחשב אותה משום שיש לנו את המפתח K_{16} ואת R_{16} וכך נחשב את הפונק' באופן קידמי, נבצע את XOR ונגיע ל- L_{15} .

הצפנות
סימטריות
Symetric
Encryption

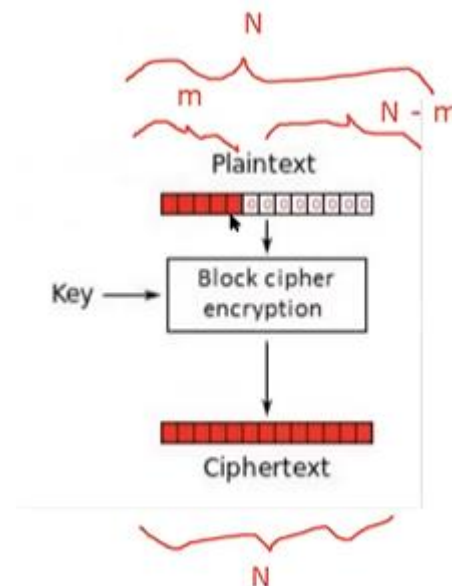
- שתי ההצפנות שראינו עכשיו הן סימטריות

עכשיו נלמד על Block cipher, כלומר מחכים שכל המידע יכנס ורק אז אפשר לעשות הצפנה (לעומת Stream cipher, ששם עושים ביט ביט):

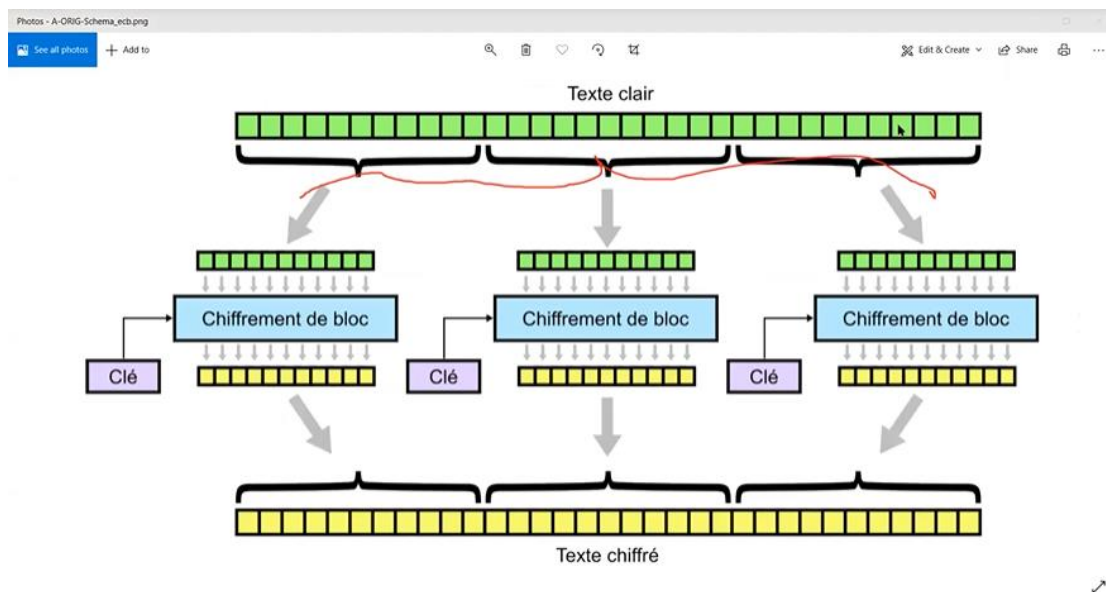


יש 2 מצבים שצריך לבדוק:
 א. שה PLAIN קטן יותר.
 ב. שהוא גדול יותר.

מצב א': נכנס לי נניח M ביטים שזה פחות מ- N . אבל התוצאה תמיד תצא N . לכן, כפתרון ראשוני **נרפד באפסים** את הכניסה (יש עוד שיטות שנראה בהמשך). אך כדי לדעת לפענח נצטרך לדעת מה גודלו האמיתי של המידע בכניסה לפני הריפוד:



מצב ב': נכנס לי W ביטים שזה נניח פי 3 מ- N . נפצל אותו לכמה מצפינים:

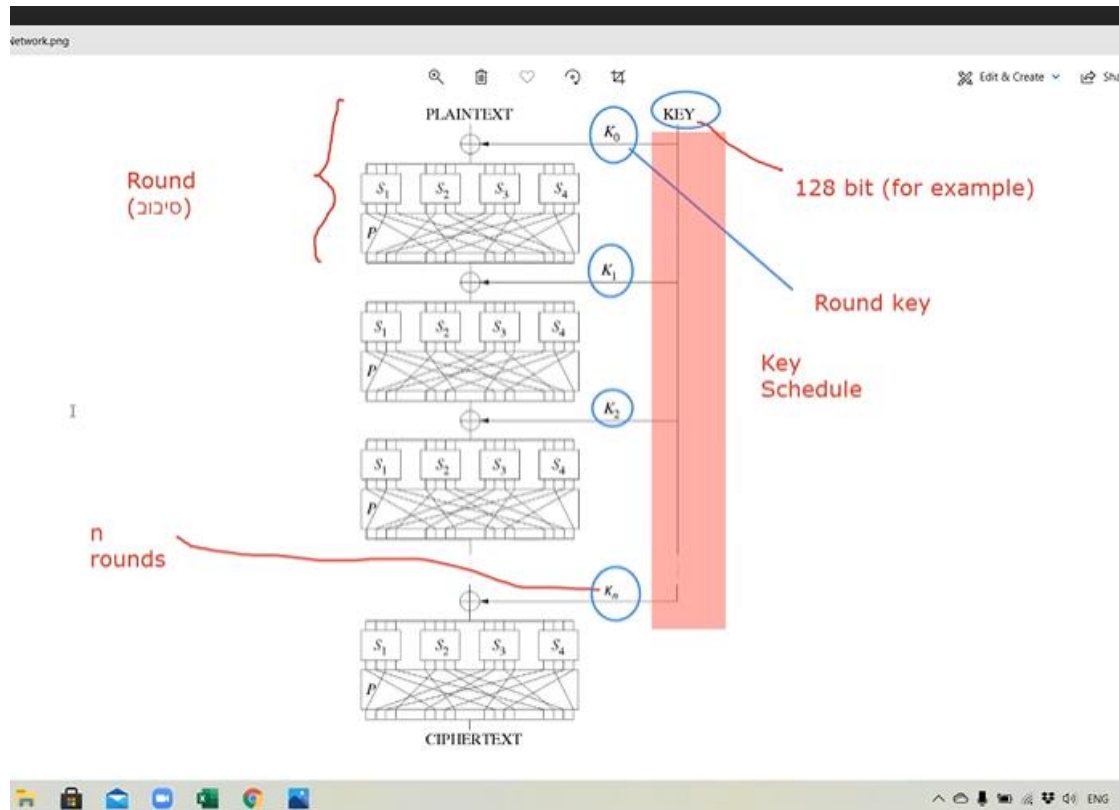


המבנה של ההצפנה:

ישנם כמה דרכים, אלה הנפוצות -

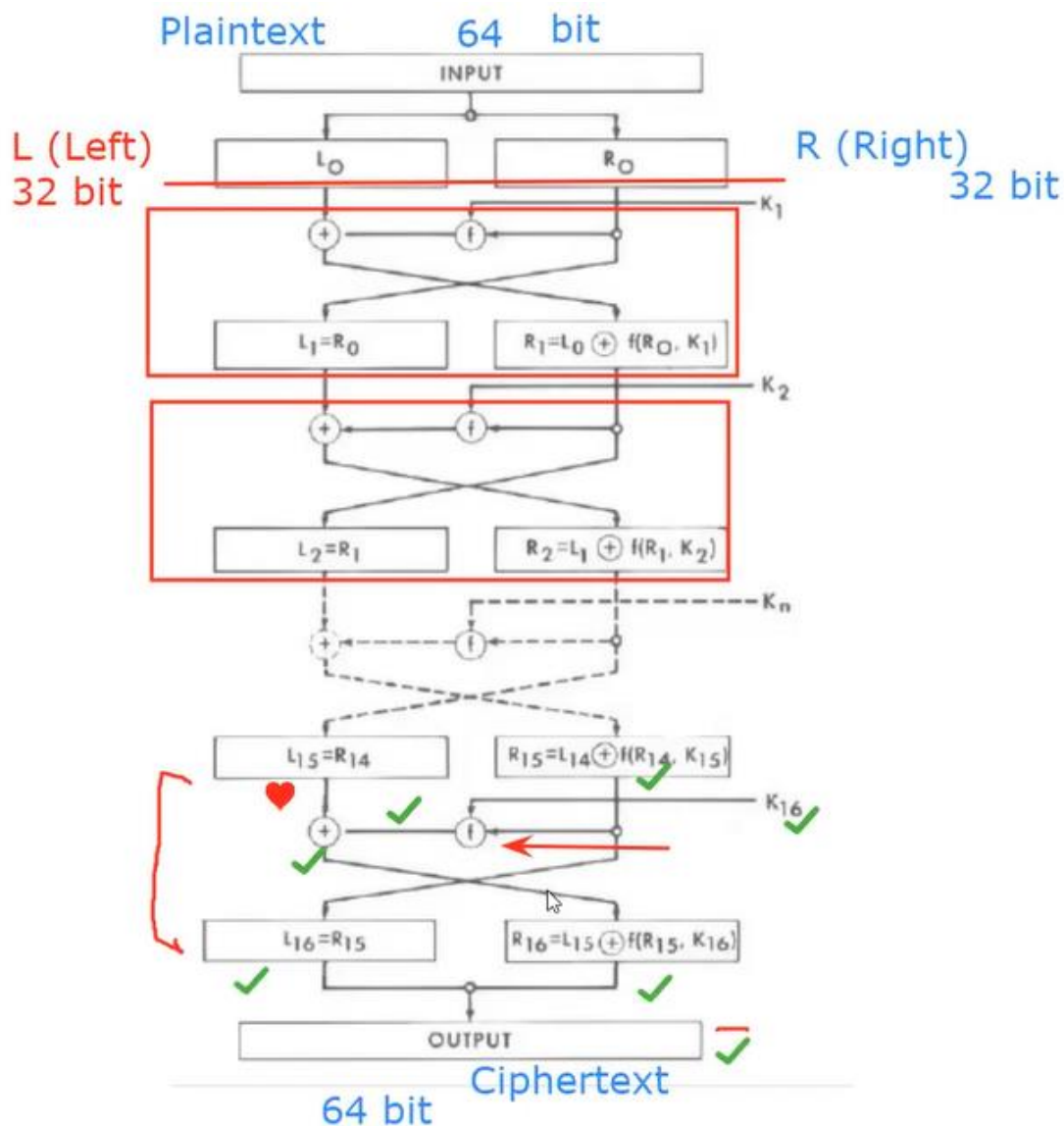
א. SP: המפתח עושה XOR עם המידע, ואז SBOX ואז פרמוט', כמה סבבים בהתאם ל???

(מה היחס בין גודל המפתח לסיבובים? מה זה קי סקדג'ואל?)



ככל שיהיה יותר סיבובים, כך אפקט המפולת יהיה חזק יותר, כלומר שינוי אפילו של בי אחד יגרור שינוי ענק במוצא.

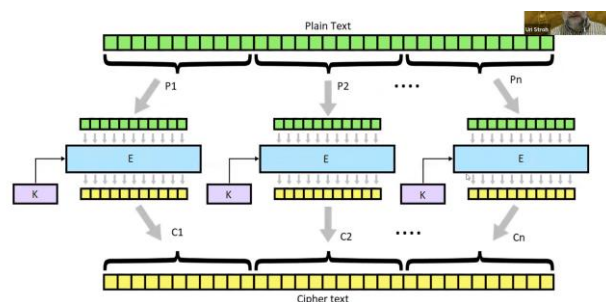
ב. **Feistel Network**: המידע מפוצל לחלק ימני ושמאלי. אחרי "סיבוב", הביטים הימניים יעברו שמאלה, והביטים הימניים יהיו קומבינציה ביניהם והמפתח, תלוי פונקציה (חד ערכית).



מספר הסיבובים – סיבוב אחד בוודאי לא יספיק לי, מכיוון שהצד הימני מגיע כמו שהוא לשמאלי. שוב, מספר הסיבובים הוא בהתאם למפולת שנרצה... יש כאלה עם 16/32. הערה- ככל שיהיו בלוקים גדולים יותר, יהיה יותר קשה לפצח את ההצפנה. כי עקרונית אפשר לעשות ניסוי וטעיה ולקבל את כל היציאות האפשריות ולבנות טבלה וכו'.... מקובל מידע 128 ומפתח 192 ביט.

שיעור 8

Plain Text שארוך יותר מאורך הבלוק (מה שיש בד"כ)

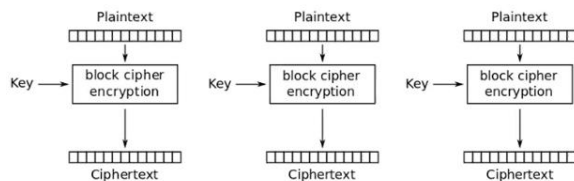


מחלקים לחתיכות ומצפינים כל חתיכה. וכך בפיענוח.

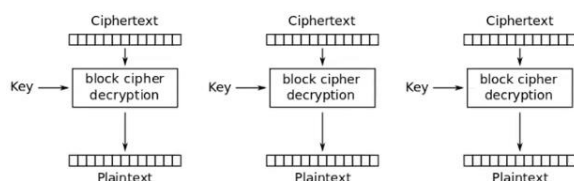
:ECB

Electronic codebook (ECB) [edit]

The simplest (and not to be used anymore) of the encryption modes is the **electronic codebook (ECB)** mode (named after conventional physical codebooks^[19]). The message is divided into blocks, and each block is encrypted separately.



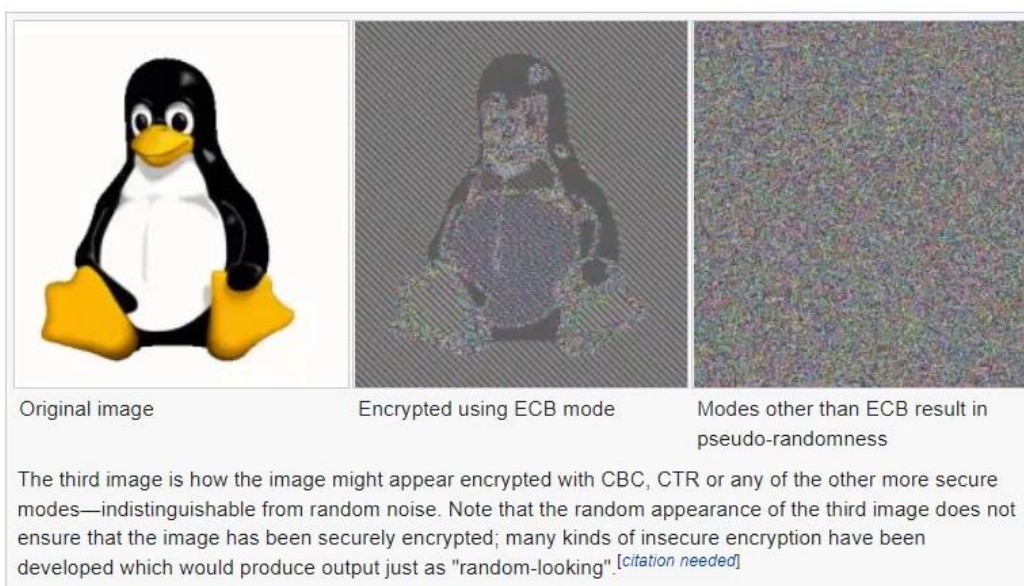
Electronic Codebook (ECB) mode encryption



Electronic Codebook (ECB) mode decryption

ECB	
Electronic codebook	
Encryption parallelizable	Yes
Decryption parallelizable	Yes
Random read access	Yes

ניתן לבצע הצפנה בצורה מקבילית.



כאן רואים הצפנה של התמונה, אולם בESB עדיין רואים את התמונה באופן די ברור, ויכולו לפענח ע"י אלגוריתמים של עיבוד תמונה, ללא קשר לפיענוח, את המידע, התמונה.

בשיטת הצפנה זו יש חסרון, המידע חוזר על עצמו, ההצפנה היא אותו דבר עבור פיסות מידע (צבע) דומות ולכן יכולו לעבד תמונה זו.

לכן הצפנה רק ע"י ECB היא לא מספיק טובה, ולכן ישנן גם דרכים אחרות.

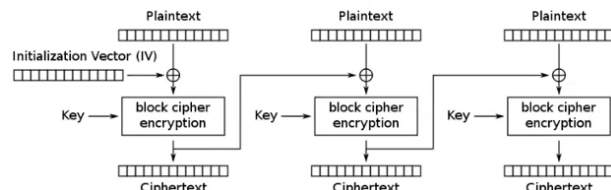
דרכי הצפנה נוספות:

ECB, CBC, OFB, CFB, CNTR, PCBC (Sensitivity to single errors, errors propagation)	הצפנת Block Cipher: אופני הפעלה (Modes of operation)
---	--

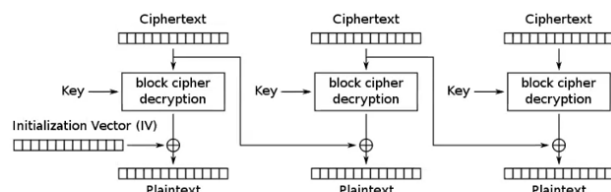
CBC:

Cipher block chaining (CBC) [edit]

Eharsam, Meyer, Smith and Tuchman invented the cipher block chaining (CBC) mode of operation in 1976.^[23] In CBC mode, each block of plaintext is XORed with the previous ciphertext block before being encrypted. This way, each ciphertext block depends on all plaintext blocks processed up to that point. To make each message unique, an initialization vector must be used in the first block.



Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

CBC	
Cipher block chaining	
Encryption parallelizable	No
Decryption parallelizable	Yes
Random read access	Yes

וכאן נדרשים להבין את השימוש במידע שהוא גדול מה-block cipher.

מה שעושים, לוקחים את פיסת המידע שרוצים להצפין ובפעולת xor עם מידע רנדומלי, המידע המוצפן, נצפין את פיסת מידע זו לאחר xor.

בבלוק הראשון אין פיסת מידע מוצפנת ע"מ לבצע xor ולכן יש וקטור שמור לשם כך והוא לא מפתח אלא קבוע, הוא חלק מהאלגוריתם והמצפין והמפענח צריכים לדעת אותו.

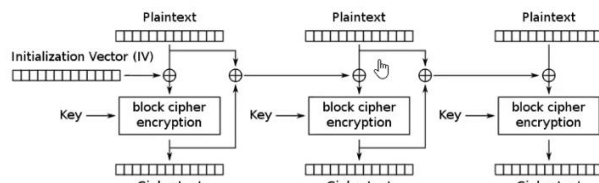
א"א לבצע בצורה מקבילית אלא טורית.

- כשישנה טעות קטנה – ECB נקבל לכלוך קטן בפיענוח.
- בCB נקבל תמונה לא ברורה כי הטעות נגררת לכל הפיענוח.
- בשתי שיטות אלו block cipher הוא אותו דבר אלא שהשימוש בו הוא שונה.

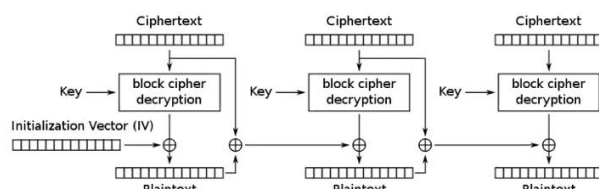
:PCBC

Propagating cipher block chaining (PCBC) [edit]

The *propagating cipher block chaining*^[25] or *plaintext cipher-block chaining*^[26] mode was designed to cause small changes in the ciphertext to propagate indefinitely when decrypting, as well as when encrypting. In PCBC mode, each block of plaintext is XORed with both the previous plaintext block and the previous ciphertext block before being encrypted. Like with CBC mode, an initialization vector is used in the first block.



Propagating Cipher Block Chaining (PCBC) mode encryption



Propagating Cipher Block Chaining (PCBC) mode decryption

PCBC	
Propagating cipher block chaining	
Encryption parallelizable	No
Decryption parallelizable	No
Random read access	No

לא משתמשים בשרשור זה הרבה משום כמה בעיות וגם הצליחו להתקיף שרשור זה.

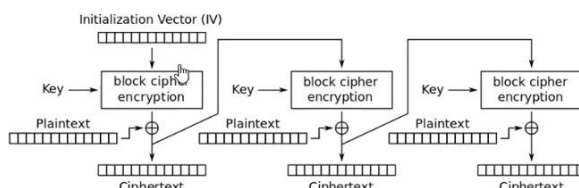
:CFB

Full-block CFB [edit]

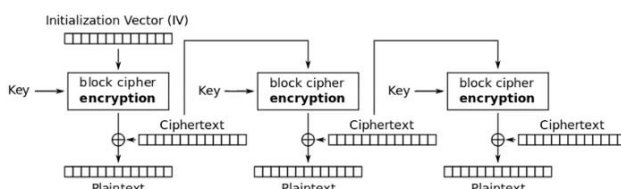
The *cipher feedback* (CFB) mode, in its simplest form uses the entire output of the block cipher. In this variation, it is very similar to CBC, makes a block cipher into a self-synchronizing stream cipher. CFB decryption in this variation is almost identical to CBC encryption performed in reverse:

$$C_i = \begin{cases} IV, & i = 0 \\ E_K(C_{i-1}) \oplus P_i, & \text{otherwise} \end{cases}$$

$$P_i = E_K(C_{i-1}) \oplus C_i,$$



Cipher Feedback (CFB) mode encryption



Cipher Feedback (CFB) mode decryption

CFB	
Cipher feedback	
Encryption parallelizable	No
Decryption parallelizable	Yes
Random read access	Yes

כאן ניתן לראות שהCiphertext הוא חלק מההצפנה וזה נעשה אחרי `xor`, ומכיון שיש צורך בהצפנה טורית יקח זמן רב יותר כי יש לחכות ל`xor`.

לכן בOFB לקחו את הרנדום שהוא לפני `xor`. וכך זמן העיבוד קטן יותר.

OFB

Output feedback (OFB)

The **output feedback** (OFB) mode makes a block cipher into a synchronous **stream cipher**. It generates **keystream** blocks, which are then XORed with the plaintext blocks to get the ciphertext. Just as with other stream ciphers, flipping a bit in the ciphertext produces a flipped bit in the plaintext at the same location. This property allows many error-correcting codes to function normally even when applied before encryption.

Because of the symmetry of the XOR operation, encryption and decryption are exactly the same:

$$C_i = P_i \oplus O_i,$$

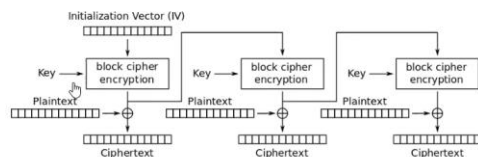
$$P_i = C_i \oplus O_i,$$

$$O_i = E_K(I_i),$$

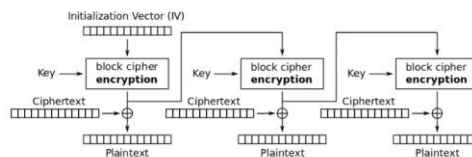
$$I_i = O_{i-1},$$

$$I_0 = IV.$$

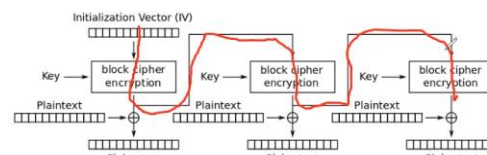
OFB	
Output feedback	
Encryption parallelizable	No
Decryption parallelizable	No
Random read access	No



Output Feedback (OFB) mode encryption



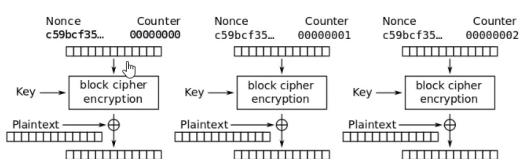
Output Feedback (OFB) mode decryption



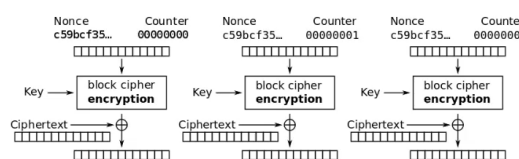
מסלול הרנדום הוא עצמאי.

CTR

CTR	
Counter	
Encryption parallelizable	Yes
Decryption parallelizable	Yes
Random read access	Yes



Counter (CTR) mode encryption



Counter (CTR) mode decryption

את הרנדום מייצרים ע"י מונה שמשתנה עבור כל פיסת מידע.

יש כאן בעיה חמורה – כאשר יש אינפורמציה שכולה או רובה אפסים נוצרת חולשה מבחינת block cipher משום שבהתקפות דיפרנציאליות, שינוי של ביט אחד, אפשר לפענח את המידע. ואז הרנדום הוא לא מספיק חזק כי אפשר לפצח אותו. הבעיה נעשית יותר חמורה כאשר יש יותר אפסים, במונה של 64 bit זו בעיה, ובעיה גדולה יותר במונה של 128 bit.

לשם כך יצרו את Nonce שהוא קבוע בחלקו הימני של המונה ולפי המימוש קובעים את אורכו. וכך הבעיה קטנה יותר משמעותית.

תכנון ברמת הבלוק: מימוש בחומרה.

שנה"ל תשע"ח, סמסטר ב', מועד א'

שאלון בחינה בקורס: אלגוריתמים בקריפטוגרפיה ומימוש ב-Verilog
מספר קורס: 130330

שאלה 4: [20 נקודות]

Block Cipher - תכנון ברמת הבלוק

נתון Block Cipher ברוחב מידע 128 ביט וברוחב מפתח 256 ביט. האלגוריתם להצפנה (E) והאלגוריתם לפענוח (D) נתונים וממומשים

ענה על השאלות הבאות המתייחסות לתכנון חומרה המשתמשת במנגנון זה:

א. [5 נקודות] שרטט סכמה כללית של תהליך הצפנה במוד OFB

ב. [5 נקודות] שרטט סכמה כללית של תהליך פענוח במוד OFB

ג. [4 נקודות] הסבר ונמק את השווה ואת השונה בין הסכמות ששרטטת בסעיפים א' ו-ב'

ד. [6 נקודות] שרטט דיאגרמת בלוקים של מימוש חומרה משולב המבצע הצפנה ופענוח במוד OFB

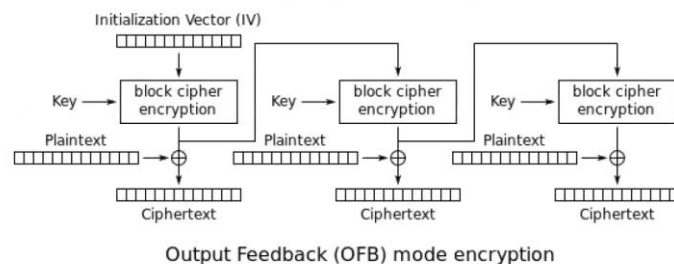
שים לב: הכניסות הפונקציונליות של המימוש המשולב הן:

- DATA_IN[127:0]
- DATA_OUT[127:0]
- KEY_IN[255:0]
- סיגנל Encrypt_Decrypt לבחירה בין הצפנה לבין פענוח
- סיגנל First_Block המציין אם כרגע מטופל הבלוק הראשון
- לשם פשטות הדיאגרמה: אין צורך לכלול בשרטוט כניסות בקרה נוספות כגון reset, clock וכד'

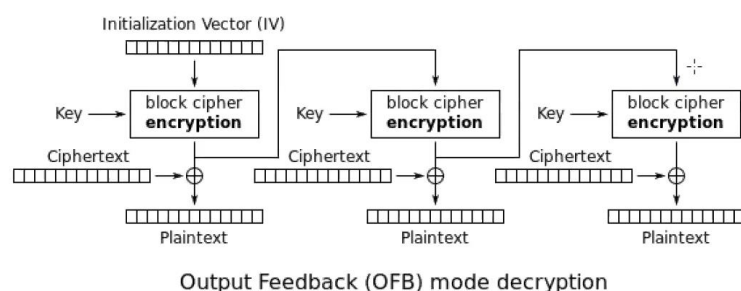
תשובה 4: [20 נקודות]

Block Cipher - תכנון ברמת הבלוק

א. [5 נקודות] סכמה כללית של תהליך הצפנה במוד OFB



ב. [5 נקודות] סכמה כללית של תהליך פענוח במוד OFB

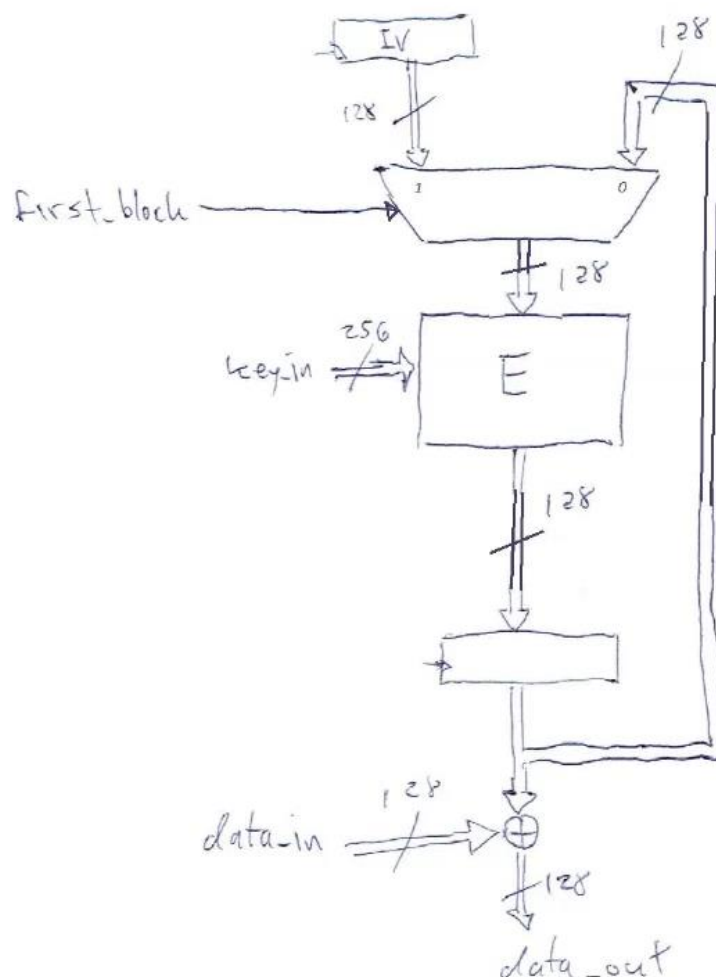


ג. [4 נקודות] 2 הסכמות זהות זו לזו. ההבדל הוא בהחלפת כניסות Plaintext ו-Ciphertext בין הסכמות.

נימוק והסבר: אופן הפעלה של Block Cipher במוד OFB מתבסס על "שרשרת" של ייצור מידע אקראי (random) מה-initial value באמצעות מצב Encrypt של ה-Block Cipher. ההצפנה עצמה מבוססת על פעולת XOR בין ה-random לבין המידע הנכנס, בתצורה של Stream Cipher.

בהינתן שפעולת XOR היא הפיכה, הרי שאופן הפעלה של OFB זהה, הן להצפנה והן לפיענוח, וההבדל הוא רק אם כניסת data_in תקבל את ה-Plaintext (בהצפנה) או את ה-Ciphertext (בפיענוח).

ד. [6 נקודות] דיאגרמת בלוקים למימוש OFB משולב להצפנה ופיענוח (לא נדרש סיגנל Encrypt_Decrypt):



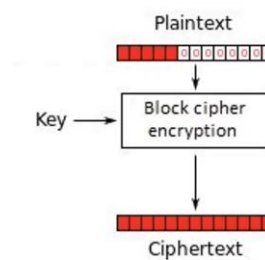
זה מימוש של block cipher אחד. אפשר להכפיל מימוש זה הרבה פעמים וזה לוקח הרבה aera.

מימוש זה ניתן לממש בverilog.

בלוק מידע לא מלא:

אורך המידע לא כאורך הבלוק.

ריפוד באפסים:



יש כאן שתי בעיות: 1. בעיית אבטחה 2. בעיית מקום.

אחסון – נקבל Ciphertext ארוך יותר מהמידע.

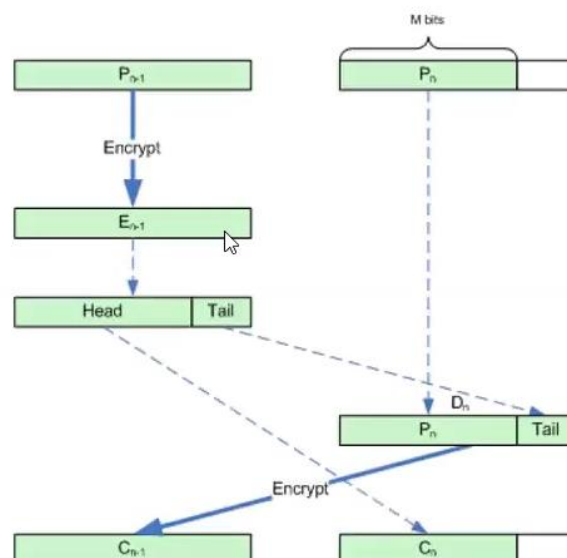
לשם פתרון לבעיית האבטחה נרפד בNonce. את פתרון המקום לא פתרנו.

עבור פתרון האחסון – נבצע הצפנה לשארית, Residue, למידע הקטן מהבלוק, וכך אין צורך בהגדלת הבלוק.

כביכול זה לא פתרון טוב משום שפתרנו את בעיית המקום אך יש מידע שהוא לא מוצפן ונוצרת בעיית אבטחה (בחלק מהמקרים).

זה מביא אותנו לפתרון נוסף

CTS:

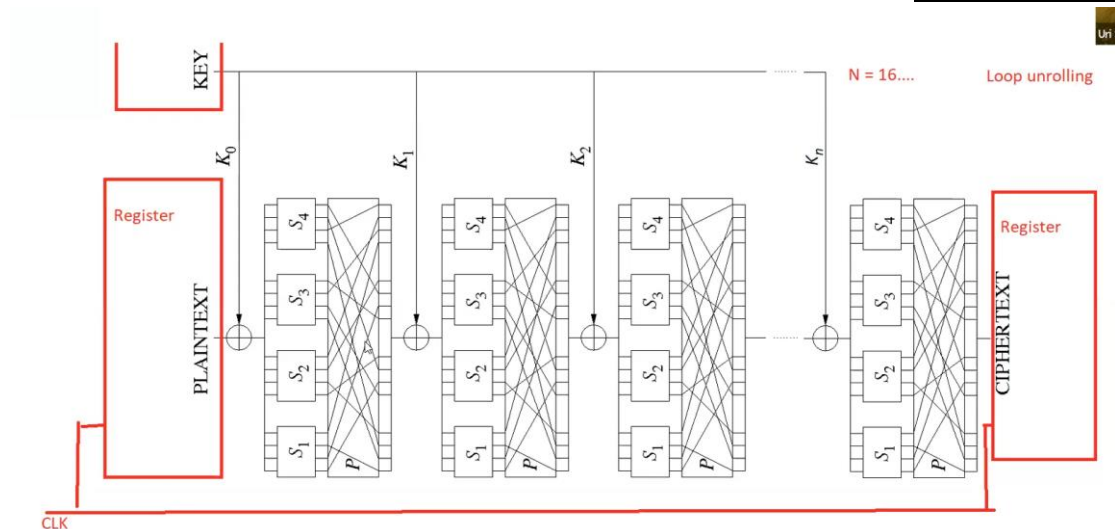


בבלוק האחד לפני האחרון נחלק את המידע המוצפן ל2 ומשם נמשיך.

זה לוקח קצת יותר זמן אך כל המידע מוצפן.

מימוש בחומרה של רשת Feistel או SP:

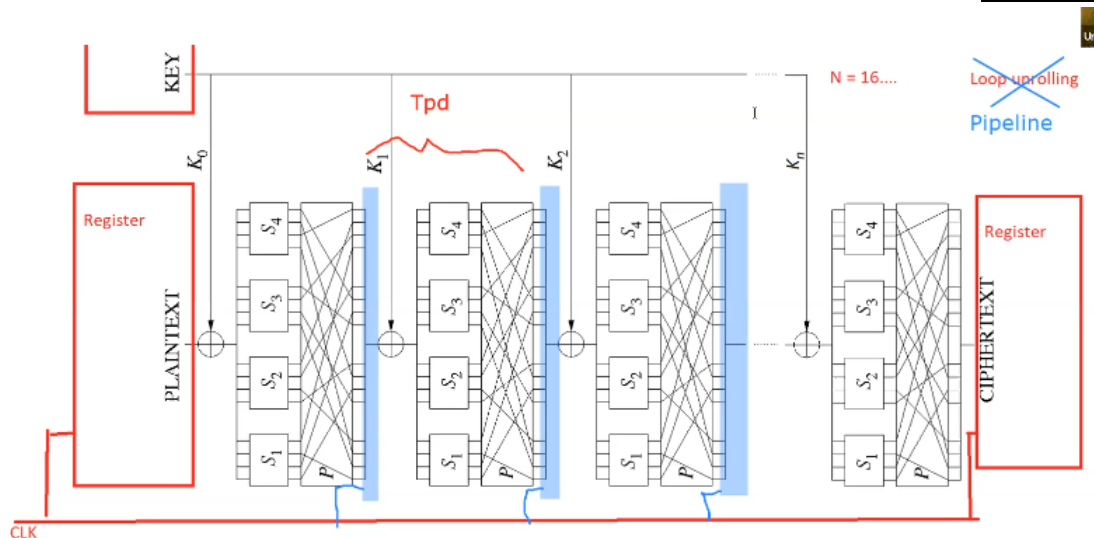
:Loop Unrolling



במימוש זה ישנן 2 בעיות – 1. בעיית זמנים – עבור כל לוגיקה של SBOX יש דילי וכן כפול מספר הלוגיקות שיש.
2. בלוגיקה עם הרבה לוגיקות ישנה בעיית area. זו בעיה שתעלה הרבה כסף.

לכן נממש בשיטה אחרת

:Pipeline



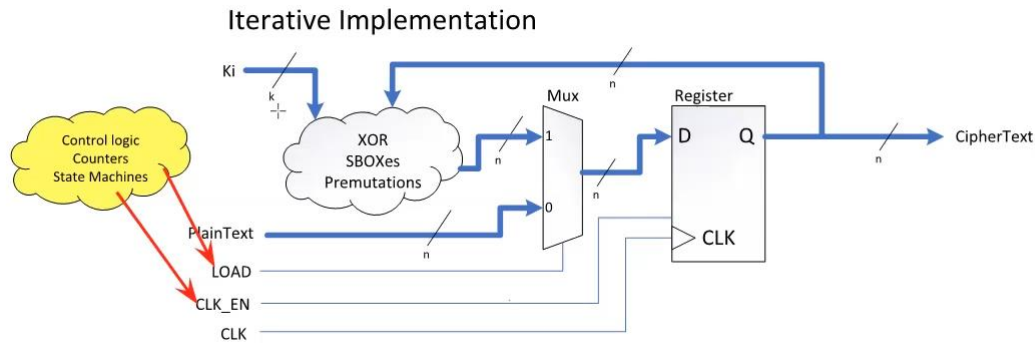
נוסיף רגיסטר אחרי כל לוגיקה של SBOX.

הרווחנו זמן אך שילמנו בarea.

Comparison Between Different Hardware Implementations of SP/Feistel Network Block Ciphers		
N = number of rounds		
	Loop Unrolling	Pipeline Implementation
Area	$N \times \text{Area}(\text{Round Logic})$	$N \times \text{Area}[(\text{Round Logic}) + \text{Register}]$
Performance (Clock Cycles)	1 Cycle	N Cycles
Timing (Critical Path)	$N \times \text{Tpd}(\text{Round Logic})$	$\text{Tpd}(\text{Round Logic})$
Continuous Operation	Yes	Yes
Note		

היתרון של שתי שיטות אלו הוא פשטות.

המימוש הבא עוזר לנו לא לשכפל את הלוגיקה n פעמים ופותר את הבעיות.



אלא שיש להוסיף עוד לוגיקה חיצונית שמנהלת את המימוש הזה, כגון מכונת מצבים.

Comparison Between Different Hardware Implementations of SP/Feistel Network Block Ciphers			
N = number of rounds			
	Loop Unrolling	Pipeline Implementation	Iterative Implementation
Area	$N \times \text{Area}(\text{Round Logic})$	$N \times \text{Area}[(\text{Round Logic}) + \text{Register}]$	$1 \times \text{Area}[(\text{Round Logic}) + \text{Register}] + \text{Mux}$
Performance (Clock Cycles)	1 Cycle	N Cycles	$(1 \text{ Load} + N) \text{ Cycles}$
Timing (Critical Path)	$N \times \text{Tpd}(\text{Round Logic})$	$\text{Tpd}(\text{Round Logic})$	$\text{Tpd}(\text{Round Logic}) + \text{Tpd}(\text{Mux})$
Continuous Operation	Yes	Yes	No (Wait N cycles for next block load)
Note			Significant control logic, counters and S.M.

נתחיל להתבונן על מימוש אמיתי

מימוש DES:

FIPS PUB 46-3

FEDERAL INFORMATION
PROCESSING STANDARDS PUBLICATION

Reaffirmed
1999 October 25

U.S. DEPARTMENT OF COMMERCE/National Institute of Standards and Technology

תיאור החומרה הוא רק בעמודים 12-13, אם כך, מה יש עד עמודים אלו?
למה משרד התעשייה אמון על האלגוריתם? שכל אחד יעשה מה שהוא רוצה?
למה צריך תקן להצפנה?

ע"מ שיוכלו לעבוד יחד בארה"ב וכולם יוכלו להשתלב בתעשייה ולא שכל המידע יהיה בחברה אחת.

עדיין זה אלגוריתם הצפנה, למה לפרסם?
אלא, שהעיקר הוא לא האלגוריתם אלא המפתח, key.

יש לך גם סיבות רעות -

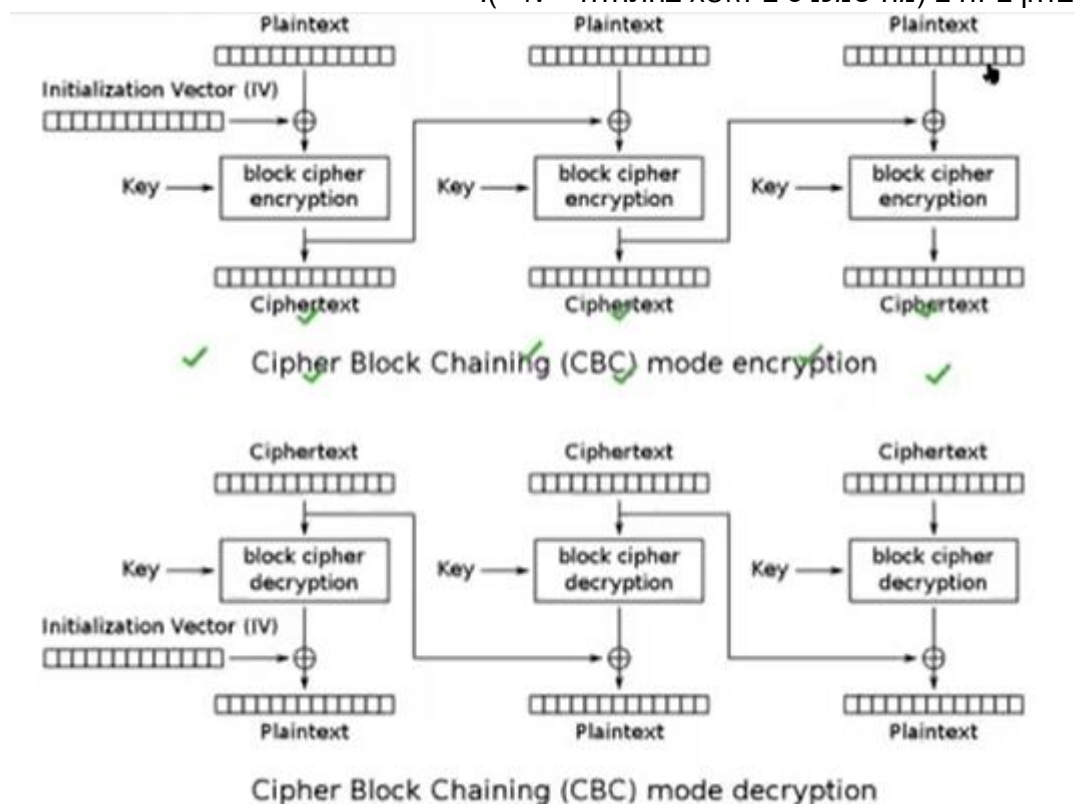
➤ התוצאה: במסגרת Venona Project (בין 1943 ל- 1980, פורסם רק ב- 1995) חשפו שירותי הביון של ארה"ב תשדורות סובייטיות סודיות, כתוצאה משימוש לא נכון (שימוש חוזר במפתחות) והפלילו את מרגלי האטום הסובייטים בארה"ב

כשמשלת ארה"ב נותנת את האלגוריתם, היא מן הסתם גם יודעת את הפרצות באלגוריתם ותכנס בדרכים שיש רק לה. ולכן נותנת חוקים לך.

אופני הפעלה:

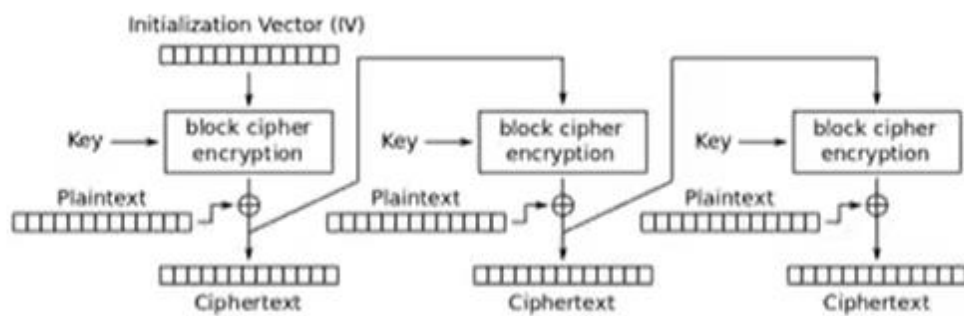
1. מה שראינו עד עכשיו היה EBC. כלומר הכל קורה דיגיטלי. אך יש בעיה כאשר יש רקע אחיד למשל (פינגווין).

2. CBC: כאן כל בלוק שמוצפן תלוי במידע המוצפן שיצא לפניו, ואז יש שינוי בהצפנה גם לבלוקים זהים (מה שמכניסים ל XOR בהתחלה - IV -):

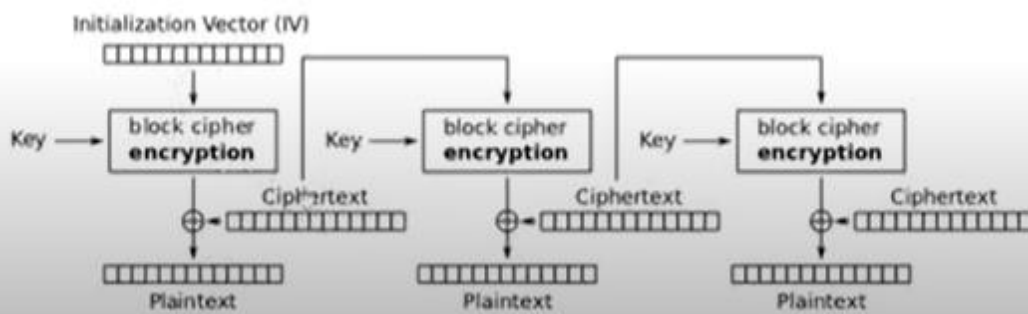


הבעיה כאן שברגע שיש טעות אחת אז הרעש הזה ישפיע על כל הבלוקים.

3. CFB: עיקרון דומה לקודם, רק שפה כל פעם 'מצפינים' את מה שכבר מוצפן ואז עושים XOR עם המידע החדש. דומה קצת ל'one time pad'.



Cipher Feedback (CFB) mode encryption



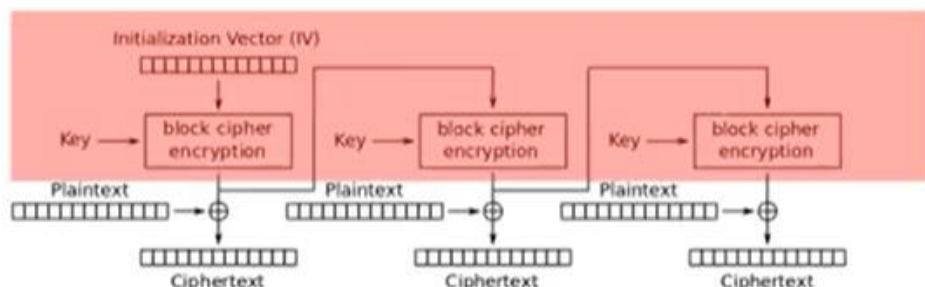
Cipher Feedback (CFB) mode decryption

פה גם יש גרירת טעויות אפשרית.

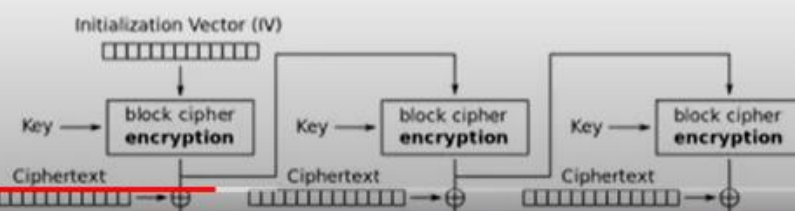
*האם יש דרך להקדים את ההצפנה של הבלוק הבא? ע"י:

4. OFC: כאן מעבירים הלאה את מה שיוצא מהפונקציה, ולא מחכים XOR בינו לבין המידע.

כך זה יותר מהיר ואפשר להתקדם במקביל:



Output Feedback (OFB) mode encryption

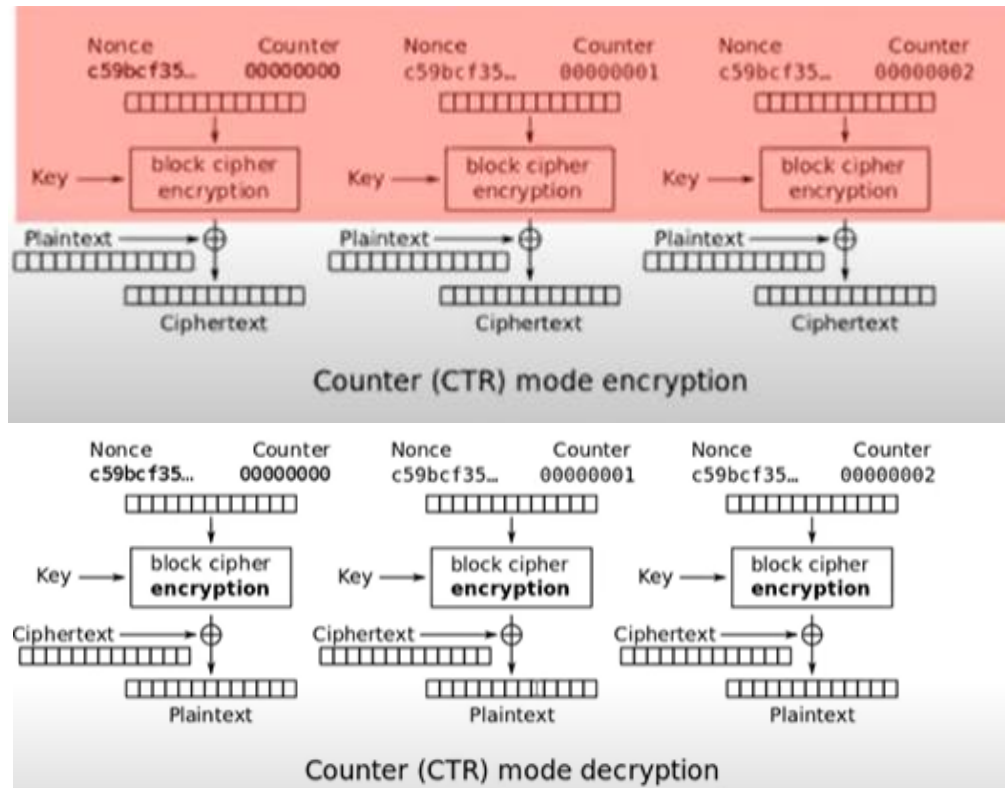


כאן רק טעות בפונקציה תגרור טעות נוספת.

5. CTR: כאן חוסכים את ההעברה של התוצאה של הפונקציה, ועושים פשוט הפעלה על

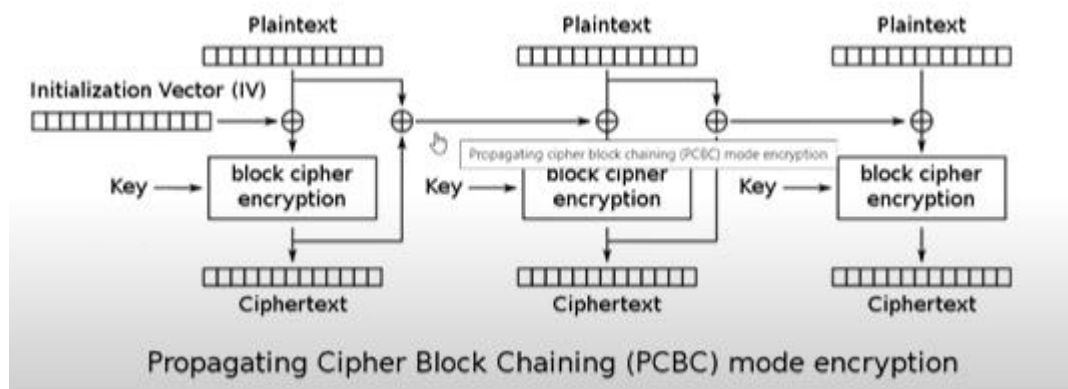
CNT. ואז לא תהיה גרירת טעויות כלל, ומצד שני גם בלוקים זהים יקבלו הצפנה שונה.

שילוב של 1 ו-4.

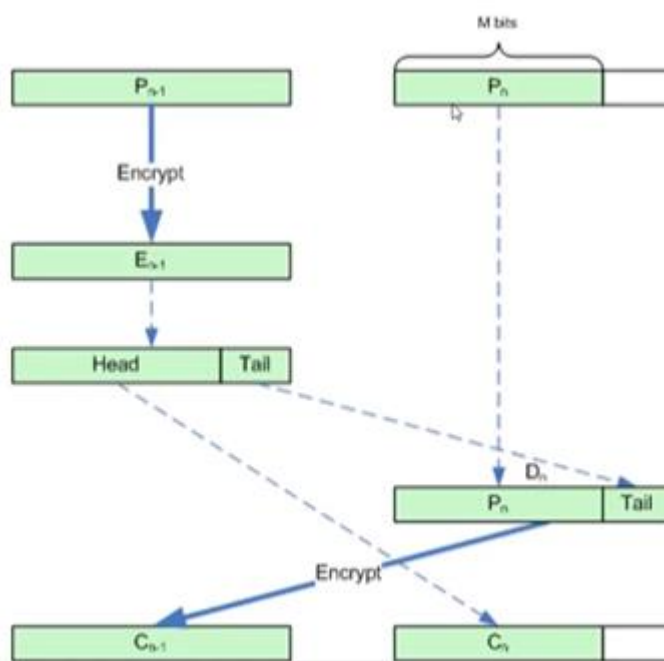


הבעיה ש-מונה זה דבר איטי, והצפנות נעשות מאוד מהר. דבר נוסף שזה קל יותר לפריצה כי אין הרבה משתנים, כי המונה ידוע... (אפשר לשים קבוע בהתחלה של המונה (Nonce) שגם הוא סוג של מפתח, וזה פותר קצת את הבעיה).

6. PCBC: שדרוג של CBC, שעושים XOR גם עם המידע הריק:

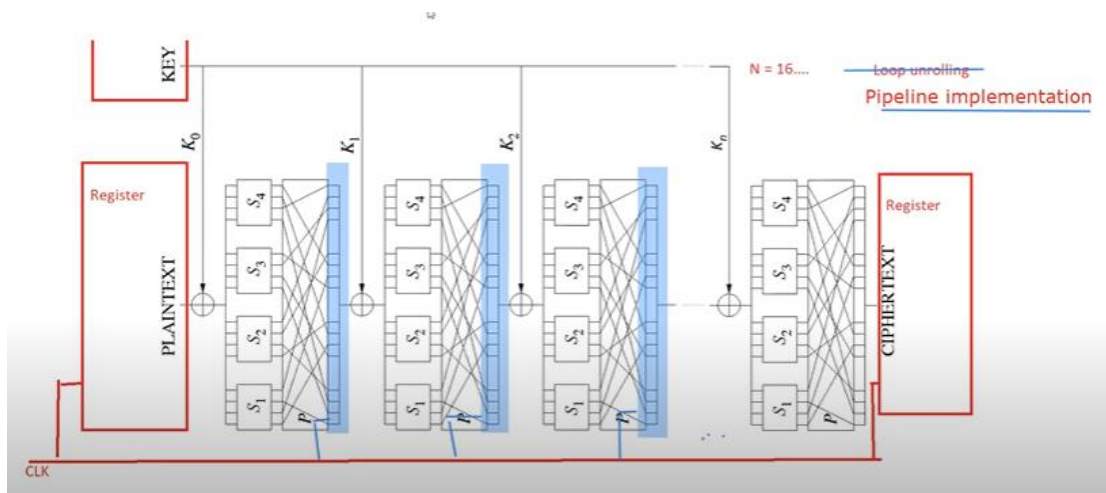


נחזור לבעיית הגדלים, שפתרנו בעזרת ריפוד באפסים. יש בזה בעיה, כי החוזק של ההצפנה יורד, כי תמיד מוסיפים אפסים. פתרון – להוסיף יזה קבוע. בעיה – לוקח הרבה זיכרון. פתרון- גניבת הצפנה "ciphertext stealing":

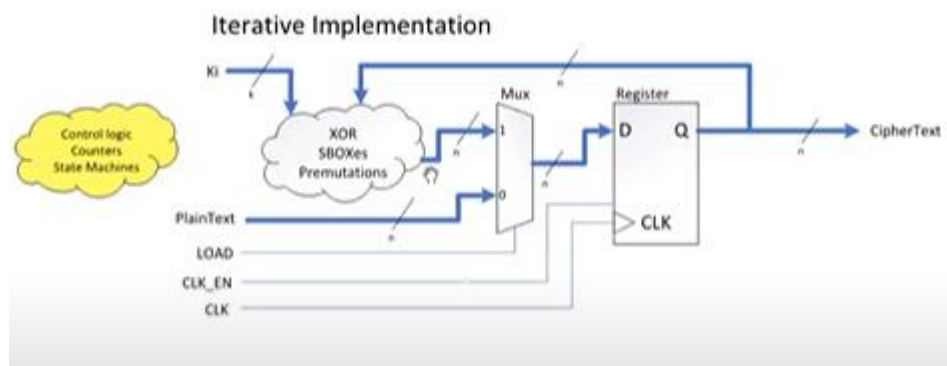


גונבים חלק מהמידע שכבר הוצפן בבלוק הקודם, ואותו מכניסים למידע הריק החדש ומרפדים איתו.
(לא הבנתי למה אח"כ מחליפים מיקומים שלהם, משהו שקשור לזה שלא צריך לאחסן הכל??).

נחזור לSP, כלומר המבנה של ההצפנה (עבור כל בלוק בודד).
רואים שיש פה הרבה פעולות עבור כל הצפנה, כלומר עלולה להיות פה בעיית זמנים רצינית (וגם AREAD, אבל נעזוב אותה כרגע). כי בין שתי FF של המידע הריק והמידע המוצפן יש המון פעולות שחוזרות על עצמם כדי לחזק את אפקט המפולת.
פתרון אפשרי – PIPELINE, שזה סוג של Retiming, כלומר לשים רגיסטרים אחרי כל סיבוב:



פתרון נוסף שיפתור גם את בעיית AREAD:



שמים את הלוגיקה פעם אחת, וחוזרים עליה N פעמים בהתאם להצפנה. צריך לעשות פה איזה יחידת שליטה (למשל מכונת מצבים) שתחליט האם מכניסים את המידע הנקי או את מה שיוצא מה FF. יש לשיטה זו חסרון אחד מרכזי, שאי אפשר להכניס מידע חדש עד שמסתיימת כל ההצפנה, מה שלוקח הרבה מחזורי שעון. (בניגוד למימוש הבסיסי או PIPELINE ששם אפשר.)

טבלת הבדלים בין שלושת השיטות:

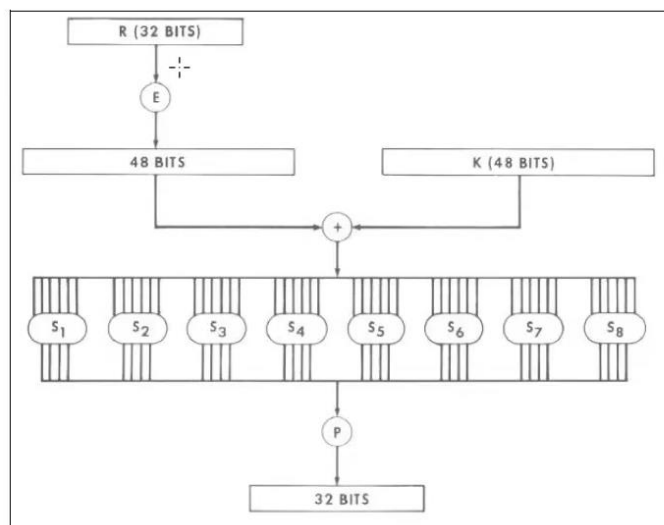
	Loop Unrolling	Pipeline Implementation	Iterative Implementation
Area	$N \times (\text{Round Logic})$	$N \times [(\text{Round Logic}) + \text{Register}]$	$1 \times [(\text{Round Logic}) + \text{Register} + \text{Mux}]$
Performance (Clock Cycles)	1 Cycle	N Cycles	$(1 \text{ Load} + N)$ Cycles
Timing (Critical Path)	$N \times \text{Tpd}(\text{Round Logic})$	$\text{Tpd}(\text{Round Logic})$	$\text{Tpd}(\text{Round Logic}) + \text{Tpd}(\text{Mux})$
Continuous Operation	Yes	Yes	No (Wait N cycles for next block load)
Note			Significant control logic, counters and S.M.

שיעור 9

נמשיך לדבר על מימוש DES.

A DES key consists of 64 binary digits ("0"s or "1"s) of which 56 bits are randomly generated and used directly by the algorithm. The other 8 bits, which are not used by the algorithm, may be used for error detection. The 8 error detecting bits are set to make the parity of each 8-bit byte of the key odd, i.e., there is an odd number of "1"s in each 8-bit byte¹. A TDEA key consists of three DES keys, which is also referred to as a key bundle. Authorized users of encrypted computer data must have the key that was used to encipher the data in order to decrypt it. The encryption algorithms specified in this standard are commonly known among those using the standard. The cryptographic

זה מימוש שיש בו 64 ביט אך רק 56 ביטים בשימוש וזה אורך key.



כאן יש הריבה מ32 ביט ל48 ואח"כ צמצום חזרה ל32 ביט.
ההרחבה ל48 ביט היא פרמוטציה לא הפיכה.
ובהרחבה זו ישנם מספר ביטים שיעברו פעמיים.

Let E denote a function which takes a block of 32 bits as input and yields a block of 48 bits as output. Let E be such that the 48 bits of its output, written as 8 blocks of 6 bits each, are obtained by selecting the bits in its inputs in order according to the following table:

<u>E BIT-SELECTION TABLE</u>					
32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

את הטבלה א"א להפוך.
גם הSBOXים לא הפיכים, כי נכנסים 48 ביטים ויוצאים 32.

הרבה חחשו שממשלת ארה"ב משאירה לעצמה דלת אחורית בSBOXים כי הכל ידוע
חוץ מאיך הSBOXים פועלים.
ולכן כשיש SBOXים יש לפרסם את נוסחתם, כדי להוכיח את תום הלב ושאינן כוונת זדון
לדלת אחורית.
(א"א לפרסם אלגוריתם ולשים אלגוריתם אחר משום שבדקים את זה. אלא להפך,
הפרסום המתמטי של האלגוריתם הוא לא דבר רע)

:Tripe DES

1. TDEA encryption operation: the transformation of a 64-bit block I into a 64-bit block O that is defined as follows:

$$O = E_{K3}(D_{K2}(E_{K1}(I)))$$
2. TDEA decryption operation: the transformation of a 64-bit block I into a 64-bit block O that is defined as follows:

$$O = D_{K1}(E_{K2}(D_{K3}(I)))$$

משתמשים ב3 מפתחות בשביל output, 56X3 ביט = 168.
ובשביל להפוך עושים את זה הפוך.

The standard specifies the following keying options for bundle (K_1, K_2, K_3)

1. Keying Option 1: K_1, K_2 and K_3 are independent keys;
2. Keying Option 2: K_1 and K_2 are independent keys and $K_3 = K_1$;
3. Keying Option 3: $K_1 = K_2 = K_3$.

3 אופנים לשימוש במפתחות באלגוריתם זה.

סיכום התהליך:

TDEA Encryption Operation:

$$I \rightarrow \boxed{\text{DES } E_{K1}} \rightarrow \boxed{\text{DES } D_{K2}} \rightarrow \boxed{\text{DES } E_{K3}} \rightarrow O$$

TDEA Decryption Operation:

$$I \rightarrow \boxed{\text{DES } D_{K3}} \rightarrow \boxed{\text{DES } E_{K2}} \rightarrow \boxed{\text{DES } D_{K1}} \rightarrow O$$

- איך לוקחים את ה 56 ביטים והופכים 16 מפתחות?

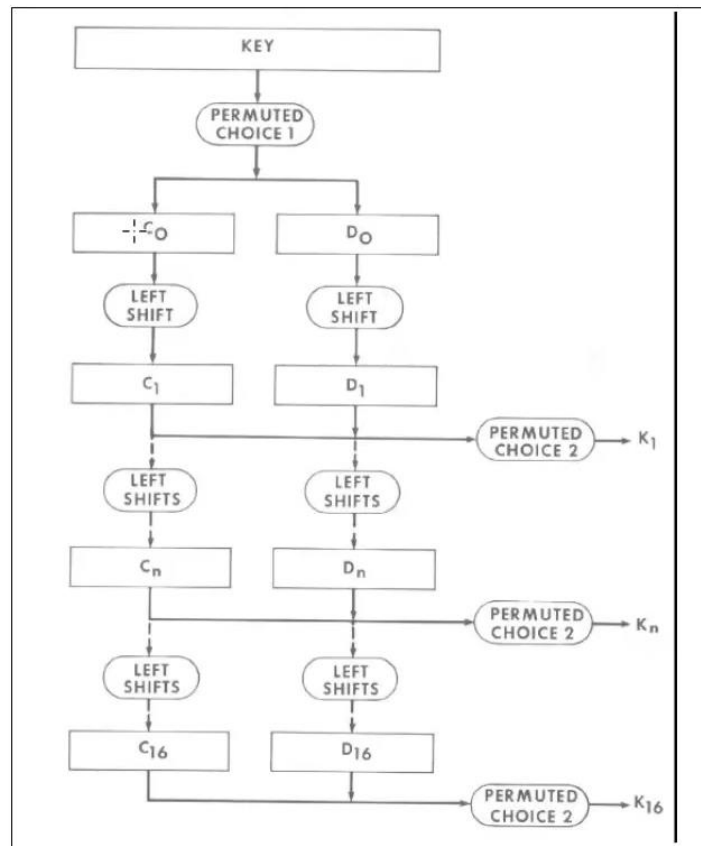


Figure 3. Key schedule calculation

PC-1

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

The table has been divided into two parts, with the first part determining how the bits of C_i are chosen, and the second part determining how the bits of D_i are chosen. The bits of KEY are numbered 1 through 64. The bits of C_i are respectively bits 57, 49, 41,..., 44 and 36 of KEY , with the bits of D_i being bits 63, 55, 47,..., 12 and 4 of KEY .

With C_i and D_i defined, we now define how the blocks C_n and D_n are obtained from the blocks C_{n-1} and D_{n-1} , respectively, for $n = 1, 2, \dots, 16$. That is accomplished by adhering to the following schedule of left shifts of the individual blocks:

:Permuted Choice 1

מחלקים את 56 ביטים ל2 וכל פעם מזיזים שמאלה.

Iteration Number	Number of Left Shifts
1	1
2	1
3	2
4	2
5	2
6	2
7	2
8	2
9	1
10	2
11	2
12	2
13	2
14	2
15	2
16	1

בכל סיבוב מזיזים מספר שונה של פעמים.

שאלה:

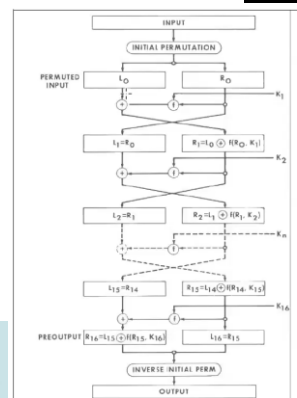


Figure 1. Encrypting computation

- Very fast operation – Single DES Encryption/Decryption takes only 9-clock cycles in fastest version

אלגוריתם

יש 16 סיבובים אז הייתי מצפה ל16 clk, כיצד כותבים שיש 9?

תשובה: בכניסה וביציאה יש פילפלופים והמעבר ביניהם הוא 1 clk , נשארו עוד 8 לשאר הלוגיקה ושמים רגיסטר אחרי כל 2 סיבובים וכך מקצרים את הזמן מ16 ל8 וגם לא מוותרים על הרגיסטרים.

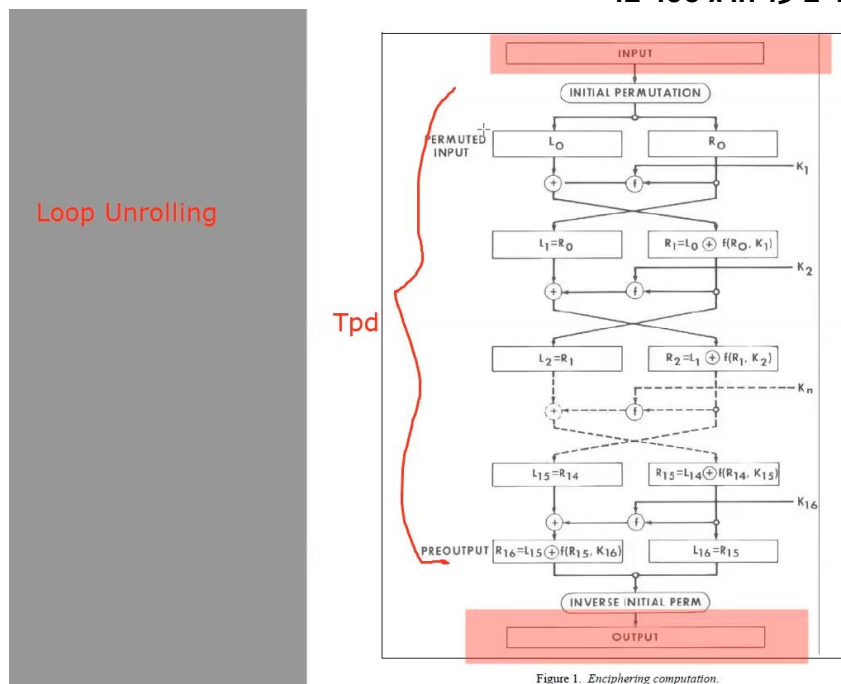


Figure 1. Enciphering computation.

כאן יש מסלול קריטי אחד עם TpD וזה 1 clk אחד פלוס.

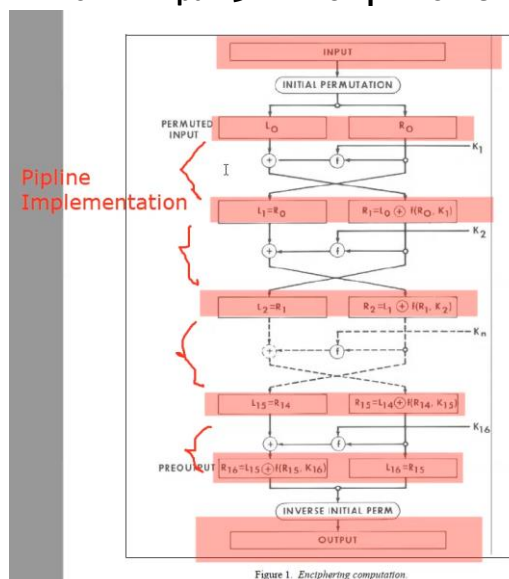


Figure 1. Enciphering computation.

כאן שמנו רגיסטר בכל סיבוב ויצרנו 16 מסלולים קריטיים ויש צורך ב16 clk .
וכשנשים רגיסטר בין 2 סיבובים נקבל 8 clk .
ויש עוד clk כשנעה load וכך נגיע ל9 clk -ים.

חישוב קצב העברת המידע:

שאלה 1: [25 נקודות]
ניתוח ביצועים, תכונות ומימשק של אלגוריתם

שאלה זו עוסקת בניתוח ביצועים, תכונות ומימשק עבור מימוש בחומרה של אלגוריתם ההצפנה AES

שים לב:

טופס השאלון כולל 2 עמודים מתוך דפי נתונים של חברת Mercora Technology עבור ליבת Verilog/VHDL של מעבד לביצוע אלגוריתם AES. עליך להשתמש בדפי נתונים אלו כדי להשיב על סעיפי השאלה הרלוונטיים

א. [10 נקודות] כתוב תיאור Verilog module הכולל את הכניסות והיציאות של הרכיב בהתאם למסמך של חברת Mercora Technology

ב. [10 נקודות] מצא בדפי הנתונים את הנתונים הנדרשים וחשב Throughput עבור התדרים הבאים, בשימוש במפתח של 128 ביט:

- 246 MHz
- 434 MHz
- 348 MHz

ג. [5 נקודות] הסבר מה תפקידם של הסיגנלים הבאים:

- KEYMODE_IN[1:0]
- KEYMODE_RDY
- KEYMODE[1:0]

תשובה:

ב. [10 נקודות] חישוב Throughput:

לפי דפי הנתונים, גודל הבלוק הוא 512 ביט ומשך העיבוד הוא 82 מחזורי שעון

- בתדר 246 MHz (4.065 ns):

$$Throughput = 128 \text{ bit} / (44 * 4.065 \text{ ns}) = 715 \text{ Mbit/sec}$$

- בתדר 434 MHz (2.304 ns):

$$Throughput = 128 \text{ bit} / (44 * 2.304 \text{ ns}) = 1.262 \text{ Gbit/sec}$$

- בתדר 348 MHz (2.874 ns):

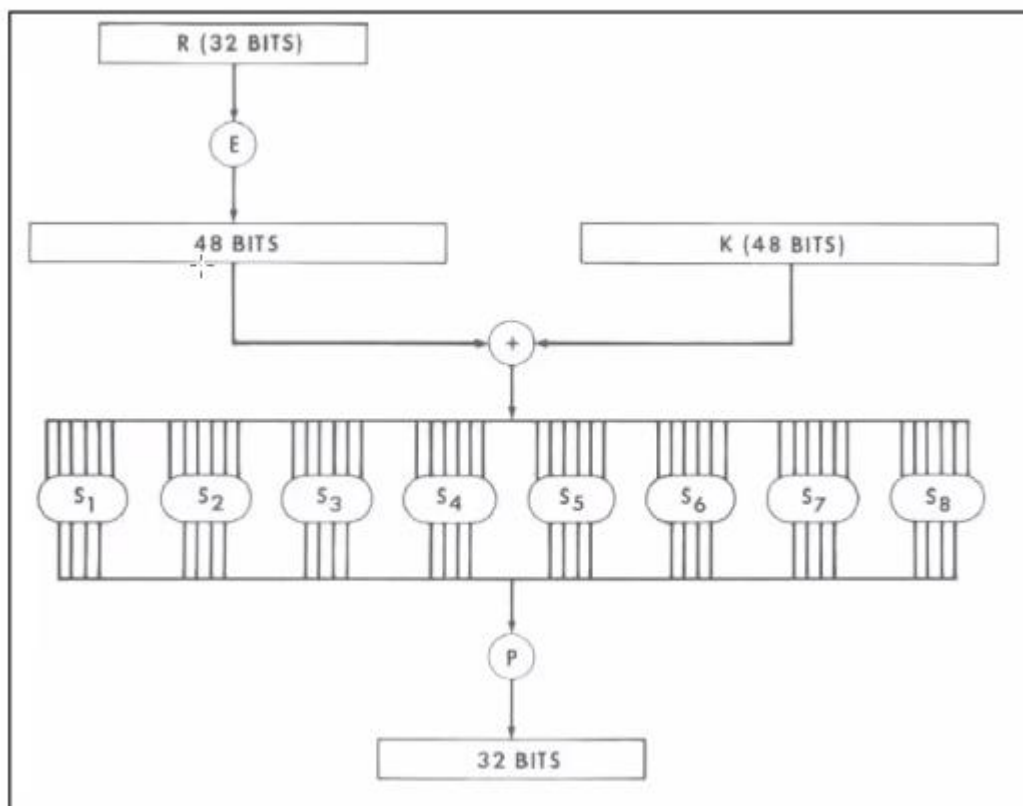
$$Throughput = 128 \text{ bit} / (44 * 2.874 \text{ ns}) = 1.012 \text{ Gbit/sec}$$

AES Standard IP Core (Encrypt Only)		
Xilinx Spartan 6-3	Xilinx Virtex 6-3	Xilinx Kintex 7-3
135 slices	132 slices	186 slices
0 blockram	0 blockram	0 blockram
246 MHz	434 MHz	348 MHz
715 Mbps	1262 Mbps	1012 Mbps

נראה עכשיו אלגוריתם הצפנה אמיתי-

DES:

אלגוריתם זה הומצא ע"י משרד המסחר האמריקאי. המפתח של האלגוריתם הוא של 64 ביט, שמתוכם 56 הם רנדומליים ומשתמשים בהם בהצפנה, וה-8 הנותרים הם חלק ממערכת שמשמשת לגילוי שגיאות. (כשמציפים גם את המפתח, אי אפשר לקחת את ה-8 האלה לשימוש ההוא). הוא משתמש ברשת פיסטל - בהתחלה הוא עושה פרמוטציה (אותה אחת שראינו כבר) עושה את האלגוריתם, ואז פרמוטציה הפוכה ומתקבל התוכן המוצפן.



הפונקציה E היא בעצם פרמוטציה כדלהלן:

E BIT-SELECTION TABLE

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

זו פונקציה לא הפיכה! כי לקחנו 32 ביטים והפכנו אותם ל48. כלומר לא חד חד ערכית, כי יש כמה ביטים שמקבלים את אותו הערך.
הSBOX הראשון מתוך 8:

S₁

Column Number

Row No.	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

כך הופכים 6 ביטים ל4, כלומר 48 ל32.
(הביט הראשון והאחרון מגדירים לי את השורה (בצד שמאל) וה4 האמצעיים את העמודה (למעלה)).

***[הערה – סדר במושגים:**

יש את האלגוריתם: DES.

הארכיטקטורה שלו בנויה לפי רשת פיסטל.

אופן ההפעלה (כלומר מה עושים כשיש כמה בלוקים להצפין) לזה יש כמה אפשרויות שראינו (CBC OFC וכו').]

*מכיוון שמשרד ממשלתי פיתח את ההצפנה הזאת, היה חשש שיש להם אפשרות להיכנס 'בדלת האחורית' לכל קוד מוצפן, אפילו ללא קוד. החשד העיקרי היה טבלת הSBOX (כי לא הביאו נוסחה אליה).
מאז כל אחד צריך להראות מאיפה הגיעו הערכים שהוא הכניס לSBOX. (ערכי סינוס למשל).

פעולות על המפתח – עושים לו פרמוטציה, ואז כל פעם SHIFT בהתאם לסבב, ואז פרמוטציה שונה נוספת ביציאה (צודק?):

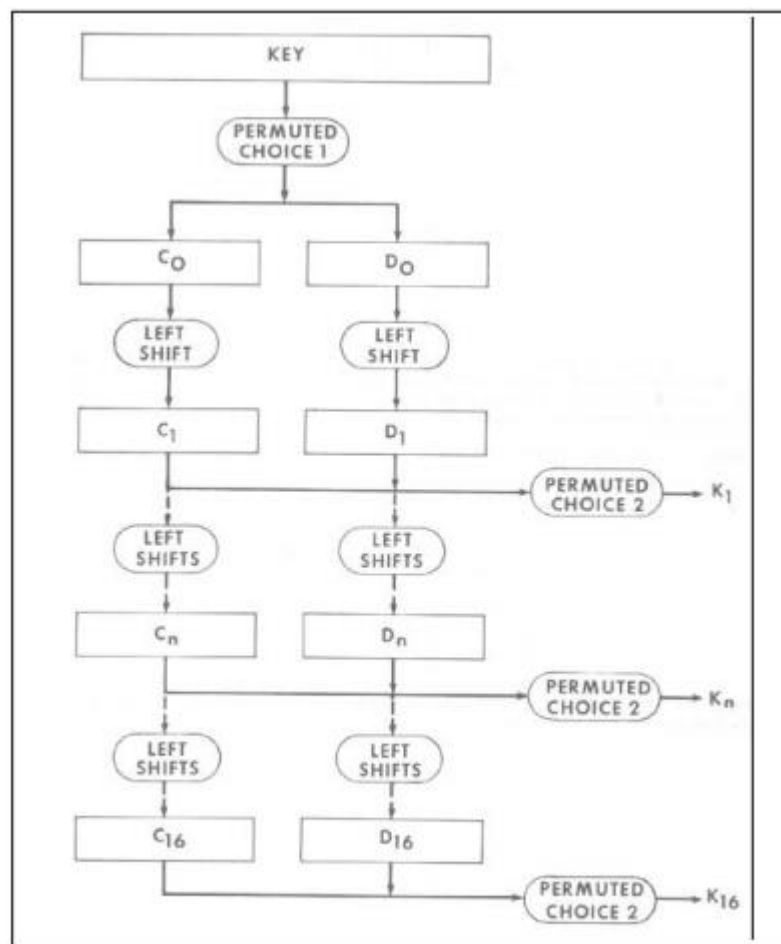


Figure 3. Key schedule calculation

כאשר:

PC-1

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

Iteration Number	Number of Left Shifts
1	1
2	1
3	2
4	2
5	2
6	2
7	2
8	2
9	1
10	2
11	2
12	2
13	2
14	2
15	2
16	1

ואז ביציאה פרמוטציה נוספת:

PC-2

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

מתישהו האלגוריתם נחלש. לכן עשו תקן חדש שעושה זאת שלוש פעמים בהתאם לסדר הבא:

TDEA Encryption Operation:

$$I \rightarrow \boxed{\text{DES } E_{K1}} \rightarrow \boxed{\text{DES } D_{K2}} \rightarrow \boxed{\text{DES } E_{K3}} \rightarrow O$$

TDEA Decryption Operation:

$$I \rightarrow \boxed{\text{DES } D_{K3}} \rightarrow \boxed{\text{DES } E_{K2}} \rightarrow \boxed{\text{DES } D_{K1}} \rightarrow O$$

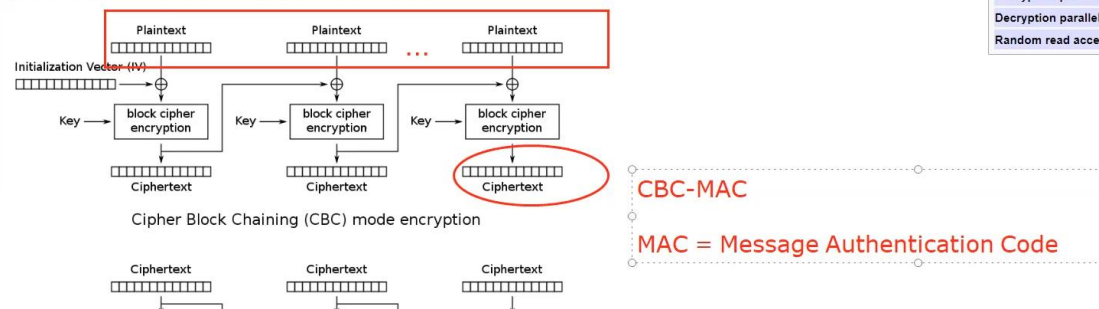
Throughput:

מדד לביצוע של אלגוריתמים. הוא בודק כמה ביטים עוברים מהכניסה ליציאה בשנייה.

חתימה דיגיטלית

Cipher block chaining (CBC) [edit]

Ehrsam, Meyer, Smith and Tuchman invented the cipher block chaining (CBC) mode of operation in 1976.^[23] In CBC mode, each block of plaintext is XORed with the previous ciphertext block before being encrypted. This way, each ciphertext block depends on all plaintext blocks processed up to that point. To make each message unique, an initialization vector must be used in the first block.



הבלוק האחרון זו תביעת אצבע שמייצגת את כל ההודעה.

מה זה חתימה דיגיטלית?

תנאי פונקציית HASH

אלגוריתמים בקריפטוגרפיה ומימוש ב-Verilog (מספר קורס – 130330)
שם המרצה: אורי שטרן

מתמטיות לפונקציית HASH

1. תנאי Preimage

- נתון פלט H_0
- מציאת הודעה $M_0 = \{m_1, m_2, \dots\}$ הגורמת לפלט H_0 קשה מבחינה חישובית

2. תנאי Second Preimage

- נתונה הודעה $M_1 = \{m_1, m_2, \dots\}$
- מציאת הודעה אחרת M_2 המובילה לפלט זהה ל- $H(M_1)$ קשה מבחינה חישובית

3. תנאי Collision

- מציאת שני קלטים שונים M_0 ו- M_1 המובילים לאותו פלט H , קשה מבחינה חישובית

M_0 זה שיחזור ההודעה אחורה.

בפונקציית HASH ניתן לשחזר את ההודעה מהפלט.

פונקציית חתימה שאיננה מקיימת את דרישות פונקציית HASH (ולכן איננה מוגדרת כ-HASH)

CBC-MAC

כאשר מבצעים הצפנה במוד CBC מתבצע למעשה "סיכום" של ההודעה באמצעות ביצוע XOR בין תוצאת הצפנה (E) של כל בלוק. תוצאת ההצפנה של הבלוק האחרון היא החתימה הדיגיטלית של כל ההודעה

תנאי ה-HASH על ידי MAC-CBC

סתירת תנאי Preimage ב-CBC-MAC:

באמצעות שימוש בפונקציית הפענוח ($D=E^{-1}$) ניתן למצוא לפחות הודעה אחת M המקיימת:

$$M = \{mx\} = D(H0, K) \oplus IV$$

1.

דיקריפט של mx עם המפתח ואח"כ xor יתן לנו לפחות שחזור אחד. ולכן CBC-MAC לא יכול להיות HASH כי לא מקיים את התנאי הראשון.

תנאי ה-HASH על ידי MAC-CBC

סתירת תנאי Second Preimage ב-CBC-MAC:

מכיוון שנתונה M1, נחשב $H1$, נחשב $H(M1)$

עתה נייצר הודעה חדשה: $M2 = \{M1, my\} = \{m1, m2 \dots mx, my\}$

ניתן לחשב את my באמצעות שימוש בפונקציית הפענוח ($D=E^{-1}$):

$$m_y = D(H1, K) \oplus H1$$

כלומר, מצאנו הודעה $M2 \neq M1$ המייצרת את אותו הפלט של M1

2.

תנאי 2 לא מתקיים.

תנאי ה-HASH על ידי MAC-CBC

סתירת תנאי Collision ב-CBC-MAC:

ניקח הודעה אקראית: $M1 = \{m1, m2, mx\}$ ונחשב עבודה את $H1 = H(M1)$

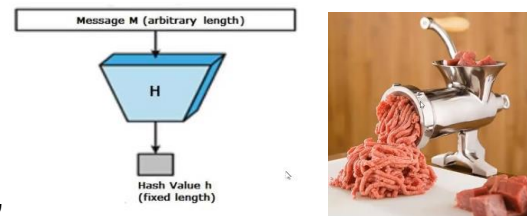
בהתאם לחישוב שביצענו בסתירת תנאי ה-Second Preimage $H(M2) = H(M1) = H$ שעבודה M2, הודעה אחרת M2, שבעודה $H(M2) = H(M1) = H$

3.

תנאי 3 לא מתקיים.

CBC ניתן לחזור אחורה ולכן זה לא פונקציית HASH.

אז מה זה פונקציית HASH?



אפקט מטחנת הבשר – הבשר שהכנסנו יוצא ויש לכל בשר את החתימה שלו, את איך שהוא נראה, ויוצא בגודל פתחי המטחנה.

Input		Digest
Fox	cryptographic hash function	DFCD 3454 BBEA 788A 751A 696C 24D9 7009 CA99 2D17
The red fox jumps over the blue dog	cryptographic hash function	0086 46BB FB7D CBE2 823C ACC7 6CD1 90B1 EE6E 3ABC
The red fox jumps over the blue dog	cryptographic hash function	8FD8 7558 7851 4F32 D1C6 76B1 79A9 0DA4 AEFE 4819
The red fox jumps oevr the blue dog	cryptographic hash function	FCD3 7FDB 5AF2 C6FF 915F D401 C0A9 7D9A 46AF FB45
The red fox jumps oer the blue dog	cryptographic hash function	8ACA D682 D588 4C75 4BF4 1799 7D88 BCF8 92B9 6A6C

כאן יש את אפקט המפולת – שינוי קטן בהתחלה (בכניסה) יביא לשינוי גדול ביציאה.

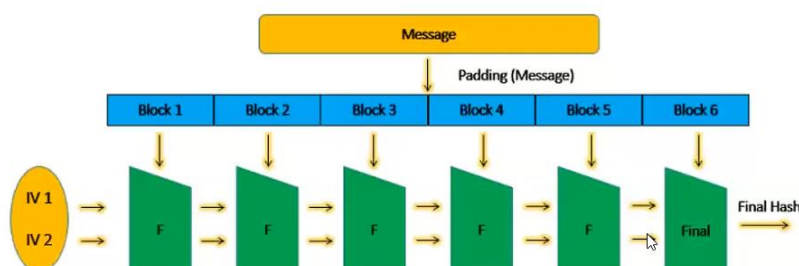
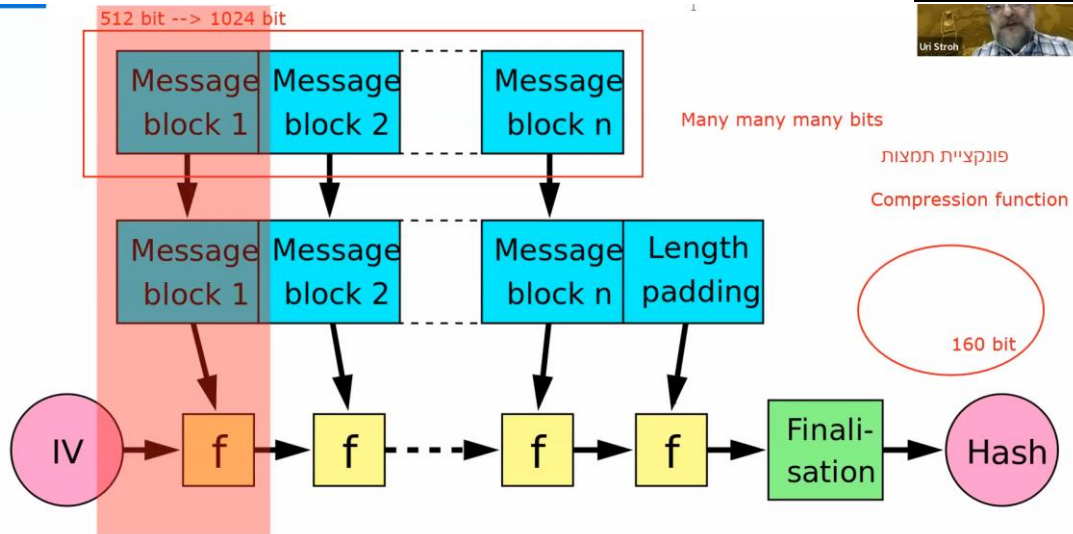


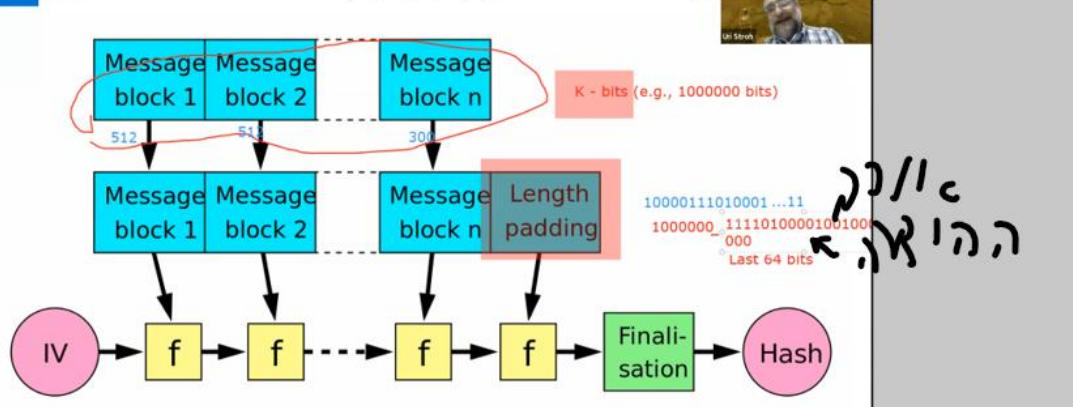
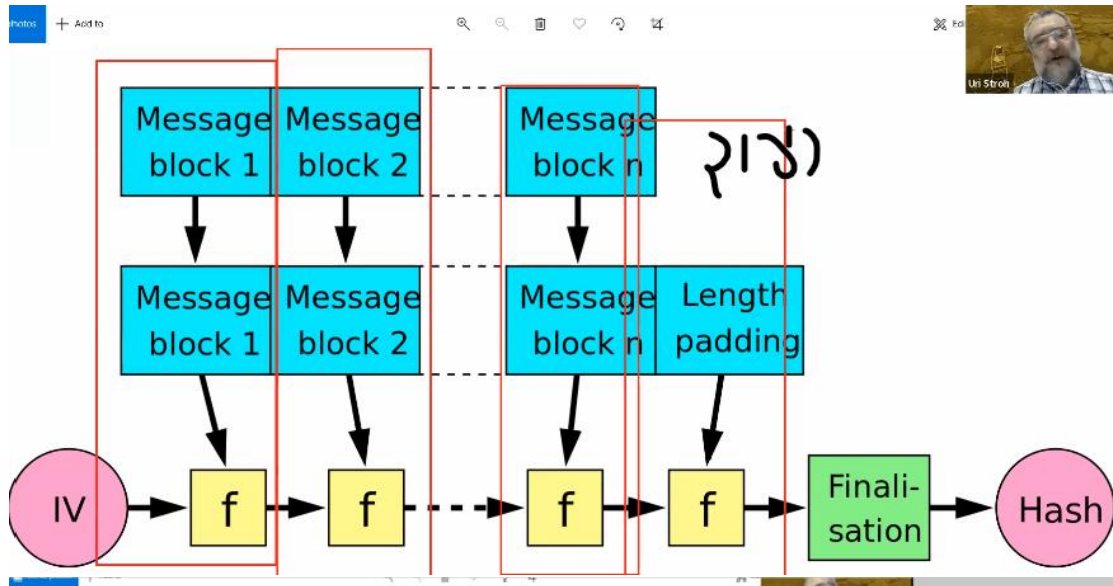
Image: The Avalanche Effect (the butterfly effect)

אפקט המפולת:

סכמה כללית:



שירשור יציאות HASH. נקבל ביציאה תמיד 160 bit (לדוגמה).



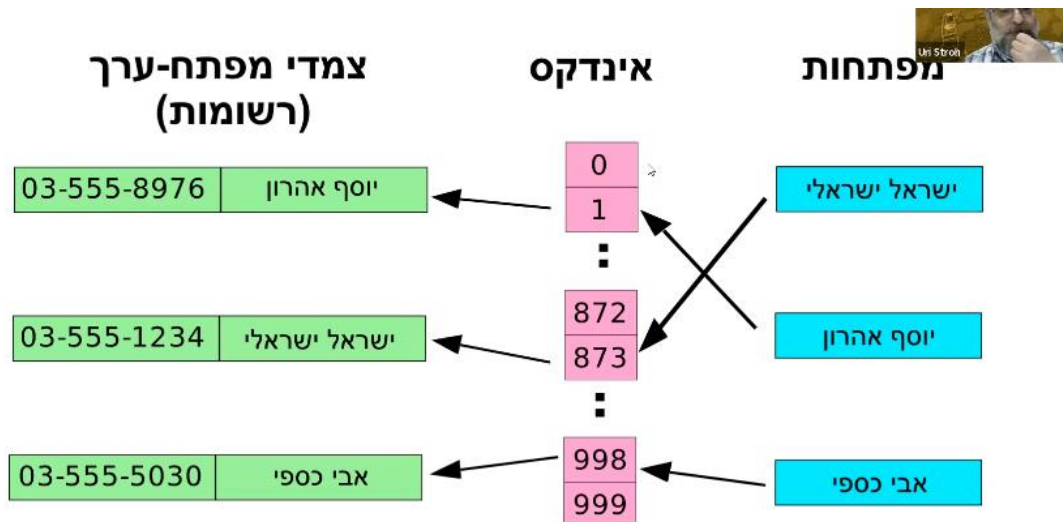
שימושים של HASH:

1. חתימה להודעה.
2. מניעת שיחזור המידע – כשמים HASH של סיסמה יחד עם התחלתה א"א אפשר לשחזר את הסיסמה. זו הגנה, כגון בכספומט. ולכן כששומרים את שם המשתמש והסיסמה לא יצליחו לשחזר מהכספומט את הסיסמה.
3. אימות הורדת קובץ:

קוד MD5:

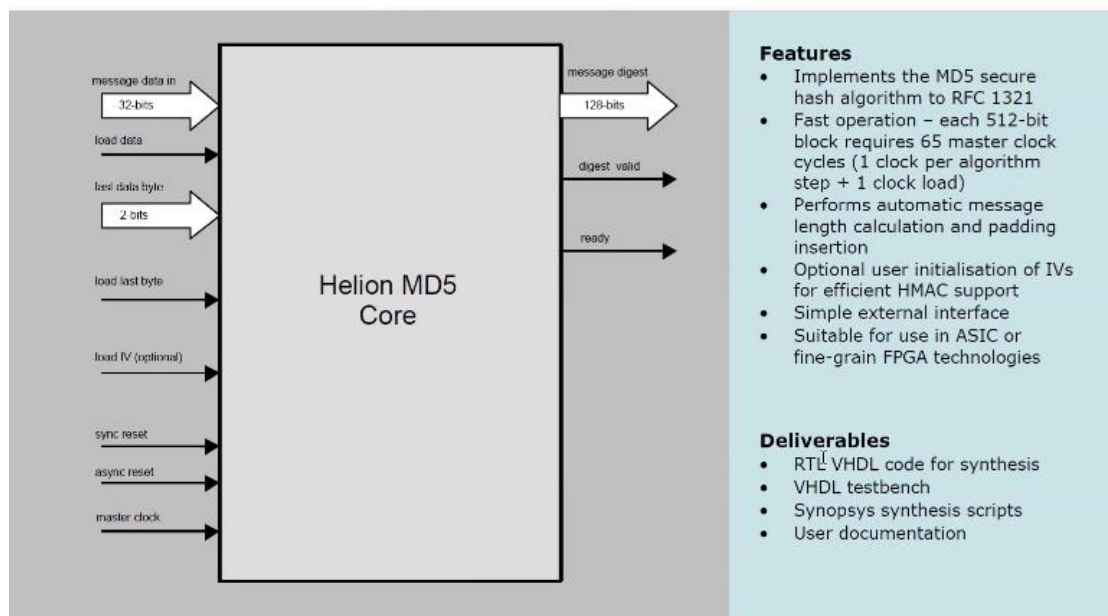
Xilinx Unified Installer 2022.1 SFD (TAR/GZIP - 73.81 GB)
MD5 SUM Value: 0bf810cf5eab28a849ab52b9bfdd20a5

4. כאשר לא תקבל את הערך בפונקציית HASH הזאת, ז"א שהקובץ לא ירד בשלמותו. טבלאות HASH:



עבור מספרי טלפון של אנשים.

אלגוריתם MD5:



מדדי ביצועים:

technology	0.18um CMOS ASIC
typical core gate count	16k gates
max master clock	145 MHz
max data rate	1140 Mbps

Network Working Group
Request for Comments: 1321

R. Rivest
MIT Laboratory for Computer Science
and RSA Data Security, Inc.
April 1992

The MD5 **Message-Digest** Algorithm

Status of this Memo

...

3. MD5 Algorithm Description

We begin by supposing that we have a b -bit message as input, and that we wish to find its message digest. Here b is an arbitrary nonnegative integer; b may be zero, it need not be a multiple of eight, and it may be arbitrarily large. We imagine the bits of the message written down as follows:

$m_0 m_1 \dots m_{b-1}$

The following five steps are performed to compute the message digest of the message.

3.1 Step 1. Append Padding Bits

The message is "padded" (extended) so that its length (in bits) is congruent to 448, modulo 512. That is, the message is extended so

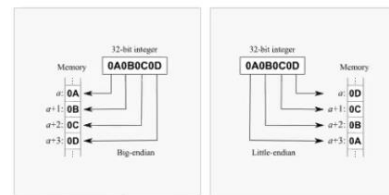
Big Endian
Little Endian

"Endianess"

321

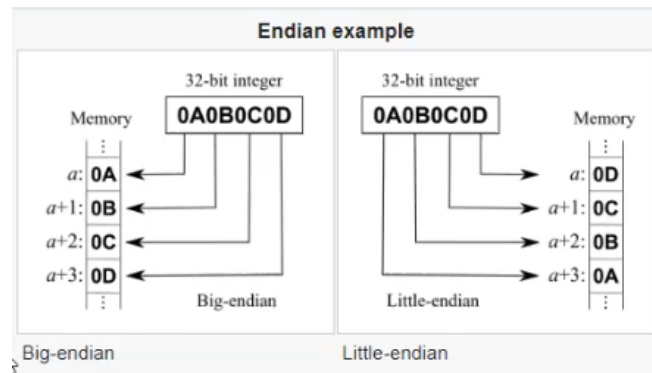
MD5 Message-Digest Algorithm

word A: 01 23 45 67
word B: 89 ab cd ef
word C: fe dc ba 98
word D: 76 54 32 10



סדר בתים גדול: בכתובת
הקטנה ביותר יישמר 0x0A

סדר בתים קטן: בכתובת
הקטנה ביותר יישמר 0x0D



סדר הביטים יכול להפיל את התוכנית ולכן יש לשים לב לסדר הביטים, מהקטן לגדול או להפך.

פעולות באלגוריתם:

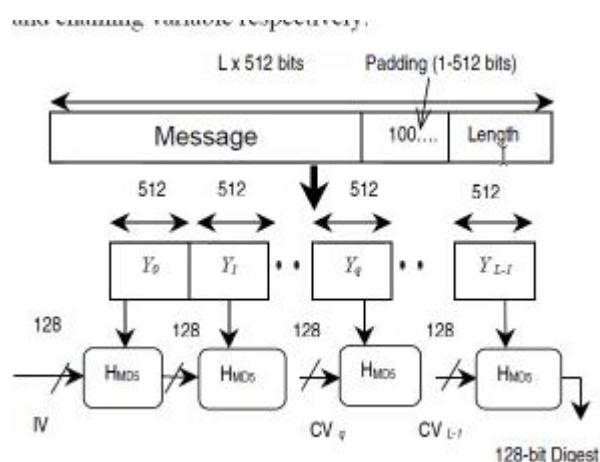
```
a = b + ((a + F(b,c,d) + X[85] + T[1]) <<< 8) . */
/* Do the following 16 operations. */
[ABCD 0 7 1] [DABC 1 12 2] [CDAB 2 17 3] [BCDA 3 22 4]
[ABCD 4 7 5] [DABC 5 12 6] [CDAB 6 17 7] [BCDA 7 22 8]
[ABCD 8 7 9] [DABC 9 12 10] [CDAB 10 17 11] [BCDA 11 22 12]
[ABCD 12 7 13] [DABC 13 12 14] [CDAB 14 17 15] [BCDA 15 22 16]

/* Round 2. */

of the four register:
was started.) */
A = A + AA
B = B + BB
C = C + CC
D = D + DD
end /* of loop on i */
```

A, B, C, D הם חתימה דיגיטלית.

מימוש נוסף של האלגוריתם:



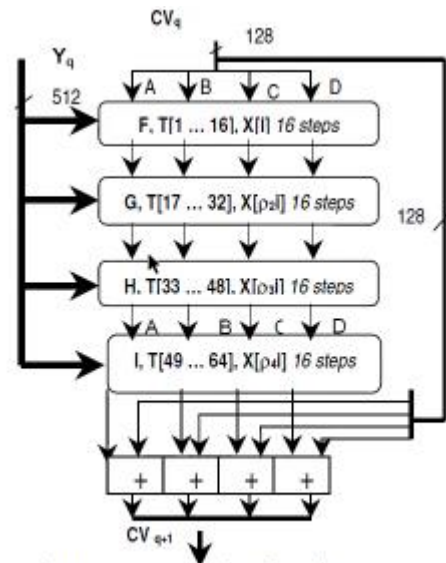


Figure 2. Compression function H_{MD5}

Shift amounts (S) are used each round and are different from round to round. Each step is of the following form [1]:

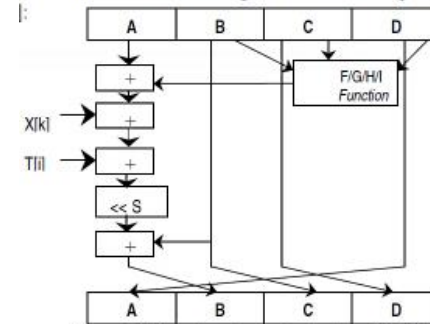


Figure 3. Operations in a single step of MD5

מימוש האלגוריתם בחומרה:

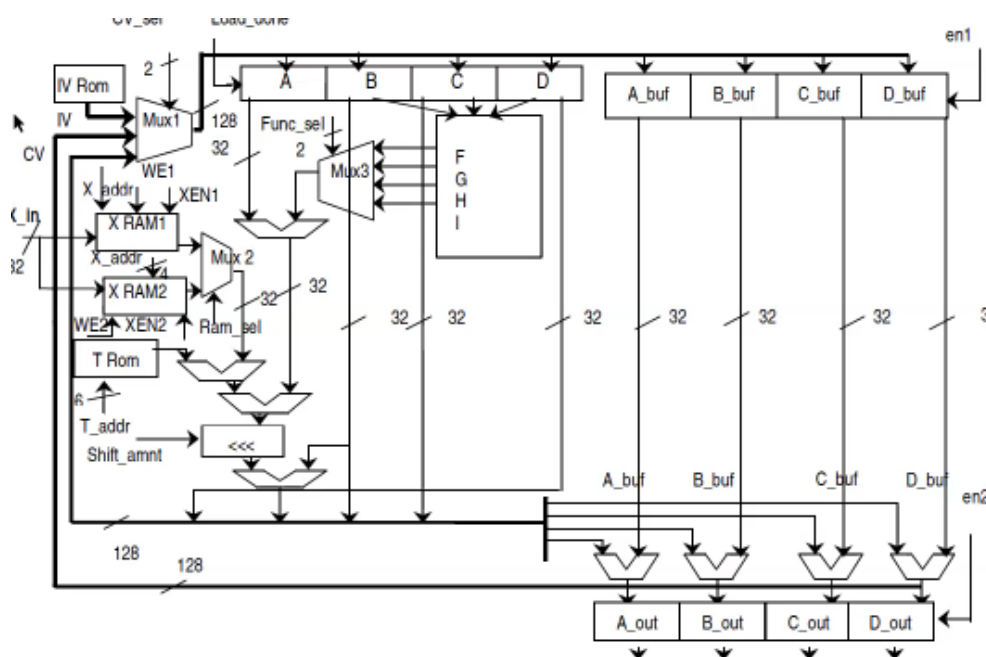


Figure 6. MD5 iterative core

מימוש על FPGA:

diagram of the iterative design is shown in Figure 5.

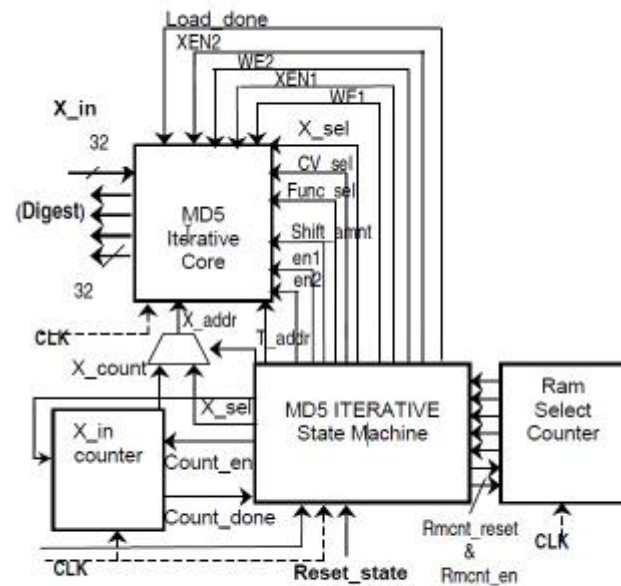
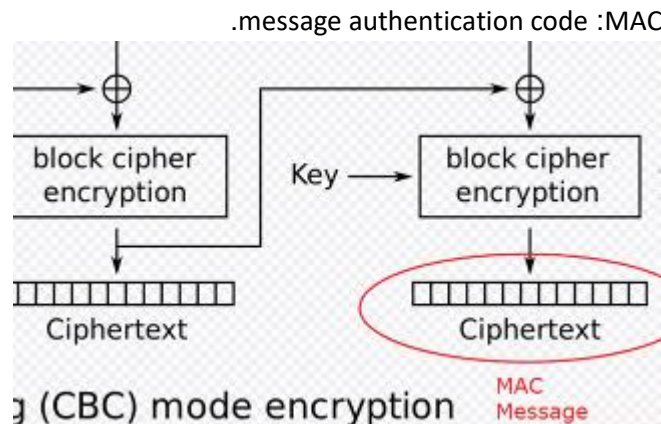


Figure 5. Block diagram of MD5 iterative design (Iterate_MD5).

CV_sel = Load done



כלומר הבלוק האחרון הוא סוג של חתימה של ההודעה, כי הוא "מסכם" את ההצפנה של כל הבלוק. בעזרתו ניתן לראות האם הפעולות התבצעו כראוי.

HASH: נניח יש לי פונקציה מתמטית שנותנת את הערך בסוף של החתימה.
כדי שתיחשב חזקה – HASH – יש לה 3 תנאים שמגדירים את החוזק שלה (ביחסי פלט-קלט):

דרישות מתמטיות לפונקציית HASH

1. תנאי Preimage

- נתון פלט H_0
- מציאת הודעה $M_0 = \{m_1, m_2, \dots, m_x\}$ הגורמת לפלט H_0 קשה מבחינה חישובית

2. תנאי Second Preimage

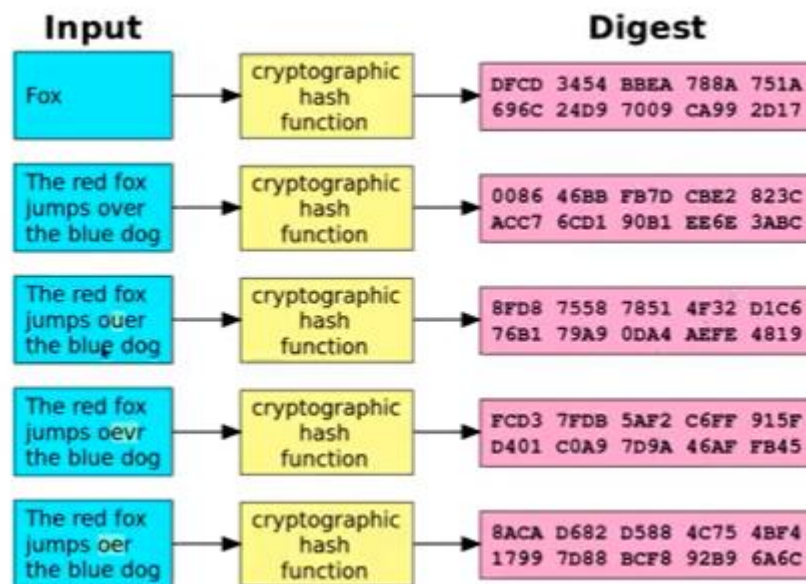
- נתונה הודעה $M_1 = \{m_1, m_2, \dots, m_x\}$
- מציאת הודעה אחרת M_2 המובילה לפלט זהה ל- $H(M_1)$ קשה מבחינה חישובית

3. תנאי Collision

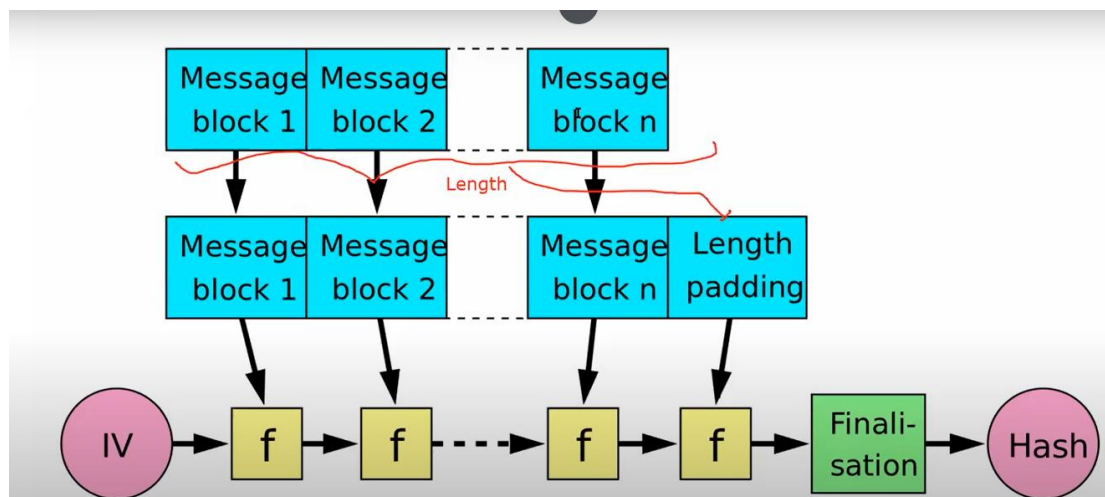
- מציאת שני קלטים שונים M_0 ו- M_1 המובילים לאותו פלט H , קשה מבחינה חישובית

(הקשיים המוזכרים כאן הם מבחינה אלגוריתמית, כלומר נניח שמצאתי את המפתח בדרך כלשהי).
הצפנה רגילה של XOR היא לא חזקה מספיק, כלומר היא לא מקיימת את התנאים.

עקרון מטחנת הבשר: לא משנה איזה גודל של בלוק הכנסתי, יוצא גודל קבוע.
כאן רואים ששינוי אפילו של אות אחת יגרור שינוי ענק (מפולת), אך אורך היציאה הוא קבוע ללא קשר לגודל הכניסה:



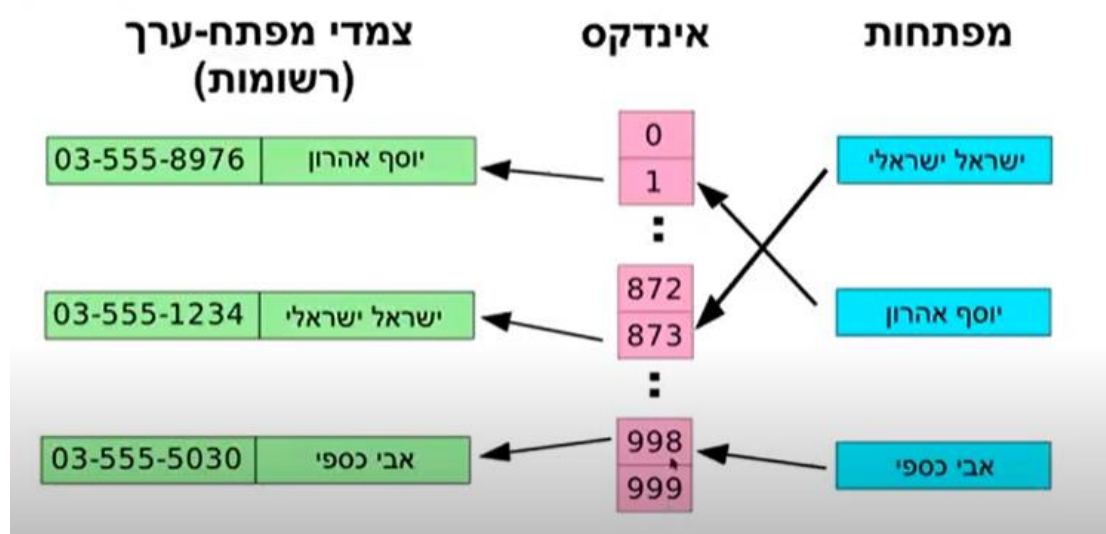
כיצד זה מתבצע?



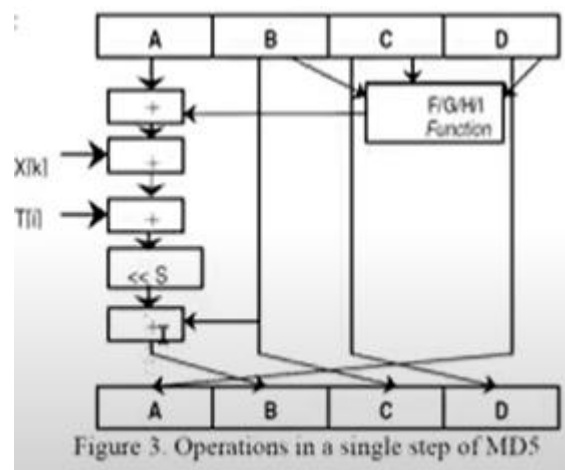
רואים שה"בלוק" האחרון שמוצפן הוא בעצם פונקציה של גודל המידע שנכנס. כלומר עבור כל גודל של מידע נקבל HASH שונה, גם אם לכאורה הם דומים. אך בסוף גודל ה HASH יהיה קבוע.

שימושים לפונקציית HASH:

1. חתימה דיגיטלית: נניח אני רוצה לוודא שקובץ שהורדתי מהאינטרנט ירד כמו שצריך. או דוגמא יותר מצוה – אני עושה LOGIN למחשב, כלומר אני מכניס שם משתמש וסיסמא. איך המחשב מוודא שזה תואם?
אם הוא ישמור טבלה עם שמות וסיסמאות, זה יהיה בעיה אם זה ייגנב. אך אם נשמור רק את הHASH של הסיסמא (שזה הרי לא מכיל מידע, אלא חתימה באורך קבוע, כלומר תוצאה של פונקציה שפועלת על בלוק או על ההצפנה שלו) אז גם זה ייגנב, לא יהיה ניתן לשחזר מתוכו את הסיסמא (כמו שראינו בהגדרות לפונקציית HASH מקודם).
2. HASHTABLE: מקודדים את השמות כך שכל שם יקבל אינדקס אחר (כלומר זה בכלל לא עניין של הצפנה) שבו יופיע הטלפון שלו:



אלגוריתם MD5: אלגוריתם ותיק שכבר נפרץ. עדיין משתמשים בו לצרכים מסוימים, שלא כוללים אבטחת מידע (למשל אימות הורדה של קובץ), למשל עבור HASH. מבנה כללי:



ובפירוט (=):

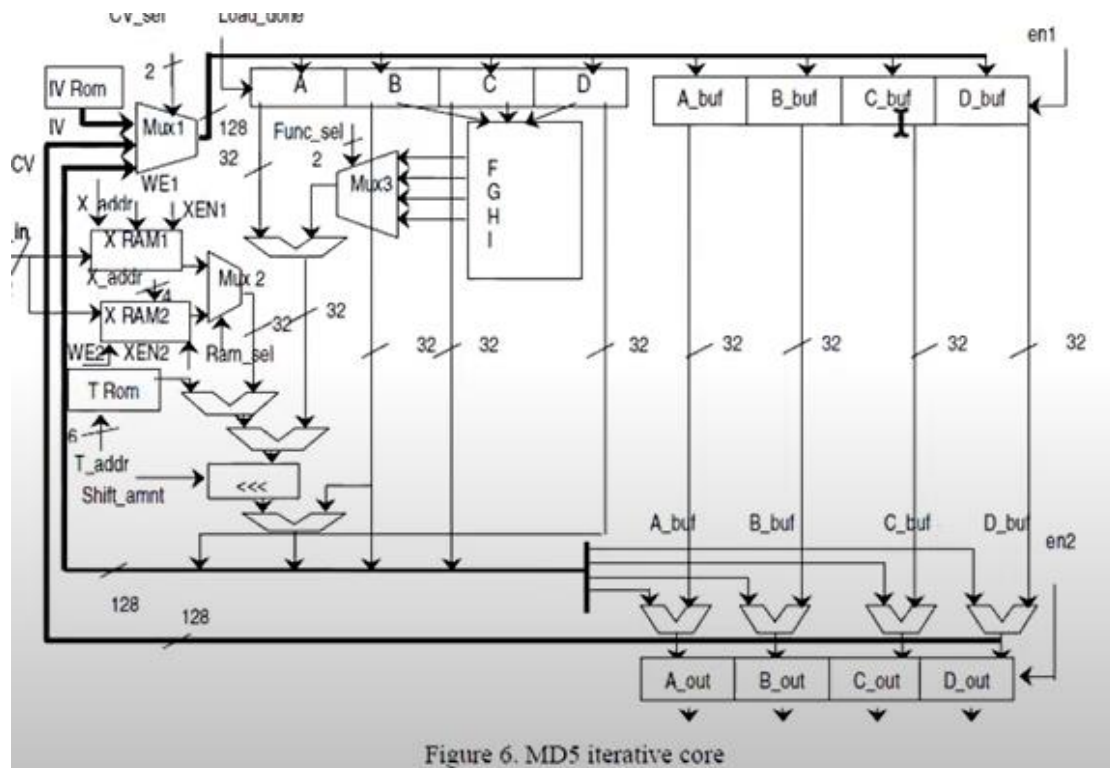


Figure 6. MD5 iterative core

***שוב-** HASH לא יעיל עבור הצפנות כי הוא לא הפיך.

HMAC: שילוב של HASH כשמשמשים במפתחות. כלומר אי אפשר לנבא את החתימה ללא ידיעת המפתח.

נקודות להבנה:

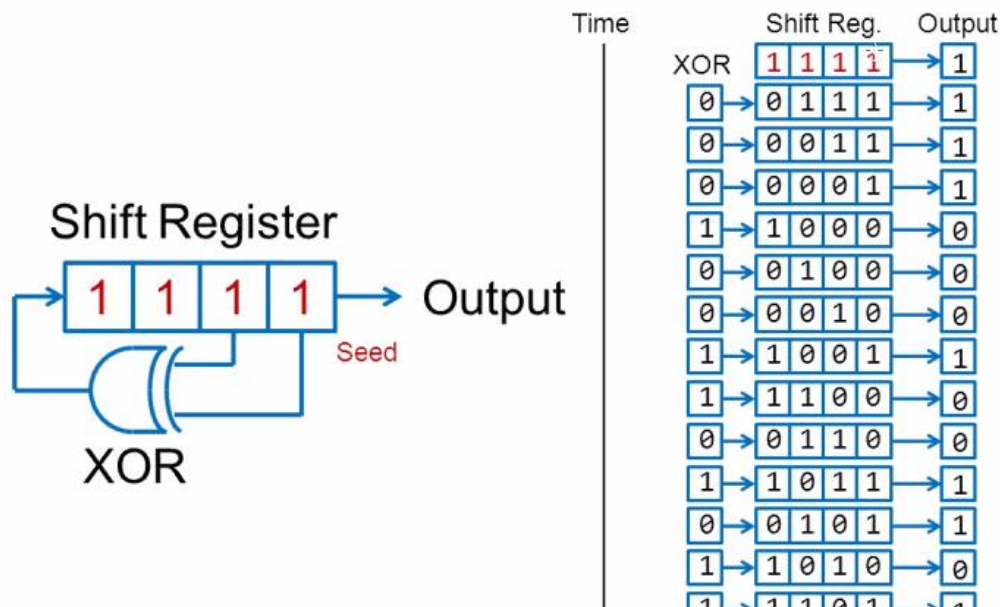
1. מה העניין של ידיעה ב-HASH? כלומר אם אני יודע את האלגוריתם אז אני אמור לפענח?.
2. איך מפענחים חזרה את DES אם ה-SBOX הוא לא חד חד ערכי?

שיעור 11

רנדומליות:

ראינו את החשיבות של ערכים רנדומליים בהצפנה. למשל כשהאלגוריתם זה רק XOR עם מפתח כלשהו, חייב שהוא יהיה רנדומלי. גם כשעושים בדיקות לרכיב, נרצה ערכים כאלה. מאיפה נשיג ערכים כאלה? אפשר לקחת מרעש אקראי. נדגום אותו ונהפוך אותו למספר בינארי. עכשיו נראה דוגמא איך עושים דבר כזה:

LFSR:



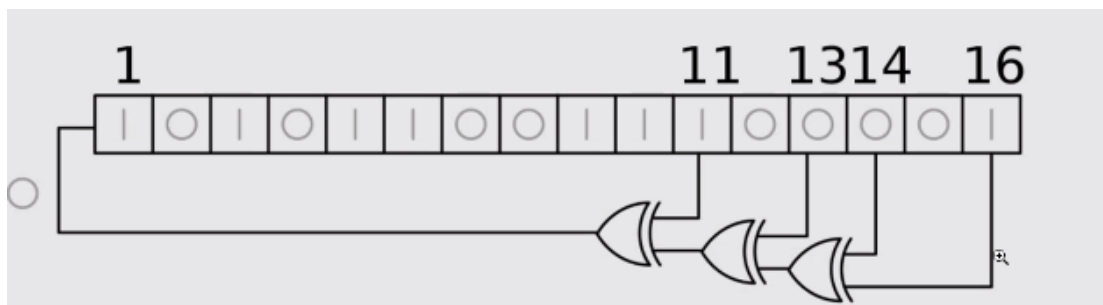
לפי ההגדרה של האלגוריתם הזה, המבנה הזה נותן לי פולינום x^4+x^3+1 . עבור וקטור של 4 ביטים, אחרי 2^4-1 פחות אחד, סיבובים, נקבל את הערך ההתחלתי בחזרה. החזקות מייצגות לי את המיקום של הביטים שנכנסים למשוב, ופלוס 1 זה תמיד. (הביטים פה מיוצגים מ-1-4).
דוגמא לפולינומים נוספים:

# bits	Polynomial	Length
L		2^L-1
4	x^4+x^3+1	15
5	x^5+x^3+1	31
6	x^6+x^5+1	63
7	x^7+x^6+1	127
8	$x^8+x^6+x^5+x^4+1$	255
9	x^9+x^5+1	511
10	$x^{10}+x^7+1$	1023
11	$x^{11}+x^9+1$	2047
12	$x^{12}+x^{11}+x^{10}+x^4+1$	4095
13	$x^{13}+x^{12}+x^{11}+x^8+1$	8191
14	$x^{14}+x^{13}+x^{12}+x^2+1$	16383
15	$x^{15}+x^{14}+1$	32767
16	$x^{16}+x^{14}+x^{13}+x^{11}+1$	65535
17	$x^{17}+x^{14}+1$	131071
18	$x^{18}+x^{11}+1$	262143
19	$x^{19}+x^{18}+x^{17}+x^{14}+1$	524287

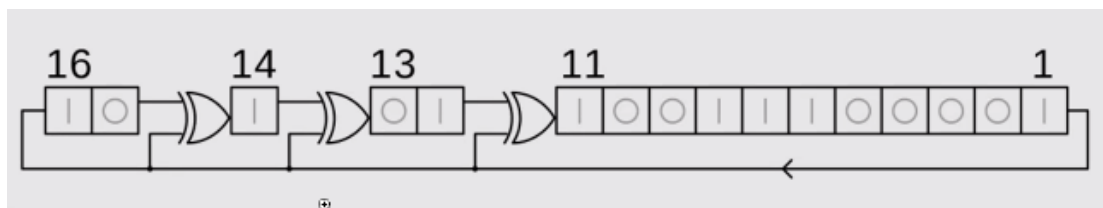
מכיוון שזה צפוי מראש, אז נשתמש בפולינומים ארוכים. כלומר מצד אחד קשה לשחזר אותו מבחון, אך מצד שני אני יודע לצפות מראש מה הערכים וכך אפשר להשתמש בזה לבדיקות הרכיב וכו'. כלומר זה חצי רנדום.

סיכום: זה נותן לי ערכים חצי רנדומליים, דהם טובים לבדיקות.
*יש מקומות כאלה שמתחילים מביט 0, אך בעיקרון ברירת מחדל מתחילה מ1.

LFSR פיבונאצ'י (many to one):



LFSR גאלוזה (one to many):



כומר שניהם עם אותה משוואה, אך זה בפועל אלגוריתם שונה, כי ה XOR נכנס למקום אחר.

כל זה למה?

שנדע לשים לב כשמשתמשים בכזה דבר בהצפנה, שנשים לב באיזה אלגוריתם מדובר, כי יש כמה סוגים לכל פולינום!

הצפנה א-סימטרית

אלגוריתם RSA:

פותח ע"י פרופסורים מMIT ו NDS. שם יש מפתח אחד עבור ההצפנה (שהוא ניתן לפרסום) ומפתח אחר עבור הפענוח. לא נכנס לכל המתמטיקה, אבל:

שלבי ביצוע אלגוריתם RSA - שימוש

סימונים:

$PU = \{e, n\}$

$PR = \{d, n\}$

ק, q - סודיים

n - פומבי

φ - סודי

e - פומבי

d - סודי

נוסחאות:

- בוחרים שני מספרים ראשוניים גדולים p ו-q
- מחשבים $n = p * q$
- מחשבים פונקציית אוילר $\Phi(n) = (p - 1)(q - 1)$
- בוחרים e שלם כך ש- $1 < e < \Phi$
- מחשבים d המקיים $d * e = 1 \mod \Phi$

(לפי אלגוריתם אוקלידס המורחב ומשפט השארית הסיני)

נוסחאות:

$$C = M^e \mod n$$

$$P = C^d \mod n$$

M

Q P = מדובר בסדר גודל של מספרים באורך של 1024 ביטים!

מימוש של חזקה: זה דבר שלוקח המון מקום וזמן. מכפל של שני מספרים של 1024 ביט זה דבר שאין בדר"כ. מה יש? 32 ביט. אז ניקח את הוקטורים ונפרק אותם ל32 חלקים של 32 ביט, כמו שעושים בכל ארוך:

```

But....
wire [1023:0] a, b;
wire [2047:0] y;

assign y = a * b; // Not good!!! 1024 * 1024 is a huge multiplier

```

x

a (1024 bit)	32b	32b	...	32b
--------------	-----	-----	-----	-----

==> How many digits of 32 bits we have? --> N=32

b (1024 bit)	32b	32b	...	32b
--------------	-----	-----	-----	-----

So a*b in multiplying digits (as in the known algorithm) $N^2 = 32^2 = 1024$ multiplications (usage of the multiplier)

We have exponent "e" of 1024 bits = how many multiplication do we need to do?

Base "A" is 1024 bit

if e is All zeros: 1024 squarings ==> 1024 multiplication of the 1024 bit base)

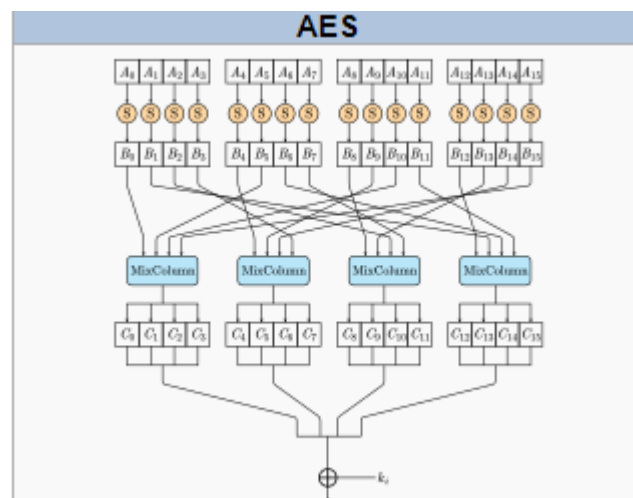
if e is All ones: 1024 squarings + 1024 multiplications --> 2048 multiplications (before we break down how many times we use the multiplier)

if e is Random: 50% 0-s & 50% 1-s --> $1.5 * 1024$ multiplications (of the 1024 bits)

שיעור 12

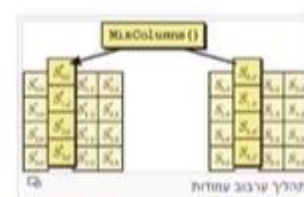
אלגוריתם AES:

אלגוריתם חדש יותר וחזק יותר מDES. הוא המקובל היום בתעשייה.



בנוי לפי ארכיטקטורה של SB NETWORK.

[*כל 16 בתים מיוצגים בתור קובייה לצורך נוחות ההסבר.]



שלב ראשון – SBOX.

שלב שני – הזזות.

שלב שלישי – הכפלת עמודות בפולינום כלשהו.

שלב רביעי – XOR עם המפתח.